



COMPLETE STUDY NOTES

Lectures 12 & 17-22 · Day 17-22 code

JavaScript Explained Simply

From array methods to the `this` keyword — every concept taught step by step, with diagrams, real outputs, and comparisons to the C++ you already know.

```
first.js

const student = { knows: "C++", learning: "JavaScript" };

async function study(notes) {
  for (const chapter of notes) {
    await chapter.read(); // slowly, with examples
  }
  return "I understand JavaScript!";
}
```

Array Methods

Set & Map

Event Loop

Callbacks

Callback Hell

Promises

JSON

Async / Await

Prototypes

Classes

Strict Mode

this

call · apply · bind

Arrow Functions

How to Use These Notes

Hello! 🙌 These notes are written for **you** — a student who is comfortable with **C++** and knows a *little bit* of JavaScript. I will teach every idea slowly, in simple words, with small examples, pictures, and the exact output you will see on your screen. Whenever a JavaScript idea is similar to something in C++, I will show you the C++ version side by side, so your brain can connect the new thing to something it already knows.

You do **not** need to read everything in one go. Read one chapter, run the code yourself, answer the quiz at the end, and only then move to the next chapter.

What is inside

#	Chapter	From lecture	Code from GitHub
0	JavaScript warm-up for a C++ brain	basics used everywhere	—
1	Array superpowers: <code>forEach</code> , <code>map</code> , <code>filter</code> , <code>reduce</code> , <code>find</code> , <code>some</code> , <code>every</code>	Lecture 12	—
2	<code>Set</code> and <code>Map</code>	Lecture 12	—
3	The Event Loop — how JavaScript runs your code	Lecture 17	Day 17
4	Callbacks and Callback Hell	Lecture 18	Day 18
5	Promises	Lecture 19	Day 19
6	JSON vs JavaScript objects	Lecture 19	Day 19
7	<code>async</code> / <code>await</code>	Lecture 20	Day 20
8	Prototypes and Classes	Lecture 21	Day 21
9	Strict mode	Lecture 22	Day 22
10	The <code>this</code> keyword, <code>call</code> / <code>apply</code> / <code>bind</code> , arrow functions	Lecture 22	Day 22
11	Final revision: everything on a few pages	all	all

How every chapter is organised

1. **Opener** — a dark box that tells you what you will learn.
2. **Explanation in small steps** — first the *idea*, then the *syntax*, then *examples with output*.
3. **Diagrams** — whenever a picture makes it easier (event loop, promise states, prototype chain...).
4. **Code from GitHub (Day 17 – Day 22)** — every experiment from the course files, explained line by line, with the output.
5. **Things to Remember** — the points you must not forget (perfect for revision before an exam or interview).

6. **Quick Quiz + Answers** — test yourself. Try first, then check the answer box.

The coloured boxes you will see

C++ Corner

Compares the JavaScript idea with C++ (STL, pointers, threads, classes...).

Things to Remember

The most important points. Read these again before exams.

Common Mistake

A bug that almost every beginner makes — and how to avoid it.

Real-Life Analogy

A real-world story (Zomato order, token at a food counter...) that makes the idea obvious.

Deep Dive (optional)

Extra detail for the curious. Safe to skip on the first reading.

Code from GitHub

Explains the code from the course repository (Day 17 – Day 22).

How to run the code yourself

There are three easy ways. Use whichever is convenient:

Way	How	Good for
Browser console	Open any web page → press <code>F12</code> (or right-click → <i>Inspect</i>) → Console tab → paste code → <code>Enter</code>	Quick experiments
HTML + JS file	Create <code>index.html</code> with <code><script src="./index.js"></code> <code></script></code> and open it in the browser (VS Code <i>Live Server</i> is great)	Anything that uses buttons, <code>document</code> , the page (DOM)
Node.js	Save as <code>index.js</code> , then run <code>node index.js</code> in the terminal	Plain JavaScript without a web page

About the GitHub code in these notes

The course files keep many small experiments in one file, most of them commented out with `//`. In these notes I show **each experiment separately, un-commented**, in the same order as the file. I only changed spacing/indentation and split very long lines so they fit on the page — the logic is exactly the same. Spelling in the original strings (like `"persent"` or `"succesfully"`) is kept, so the output matches what you will see when you run the real file.

Contents

How to Use These Notes	2
CHAPTER 0 JavaScript Warm-up for a C++ Brain	8
0.1 Where does JavaScript run?	8
0.2 Variables: <code>let</code> , <code>const</code> and <code>var</code>	9
0.3 Types: no <code>int</code> , no <code>string</code> keyword	9
0.4 Printing and strings	10
0.5 <code>===</code> instead of <code>==</code>	11
0.6 Objects — like a <code>struct</code> , but flexible	11
0.7 Arrays — like <code>std::vector</code>	12
0.8 Functions are values	12
0.9 Callbacks — the idea behind this whole book	13
0.10 Loops, destructuring and spread	14
0.11 Errors: <code>throw</code> , <code>try</code> , <code>catch</code>	14
0.12 Things to Remember	15
CHAPTER 1 Array Superpowers: <code>forEach</code>, <code>map</code>, <code>filter</code>, <code>reduce</code>	16
1.1 The big idea: tell the array <i>what</i> , not <i>how</i>	16
1.2 Our sample data	17
1.3 <code>forEach()</code> — the simple loop	18
1.4 <code>map()</code> — the transformer	19
1.5 <code>filter()</code> — the sieve (the bouncer)	20
1.6 <code>reduce()</code> — the accumulator (the snowball)	22
1.7 <code>find()</code> , <code>some()</code> and <code>every()</code>	24
1.8 Chaining: using methods one after another	26
1.9 Cheat sheet: which method should I use?	26
1.10 Things to Remember	27
CHAPTER 2 Set and Map	28
2.1 <code>Set</code> — a list that never has duplicates	28
2.2 <code>Map</code> — keys can be <i>anything</i>	32
2.3 Things to Remember	37
CHAPTER 3 The Event Loop: How JavaScript Runs Your Code	38
3.1 What does “single-threaded” mean?	38
3.2 The call stack (you already know it from C++)	39
3.3 Synchronous = blocking	40
3.4 Web APIs: JavaScript’s helpers	40
3.5 The event loop — the coordinator	41

3.6 Step-by-step example: A, E, C, D, B	44
3.7 JavaScript never waits	45
3.8 What is an API?	49
3.9 Day 17 code, experiment by experiment	49
3.10 Things to Remember	53

CHAPTER 4 Callbacks and Callback Hell 55

4.1 The problem: some work takes time	55
4.2 What exactly is a callback?	56
4.3 The real-world story: ordering food on Zomato	57
4.4 The Day 18 code, step by step	58
4.5 Why can't we just call them one after another?	61
4.6 Why nesting works	62
4.7 The problems with callback hell	63
4.8 Key takeaways	67
4.9 Things to Remember	68

CHAPTER 5 Promises 70

5.1 What is a Promise?	70
5.2 The three states of a Promise	71
5.3 Creating a Promise	73
5.4 Consuming a Promise: <code>.then()</code> , <code>.catch()</code> , <code>.finally()</code>	75
5.5 Promise chaining — the power of Promises	76
5.6 The <code>.finally()</code> method	79
5.7 Promises and the event loop	82
5.8 <code>fetch()</code> — a real Promise from the browser	82
5.9 Day 19 code, experiment by experiment	84
5.10 Things to Remember	91

CHAPTER 6 JSON vs JavaScript Objects 93

6.1 What is JSON, and why do we need it?	93
6.2 Quick visual comparison	94
6.3 Key differences	94
6.4 Converting between JSON and JavaScript objects	95
6.5 Why does <code>fetch()</code> give us JSON?	97
6.6 Practical example: sending and receiving	97
6.7 Day 19 code: the JSON experiments	98
6.8 Summary	99
6.9 Things to Remember	100

CHAPTER 7 `async / await` 101

7.1 What is <code>async</code> / <code>await</code> ?	101
7.2 <code>async</code> functions always return a Promise	101

7.3 <code>await</code> — “pause this function until the promise settles”	102
7.4 Promises vs <code>async</code> / <code>await</code> — side by side	104
7.5 Error handling with <code>try</code> / <code>catch</code>	105
7.6 Important rules	107
7.7 Sequential vs parallel	108
7.8 Common patterns	109
7.9 Day 20 code, experiment by experiment	110
7.10 One app, three styles	114
7.11 Things to Remember	115

CHAPTER 8 Prototypes and Classes 117

8.1 Part 1 — the problem of repetition	117
8.2 Part 2 — the solution: the prototype chain	118
8.3 Every object already has a prototype	121
8.4 Part 3 — constructor functions and <code>new</code> (the old way)	123
8.5 The two “prototypes” (the most confusing part!)	125
8.6 Part 4 — the modern abstraction: <code>class</code>	126
8.7 Inheritance with <code>extends</code> and <code>super</code>	128
8.8 <code>this</code> inside a constructor	130
8.9 Day 21 code: the other experiments	132
8.10 Final summary from first principles	134
8.11 Things to Remember	134

CHAPTER 9 Strict Mode 136

9.1 The first principle: turning silent errors into loud errors	136
9.2 How to use it	137
9.3 The core differences in detail	138
9.4 Is this still relevant today?	140
9.5 Things to Remember	141

CHAPTER 10 The `this` Keyword, `call` / `apply` / `bind`, and Arrow Functions 142

10.1 The idea: “You are wearing a red shirt”	142
10.2 The first principle: the call-site	142
10.3 The global object: <code>window</code> , <code>global</code> , <code>globalThis</code>	143
10.4 Rule 1 — the method call (the most intuitive)	144
10.5 Rule 2 — the simple call (the most confusing)	144
10.6 Rule 3 — the explicit call: <code>call</code> , <code>apply</code> , <code>bind</code> (the “bossy” way)	145
10.7 Rule 4 — the constructor call with <code>new</code> (the “factory” way)	147
10.8 The big exception — arrow functions	149
10.9 <code>this</code> outside of any function (the global context)	149
10.10 Arrow functions in detail: the problem they were born to solve	151
10.11 The final intuitive summary — how to find <code>this</code>	157
10.12 All Day 22 experiments at a glance	157

10.13 Things to Remember	158
--------------------------------	-----

CHAPTER 11 Final Revision: Everything on a Few Pages 160

11.1 The big picture	160
11.2 Cheat sheets	161
11.3 Browser vs Node.js — the differences you met in these notes	162
11.4 Rapid-fire questions	162
11.5 Mixed practice — predict the output	164
11.6 A last word from your teacher	166

CHAPTER 0 Basics used in every chapter



JavaScript Warm-up for a C++ Brain

IN THIS CHAPTER YOU WILL LEARN

- Where JavaScript runs (browser vs Node.js)
- `let`, `const`, `var` — and how `const` is like a C++ constant pointer
- Types, `undefined`, `null`, `===`
- Objects and arrays (vs C++ `struct` and `std::vector`)
- Functions as values, arrow functions vs C++ lambdas
- Callbacks, destructuring, spread, `try`/`catch`

You already know C++, so JavaScript will look familiar: `if`, `for`, `while`, `{ }`, `;`, functions — all the same. The differences are in **how values and functions behave**. This short chapter teaches only the basics that the later chapters use again and again. If something in a later chapter looks strange, come back here.

0.1 Where does JavaScript run?

In C++ you **compile** your code into a program (`a.out` / `.exe`) and then run it. In JavaScript you don't compile anything yourself — a **JavaScript engine** reads your code and runs it directly. The most famous engine is **V8** (used by Chrome and by Node.js).

JavaScript has two common “homes”:

	Browser (Chrome, Firefox...)	Node.js
Used for	Web pages: buttons, forms, animations	Servers, command-line tools
Special global object	<code>window</code>	<code>global</code>
Can touch the web page (DOM)?	Yes — <code>document.getElementById(...)</code>	No page, no <code>document</code>
Extra powers (“APIs”)	timers, <code>fetch</code> , DOM events, <code>localStorage</code>	timers, <code>fetch</code> , files (<code>fs</code>), network
How to run	<code><script src="index.js"></code> in an HTML file, or the console (F12)	<code>node index.js</code>

Most of the code in these notes works in both. When something behaves differently in the browser and in Node.js, I will tell you clearly.

0.2 Variables: `let`, `const` and `var`

JAVASCRIPT

```
let age = 20;           // can be changed later
age = 21;               // ✓ fine

const name = "Rohit";  // cannot be re-assigned
// name = "Mohit";     // ✗ TypeError: Assignment to constant variable.

var old = 5;           // old style (before 2015). Avoid it in new code.
```

- Use `const` by default. Use `let` only when the value must change (like a loop counter).
- `var` is the old way. It ignores `{ }` blocks (it is *function-scoped*) and at the top level of a browser script it even becomes a property of `window`. We will see this in Chapter 10.



`const` in JavaScript = a constant *pointer*, not a constant *object*

A JavaScript `const` means “this name will always point to the same thing”. It does **not** freeze the object itself. That is exactly like a C++ **constant pointer**:

C++

```
User* const p = new User{"Rohit"};
p->name = "Mohit"; // ✓ allowed: we change the object, not the pointer
// p = new User{}; // ✗ error: the pointer itself is const
```

JAVASCRIPT

```
const user = { name: "Rohit" };
user.name = "Mohit"; // ✓ allowed (changing inside the object)
// user = {};       // ✗ TypeError: Assignment to constant variable.
```

0.3 Types: no `int`, no `string` keyword

In C++ you write the type (`int x = 5;`). In JavaScript you never write the type. A variable can hold any type, and can even change type later:

JAVASCRIPT

```
let x = 10;           // a number
x = "ten";            // now a string – JavaScript allows this!
console.log(typeof x); // "string"
```

The main types you will meet:

JavaScript type	Example	C++ cousin
<code>number</code>	<code>42</code> , <code>3.14</code> , <code>-7</code>	<code>double</code> — all numbers are 64-bit floating point
<code>string</code>	<code>"hi"</code> , <code>'hi'</code> , <code>`hi`</code>	<code>std::string</code>
<code>boolean</code>	<code>true</code> , <code>false</code>	<code>bool</code>
<code>undefined</code>	a variable with no value yet	(an uninitialised variable — but safe to read)
<code>null</code>	"intentionally empty"	<code>nullptr</code>
<code>object</code>	<code>{ name: "Rohit" }</code> , <code>[1, 2, 3]</code> , functions	<code>struct</code> / class objects

- `undefined` means "no value has been given". Reading a missing property does **not** crash — it just gives `undefined`:

JAVASCRIPT

```
const user = { name: "Rohit" };
console.log(user.age); // undefined (in C++ this would not even compile)
```

- `null` means "I am saying on purpose that there is nothing here".

0.4 Printing and strings

`console.log()` is JavaScript's `std::cout`. It can print many values separated by commas — they are printed with a space between them.

JAVASCRIPT

```
const name = "Rohit";
const age = 30;
console.log("Name:", name, "Age:", age); // Name: Rohit Age: 30
console.log(`Hello ${name}, you are ${age}`); // Hello Rohit, you are 30
```

The second line uses a **template literal**: a string written with backticks ``` in which `${ ... }` puts a value inside the string. You will see this in almost every example (C++20 has something similar:

```
std::format("Hello {}", name)).
```

✓ Arrays inside a template literal

If you put an array inside `${ }`, JavaScript joins its items with commas: ``${["Pizza", "coke"]}`` becomes `"Pizza,coke"`. Remember this — it appears in the Day 18 code.

0.5 `===` instead of `==`

Always compare with `===` (and `!==`). It checks **type and value**. The double `==` first *converts* types, which gives strange results:

JAVASCRIPT

```
console.log(5 === 5);      // true
console.log(5 === "5");   // false (number vs string)
console.log(5 == "5");    // true (!) "5" was converted to a number
```

0.6 Objects — like a `struct`, but flexible

JAVASCRIPT

```
const user = { name: "Rohit", age: 30 };

console.log(user.name);    // "Rohit" (dot notation)
console.log(user["age"]);  // 30 (bracket notation)

user.city = "Delhi";       // add a new property at any time!
delete user.age;           // remove a property
console.log(user);         // { name: 'Rohit', city: 'Delhi' }
```

A C++ `struct` has a fixed list of members decided at compile time. A JavaScript object is more like a **bag of key → value pairs** that can grow and shrink while the program runs. A good mental picture is `std::unordered_map<std::string, anything>`.

Objects are shared, like pointers

In JavaScript, a variable does not *contain* an object — it holds a **reference** to it (think: a pointer). Copying the variable copies the reference, not the object:

JAVASCRIPT

```
const a = { x: 1 };
const b = a;      // b points to the SAME object
b.x = 99;
console.log(a.x); // 99 - like two C++ pointers to one object
```

This matters a lot in Chapter 4, where the same `orderDetail` object travels through many functions.

0.7 Arrays — like `std::vector`

JAVASCRIPT

```
const arr = [10, 20, 30];
arr.push(40);           // like v.push_back(40)
console.log(arr.length); // 4      (like v.size())
console.log(arr[0]);     // 10
console.log(arr);        // [ 10, 20, 30, 40 ]
```

Arrays grow automatically and can even hold mixed types: `[1, "two", true]`.

0.8 Functions are values

This is the **most important difference** from C++ for these notes. In JavaScript a function is a normal value: you can store it in a variable, pass it to another function, and return it from a function. There are three common ways to write one:

JAVASCRIPT

```
function add(a, b) {           // 1. function declaration
  return a + b;
}

const sub = function (a, b) {  // 2. function expression (stored in a variable)
  return a - b;
};

const mul = (a, b) => a * b;    // 3. arrow function (short and modern)

console.log(add(2, 3), sub(9, 4), mul(3, 4)); // 5 5 12
```

Arrow function rules (read carefully!)

JAVASCRIPT

```
const square = x => x * x;           // 1 parameter: ( ) optional
const hello  = () => console.log("Hi"); // 0 parameters: ( ) required
const add2   = (a, b) => a + b;      // 2+ parameters: ( ) required

// Without { } the value is returned automatically.
// With { } you MUST write return yourself:
const sum = (a, b) => {
  const s = a + b;
  return s;
};

// To return an object directly, wrap it in ( ):
const makeUser = name => ({ name: name });
console.log(makeUser("Rohit")); // { name: 'Rohit' }
```

 Arrow functions ≈ C++ lambdas**C++**

```
auto square = [](int x) { return x * x; };
auto add2   = [](int a, int b) { return a + b; };
```

JAVASCRIPT

```
const square = x => x * x;
const add2   = (a, b) => a + b;
```

JavaScript arrow functions can use outside variables automatically (no capture list `[&]` needed). They also handle `this` in a special way — Chapter 10 explains it.

0.9 Callbacks — the idea behind this whole book

 Callback

A **callback** is a function that you **give to another function**, so that the other function can **call it back** — now, or later.

JAVASCRIPT

```
function doTwice(action) { // action is a function!
  action();
  action();
}

doTwice(() => console.log("Hello"));
```

OUTPUT

```
Hello
Hello
```

C++

```
// The same idea in C++
void doTwice(std::function<void()> action) { action(); action(); }
doTwice([]() { std::cout << "Hello\n"; });
```

You already used callbacks in C++ without calling them that: the comparator you pass to `std::sort` is a callback! In JavaScript they are everywhere: array methods (Chapter 1), timers and events (Chapter 3), and async code (Chapters 4–7).

0.10 Loops, destructuring and spread

`for`, `while` and `do...while` are the same as C++. JavaScript adds `for...of`, which is exactly C++'s range-based `for`:

JAVASCRIPT

```
for (const x of [10, 20, 30]) {
  console.log(x);           // 10, then 20, then 30
}
// C++: for (int x : v) std::cout << x;
```

Destructuring unpacks arrays and objects into variables (like C++17 structured bindings

```
auto [a, b] = pair;):
```

JAVASCRIPT

```
const [a, b] = [10, 20];           // a = 10, b = 20
const { name, age } = { name: "Rohit", age: 30 }; // name = "Rohit", age = 30
```

Spread `...` “opens the box” and spreads the items out:

JAVASCRIPT

```
const nums = [1, 2, 3];
const more = [...nums, 4, 5];
console.log(more); // [ 1, 2, 3, 4, 5 ]
```

0.11 Errors: `throw`, `try`, `catch`

Very similar to C++:

JAVASCRIPT

```
try {
  throw new Error("Something went wrong");
} catch (error) {
  console.log(error.message); // Something went wrong
}
```

C++

```
try {
  throw std::runtime_error("Something went wrong");
} catch (const std::exception& e) {
  std::cout << e.what();
}
```

In JavaScript you can `throw` any value (even a plain string), and a real `Error` object has a `.message` property.

0.12 Things to Remember

Things to Remember

- JavaScript runs inside an **engine** (V8). The browser gives you `window` and `document`; Node.js gives you `global` and `files`.
- Use `const` by default, `let` when the value changes, avoid `var`. A `const` object can still be modified inside.
- All numbers are 64-bit floating point (like C++ `double`).
- Missing property → `undefined` (no crash). `null` = “empty on purpose”.
- Always use `===` / `!==`.
- Objects and arrays are handled by **reference** (like pointers).
- Functions are **values**. Arrow functions: `x => x * 2`. With `{ }` you must write `return`.
- A **callback** is a function passed to another function to be called later.

CHAPTER 1 Lecture 12



Array Superpowers: forEach, map, filter, reduce

IN THIS CHAPTER YOU WILL LEARN

- What a **higher-order function** is
- `forEach` — the simple loop
- `map` — the transformer
- `filter` — the bouncer
- `reduce` — the snowball
- `find`, `some`, `every`
- Chaining methods together
- Which method to choose (cheat sheet)

1.1 The big idea: tell the array *what*, not *how*

In C++ you usually write a `for` loop and manage the index yourself. JavaScript arrays come with built-in methods where **you only say what should happen to each item** — the method does the looping for you.

JAVASCRIPT

```
const prices = [100, 200, 300];

// Old way (C++ style): we manage everything ourselves
const doubled = [];
for (let i = 0; i < prices.length; i++) {
  doubled.push(prices[i] * 2);
}

// New way: we only say WHAT to do with each price
const doubled2 = prices.map(price => price * 2);

console.log(doubled);    // [ 200, 400, 600 ]
console.log(doubled2);  // [ 200, 400, 600 ]
```

Both give the same answer, but the second one is shorter and says clearly what we want.

Higher-Order Function

A **higher-order function** is a function that **takes another function as an argument** (or returns a function). `map`, `filter`, `reduce`, `forEach` ... are higher-order functions. The function you pass in (like `price => price * 2`) is the **callback**.

What happens inside? (No magic!)

`map` is just a normal loop written for you. If you wrote it yourself, it would look like this:

JAVASCRIPT

```
function myMap(array, callback) {
  const result = [];
  for (let i = 0; i < array.length; i++) {
    result.push(callback(array[i], i, array)); // call YOUR function for each item
  }
  return result;
}

console.log(myMap([1, 2, 3], x => x * 10)); // [ 10, 20, 30 ]
```

So the array method **calls your callback once for every element**, and gives it three things: the **element**, its **index**, and the **whole array**. Most of the time we only use the first one (sometimes the second).

You already know these from the STL!

The C++ Standard Library has the same ideas, only with longer syntax:

JavaScript	C++ STL (<code><algorithm></code> / <code><numeric></code>)
<code>arr.forEach(fn)</code>	<code>std::for_each(v.begin(), v.end(), fn)</code> or <code>for (auto& x : v)</code>
<code>arr.map(fn)</code>	<code>std::transform(v.begin(), v.end(), out.begin(), fn)</code>
<code>arr.filter(fn)</code>	<code>std::copy_if(v.begin(), v.end(), std::back_inserter(out), fn)</code>
<code>arr.reduce(fn, init)</code>	<code>std::accumulate(v.begin(), v.end(), init, fn)</code>
<code>arr.find(fn)</code>	<code>std::find_if(v.begin(), v.end(), fn)</code>
<code>arr.some(fn)</code> / <code>arr.every(fn)</code>	<code>std::any_of(...)</code> / <code>std::all_of(...)</code>

1.2 Our sample data

All examples in this chapter use this array of products (from the lecture):

JAVASCRIPT

```
const products = [
  { id: 1, name: "Laptop", category: "Electronics", price: 1200, inStock: true },
  { id: 2, name: "Book", category: "Books", price: 30, inStock: true },
  { id: 3, name: "Coffee Maker", category: "Appliances", price: 150, inStock: false },
  { id: 4, name: "Headphones", category: "Electronics", price: 200, inStock: true },
];
```

It is an **array of objects**. Picture it as a small table:

index	id	name	category	price	inStock
0	1	Laptop	Electronics	1200	true
1	2	Book	Books	30	true
2	3	Coffee Maker	Appliances	150	false
3	4	Headphones	Electronics	200	true

1.3 `forEach()` — the simple loop

First thought: “I want to walk along the conveyor belt and do something with each item, but I am not creating a new list.”

Purpose: run a function once for each element. It is a modern replacement for a simple `for` loop.

JAVASCRIPT

```
array.forEach((element, index) => {  
  // ... your code ...  
});
```

- `element` — the current item being processed.
- `index` (optional) — the position of the current item.

JAVASCRIPT

```
console.log("--- Our Products ---");  
products.forEach(product => {  
  console.log(`- ${product.name}`);  
});
```

OUTPUT

```
--- Our Products ---  
- Laptop  
- Book  
- Coffee Maker  
- Headphones
```

Using the index too:

JAVASCRIPT

```
products.forEach((product, index) => {  
  console.log(`${index + 1}. ${product.name} costs ${product.price}`);  
});
```

OUTPUT

```
1. Laptop costs $1200
2. Book costs $30
3. Coffee Maker costs $150
4. Headphones costs $200
```

(In ``${product.price}`` the first `$` is just a dollar sign; `${ ... }` is the placeholder.)

Key characteristics:

- It does **not** return anything — it returns `undefined`.
- It does **not** create a new array.
- You **cannot** use `break` or `continue` inside it. A `return` inside the callback only ends the *current* call — it behaves like `continue`.

When to use: when you want to *do something* with each item and you don't need a new array — printing, updating the web page, saving each item to a database.

 Expecting forEach to give you something back

JAVASCRIPT

```
const result = products.forEach(p => p.name);
console.log(result); // undefined ❌ - forEach never returns anything
```

If you want a new array, use `map`. If you need to stop early, use a normal `for...of` loop (or `find` / `some` / `every`, which stop by themselves).

1.4 `map()` — the transformer

First thought: “I have a list of raw materials. I want to put each one through a machine to create a new list of finished products.”

Purpose: create a **new array** by transforming **every** element of the original array.

JAVASCRIPT

```
const newArray = array.map((element, index) => {
  return newValue; // this value goes into the new array at the same position
});
```

Key characteristics:

- It **always** returns a **new array**.
- The new array **always** has the **same length** as the original.
- It is **non-mutating** — the original array is not changed.

Example — a list of only the product names:

JAVASCRIPT

```
const productNames = products.map(product => {
  return product.name;
});
console.log(productNames); // [ 'Laptop', 'Book', 'Coffee Maker', 'Headphones' ]
console.log(products.length); // 4 - the original `products` array is unchanged!

// Short form (arrow function without { } returns automatically):
const names = products.map(product => product.name);
```

Example — make new objects with a 10% discount:

JAVASCRIPT

```
const sale = products.map(p => ({ name: p.name, price: p.price * 0.9 }));
console.log(sale);
```

OUTPUT

```
[
  { name: 'Laptop', price: 1080 },
  { name: 'Book', price: 27 },
  { name: 'Coffee Maker', price: 135 },
  { name: 'Headphones', price: 180 }
]
```

When to use: when you need a new array that is a modified version of the original. This is one of the most-used methods in JavaScript (and in React, later in your course!).

⚠ **Forgetting** `return` **inside** `{ }`

JAVASCRIPT

```
const names = products.map(p => { p.name; }); // ❌ curly braces, but no return
console.log(names); // [ undefined, undefined, undefined, undefined ]
```

Fix: `products.map(p => p.name)` or `products.map(p => { return p.name; })`.

1.5 `filter()` — the sieve (the bouncer)

First thought: “I have a big list. I want to run each item through a test and create a new, shorter list with only the items that pass.” Think of a bouncer at a club door: only people who pass the check get in.

Purpose: create a new array containing only the elements that meet a condition.

JAVASCRIPT

```
const newArray = array.filter((element, index) => {  
  return trueOrFalse; // true → keep this element, false → throw it away  
});
```

Key characteristics:

- It **always** returns a **new array**.
- The new array has the **same length or fewer** items — never more.
- It is **non-mutating**.
- If nothing passes, you get an **empty array** `[]` (not `undefined`).

Example — only products that are in stock **and** in the “Electronics” category:

JAVASCRIPT

```
const availableElectronics = products.filter(product => {  
  // The condition must evaluate to true or false.  
  // We combine two conditions with && (AND).  
  return product.inStock === true && product.category === "Electronics";  
});  
console.log(availableElectronics);
```

OUTPUT

```
[  
  { id: 1, name: 'Laptop', category: 'Electronics', price: 1200, inStock: true },  
  { id: 4, name: 'Headphones', category: 'Electronics', price: 200, inStock: true }  
]
```

A simple number example:

JAVASCRIPT

```
const nums = [5, 12, 8, 130, 44];  
console.log(nums.filter(n => n > 10)); // [ 12, 130, 44 ]  
console.log(nums.filter(n => n > 999)); // [] (nobody passed)
```

When to use: whenever you need to select a subset of data based on one or more conditions (search results, “in stock only”, “price below 500”...).

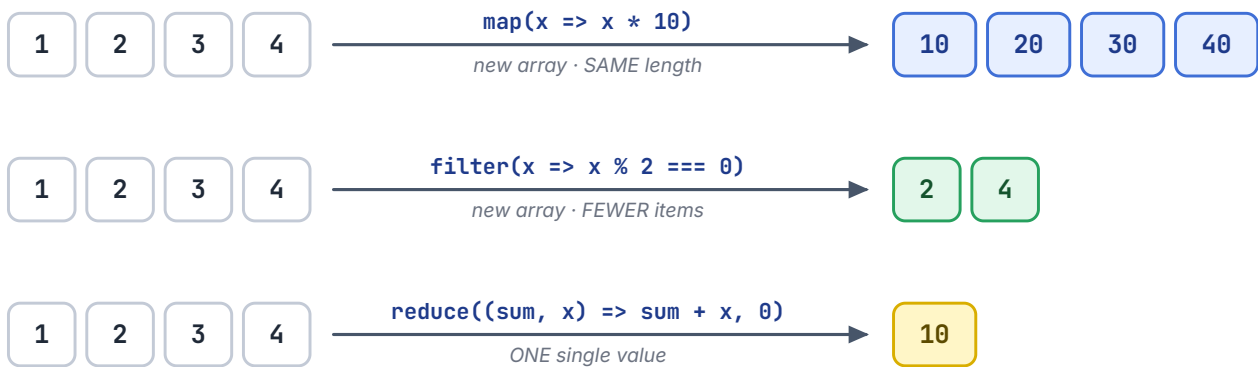


Figure 1.1 `map`, `filter` and `reduce` on the same input. `map` keeps the length, `filter` keeps only some items, `reduce` produces one value.

1.6 `reduce()` — the accumulator (the snowball)

First thought: “I want to roll this whole list into a single final value.” For example: the sum of all numbers, the total price, the most expensive item, or a count per category.

Imagine rolling a small snowball down a hill. At every step it picks up more snow. At the end you have **one** big snowball.

JAVASCRIPT

```
const finalValue = array.reduce((accumulator, currentValue, index) => {
  // ... combine accumulator with currentValue ...
  return newAccumulator; // becomes `accumulator` in the NEXT step
}, initialValue);
```

- `accumulator` — the value carried over from the previous step (the snowball).
- `currentValue` — the current element being processed.
- `initialValue` — the **starting value** of the accumulator (the small snowball you start with). This is a very important argument.

A simple example first: sum of numbers

JAVASCRIPT

```
const nums = [10, 20, 30, 40];
const total = nums.reduce((acc, curr) => acc + curr, 0);
console.log(total); // 100
```

Let's trace it step by step:

Step	acc (before)	curr	returned acc + curr → next acc
1	0 (initialValue)	10	10
2	10	20	30
3	30	30	60
4	60	40	100 ← final answer



Figure 1.2 The accumulator grows like a snowball: whatever you **return** becomes the accumulator of the next step.

Lecture example: total value of items in stock

JAVASCRIPT

```
const totalStockValue = products.reduce((total, product) => {
  console.log(`Current Total: ${total}, Current Product: ${product.name}, ` +
    `Price: ${product.price}`);
  if (product.inStock) {
    // The return value of this step becomes the 'total' for the NEXT step.
    return total + product.price;
  }
  // If not in stock, just return the current total without adding anything.
  return total;
}, 0); // Our initial value for the total is 0.

console.log(`\nFinal Total Stock Value: $${totalStockValue}`);
```

OUTPUT

```
Current Total: 0, Current Product: Laptop, Price: 1200
Current Total: 1200, Current Product: Book, Price: 30
Current Total: 1230, Current Product: Coffee Maker, Price: 150
Current Total: 1230, Current Product: Headphones, Price: 200

Final Total Stock Value: $1430
```

Notice the Coffee Maker step: it is **not** in stock, so we return **total** unchanged (1230 stays 1230). You **must return something in every step** — if a step returns nothing, the accumulator becomes **undefined**.

reduce can build any type — even an object

The lecture says reduce is the most powerful method because it can return *anything*: a number, a string, an array or an object. Here we count products per category:

JAVASCRIPT

```
const countByCategory = products.reduce((counts, product) => {
  counts[product.category] = (counts[product.category] || 0) + 1;
  return counts;
}, {}); // start with an empty object

console.log(countByCategory); // { Electronics: 2, Books: 1, Appliances: 1 }
```

`(counts[x] || 0)` means “the current count, or 0 if there is none yet” (because a missing property is `undefined`).



reduce = **std::accumulate**

C++

```
std::vector<int> nums = {10, 20, 30, 40};
int total = std::accumulate(nums.begin(), nums.end(), 0,
    [](int acc, int x) { return acc + x; }); // 100
```

Same three ingredients: the data, the **initial value** (`0`), and a function that combines the accumulator with the current element.

! Forgetting the initial value

If you skip `initialValue`, `reduce` uses the **first element** as the starting accumulator and begins from the second element. On an **empty array** this crashes:

JAVASCRIPT

```
[].reduce((a, b) => a + b); // ✗ TypeError: Reduce of empty array
                           //   with no initial value
[].reduce((a, b) => a + b, 0); // ✓ 0
```

Beginners: **always pass an initial value.**

When to use: whenever you need one summary value from an array — sum, average, maximum, grouping items into an object.

1.7 find(), some() and every()

These three are for *finding and testing*. They all **stop early** as soon as they know the answer.

find() — give me the first match

Like `filter`, but it **stops** and returns **the first element itself** that matches. If nothing matches, it returns `undefined`.

JAVASCRIPT

```
const coffeeMaker = products.find(product => product.name === "Coffee Maker");
console.log(coffeeMaker);
// { id: 3, name: 'Coffee Maker', category: 'Appliances', price: 150, inStock: false }

const phone = products.find(product => product.name === "Phone");
console.log(phone); // undefined
```

	filter	find
Returns	an array of all matches	one element — the first match
Nothing matches	<code>[]</code>	<code>undefined</code>
Stops early?	No, checks every element	Yes, at the first match

some() — is there at least one?

Returns `true` if **at least one** element passes the test, otherwise `false`. It stops as soon as it finds one.

JAVASCRIPT

```
const hasOutOfStockItems = products.some(product => product.inStock === false);
console.log(hasOutOfStockItems); // true (the Coffee Maker)
```

every() — do all of them pass?

Returns `true` only if **all** elements pass the test. It stops as soon as one fails.

JAVASCRIPT

```
const areAllItemsInStock = products.every(product => product.inStock === true);
console.log(areAllItemsInStock); // false (the Coffee Maker fails)
```

🧠 STL cousins

`find` ≈ `std::find_if` (but C++ gives you an *iterator* and you compare it with `end()`; JavaScript gives you the element or `undefined`). `some` ≈ `std::any_of`, `every` ≈ `std::all_of`. Bonus: JavaScript also has `findIndex` (returns the index or `-1`) and `includes(value)` (true/false for simple values).

1.8 Chaining: using methods one after another

`map` and `filter` return arrays, so you can call another method directly on the result. Read a chain from top to bottom like a recipe:

JAVASCRIPT

```
// Names of products that are in stock
const inStockNames = products
  .filter(p => p.inStock)    // step 1: keep in-stock products
  .map(p => p.name);        // step 2: take only their names
console.log(inStockNames); // [ 'Laptop', 'Book', 'Headphones' ]

// Total price of in-stock electronics
const total = products
  .filter(p => p.inStock && p.category === "Electronics")
  .reduce((sum, p) => sum + p.price, 0);
console.log(total);        // 1400
```

1.9 Cheat sheet: which method should I use?

Method	What it returns	New array?	Changes original?	Use it when you want to...
<code>forEach</code>	<code>undefined</code>	No	No	do something for each item
<code>map</code>	new array, same length	Yes	No	change every item
<code>filter</code>	new array, same or shorter	Yes	No	keep only some items
<code>reduce</code>	any single value	—	No	combine everything into one value
<code>find</code>	first match or <code>undefined</code>	No	No	get one item
<code>some</code>	<code>true</code> / <code>false</code>	No	No	ask "is there at least one ?"
<code>every</code>	<code>true</code> / <code>false</code>	No	No	ask "do all of them pass?"

1.10 Things to Remember

📌 Things to Remember

- These methods are **higher-order functions**: you pass them a **callback**, and they call it for each element with `(element, index, array)`.
- `map`, `filter` and `reduce` **never change the original array** (non-mutating).
- `map` → same length. `filter` → same or shorter. `reduce` → one value of any type.
- An arrow function with `{ }` **needs `return`**. Without `{ }` it returns automatically.
- In `reduce`, **return the accumulator in every step** and **always give an initial value**.
- `forEach` returns `undefined` and cannot `break`.
- `find` returns the element (or `undefined`); `filter` returns an array (maybe `[]`).
- `find`, `some` and `every` stop early. `some` = "at least one", `every` = "all".

🎯 Quick Quiz

1. What does `[1, 2, 3].map(x => x * 2)` return?
2. What does `[1, 2, 3, 4].filter(x => x > 2)` return?
3. What is `[1, 2, 3].reduce((a, b) => a + b, 10)` ?
4. What is `[5, 10, 15].find(x => x > 7)` ?
5. What is printed? `const r = [1, 2].forEach(x => x * 2); console.log(r);`
6. For `[2, 4, 5]`, what do `every(x => x % 2 === 0)` and `some(x => x % 2 === 0)` return?
7. Why does `products.map(p => { p.price * 2 })` give an array of `undefined` ?

✓ Answers

1. `[2, 4, 6]`
2. `[3, 4]`
3. `16` (the accumulator starts from 10)
4. `10` — the first match itself, not an array
5. `undefined` — `forEach` never returns anything
6. `every` → `false` (5 is odd), `some` → `true`
7. Because the arrow function uses `{ }` but has no `return`, so every call returns `undefined`.

CHAPTER 2 Lecture 12



Set and Map

IN THIS CHAPTER YOU WILL LEARN

- **Set**: a collection with **no duplicates**
- **add**, **has**, **delete**, **clear** and **.size**
- Removing duplicates from an array in one line
- **Map**: key → value pairs where the key can be **anything**
- Why plain objects fail with object keys
- **set**, **get**, **has**, **delete**, looping with **for...of**
- When to use Array vs Set, Object vs Map

2.1 Set — a list that never has duplicates

First thought: A **Set** is a list that enforces uniqueness. It is a collection where duplicates are impossible.

A members-only club

Think of a club's member list. You can add a person to the list, but you **can't add the same person twice** — the list automatically ignores the second attempt. A **Set** handles this for you, automatically.

Creating a Set

You create a Set with **new Set()**. You can optionally pass an array (any *iterable*) to fill it. Duplicates are removed automatically.

JAVASCRIPT

```
// Create an empty Set
const mySet = new Set();

// Create a Set from an array (duplicates are ignored)
const numbersArray = [1, 2, 3, 3, 4, 2, 5];
const numbersSet = new Set(numbersArray);
console.log(numbersSet); // Set(5) { 1, 2, 3, 4, 5 }
```



The Set keeps the FIRST time it sees a value and ignores the later copies (insertion order is kept).

Figure 2.1 Creating a Set from an array: the second 3 and the second 2 are dropped.

Core methods (CRUD)

Method	What it does	Returns
<code>.add(value)</code>	Adds a value. If it already exists, does nothing .	the Set itself (so you can chain <code>.add().add()</code>)
<code>.has(value)</code>	Checks if the value exists.	<code>true</code> / <code>false</code>
<code>.delete(value)</code>	Removes that value.	<code>true</code> if something was removed, else <code>false</code>
<code>.clear()</code>	Removes everything .	<code>undefined</code>
<code>.size</code>	How many values are inside (a property , no <code>()</code>).	a number

JAVASCRIPT

```
const userRoles = new Set();

// Add elements (add returns the Set, so we can chain calls)
userRoles.add("editor").add("viewer");
console.log(userRoles);           // Set(2) { 'editor', 'viewer' }

// Add a duplicate – nothing happens
userRoles.add("editor");
console.log(userRoles);           // Set(2) { 'editor', 'viewer' }

// Check for an element
console.log(userRoles.has("admin")); // false
console.log(userRoles.has("editor")); // true

// Delete an element
userRoles.delete("viewer");
console.log(userRoles.has("viewer")); // false

// Clear the entire Set
userRoles.clear();
console.log(userRoles);           // Set(0) {}
```

⚠️ `.size` , not `.length`

Arrays have `.length`, but a Set has `.size` — and it is a property, so no parentheses: `numbersSet.size` → `5`. Writing `numbersSet.length` gives `undefined`.

Looping over a Set

A Set is **iterable**, so the best way to loop is `for...of`. Items come out in **insertion order** (the order you added them).

JAVASCRIPT

```
const permissions = new Set(["read", "write", "execute"]);

for (const permission of permissions) {
  console.log(permission);
}
// read
// write
// execute

permissions.forEach(p => console.log(p.toUpperCase())); // READ, WRITE, EXECUTE
```

A Set has **no index**: `permissions[0]` is `undefined`. If you really need positions, turn it into an array first: `[...permissions][0]` → `"read"`.

Real-world use 1: remove duplicates from an array (the #1 use case)

JAVASCRIPT

```
const duplicateEmails = ["a@a.com", "b@b.com", "a@a.com"];

// Convert to a Set to remove duplicates, then spread it back into a new array
const uniqueEmails = [...new Set(duplicateEmails)];
console.log(uniqueEmails); // [ 'a@a.com', 'b@b.com' ]
```

Read `[...new Set(arr)]` from the inside out: **(1)** `new Set(arr)` removes the duplicates, **(2)** `[...]` spreads the Set's values into a brand-new array. (`Array.from(new Set(arr))` does the same.)

Real-world use 2: fast “have I seen this before?” checks

JAVASCRIPT

```
// Imagine tracking unique visitors to a page
const visitedUsers = new Set();

function userVisits(userId) {
  if (!visitedUsers.has(userId)) {
    console.log(`Welcome, new visitor #${userId}!`);
    visitedUsers.add(userId);
  } else {
    console.log(`Welcome back, visitor #${userId}!`);
  }
}

userVisits(101);
userVisits(102);
userVisits(101);
```

OUTPUT

```
Welcome, new visitor #101!
Welcome, new visitor #102!
Welcome back, visitor #101!
```

`Set.has()` is **much faster** than `Array.includes()` for large data. `includes` checks the items one by one ($O(n)$); a Set uses hashing, so `has` is about $O(1)$ — like a C++ `unordered_set`. With 10 items you won't notice; with 1,000,000 items the difference is huge.

 **Set** $\approx$ `std::unordered_set` (that remembers order)

JavaScript <code>Set</code>	C++ <code>std::unordered_set</code>
<code>s.add(x)</code>	<code>s.insert(x)</code>
<code>s.has(x)</code>	<code>s.count(x)</code> / <code>s.contains(x)</code> (C++20)
<code>s.delete(x)</code>	<code>s.erase(x)</code>
<code>s.clear()</code> / <code>s.size</code>	<code>s.clear()</code> / <code>s.size()</code>

One difference: a JavaScript Set **remembers insertion order**. `std::unordered_set` has no order, and `std::set` **sorts** the values. A JS Set never sorts.

 Objects are compared by reference

For objects, a Set checks “is it the same object in memory?” — not “does it look the same?”:

JAVASCRIPT

```
const s = new Set();
s.add({ id: 1 });
s.add({ id: 1 }); // a different object that only LOOKS the same
console.log(s.size); // 2
```

It is like a C++ set of **pointers**: two different pointers are two different keys, even if the objects they point to have equal contents. Strings and numbers are compared by value:

```
new Set(["a", "a"]).size → 1.
```

Use... When you need...

Array	An ordered list where duplicates are allowed and you need index-based access (<code>arr[0]</code>).
Set	A collection of unique values where the main jobs are adding, deleting and checking whether something exists.

2.2 Map — keys can be *anything*

First thought: A `Map` is like an object, but its keys can be anything (not just strings).

This is the most important difference. In a normal object `{}`, keys are **automatically converted to strings**. A `Map` keeps the key's real type, so you can use objects, functions, numbers, booleans... as keys.

The problem with plain objects

JAVASCRIPT

```
let myObject = {};
let keyObject1 = { id: 1 };
let keyObject2 = { id: 2 };

// Both keys get converted to the SAME string: "[object Object]"
myObject[keyObject1] = "Value for key 1";
myObject[keyObject2] = "Value for key 2"; // this overwrites the first one!

console.log(myObject); // { '[object Object]': 'Value for key 2' }
```

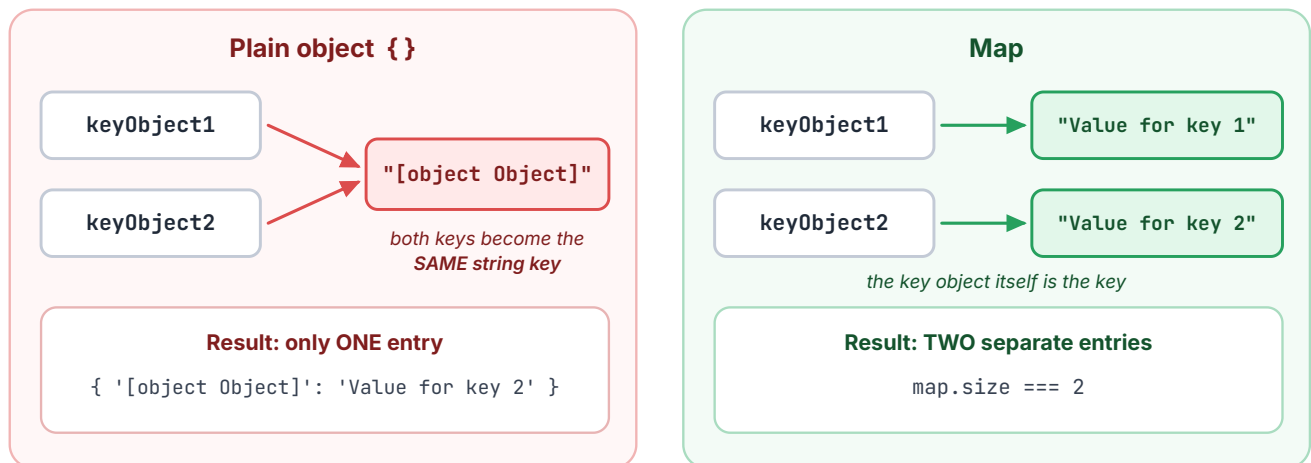


Figure 2.2 With an object, every object key turns into the same string `"[object Object]"`. A Map keeps each object as its own key.

The same thing happens with numbers: `obj[1]` and `obj["1"]` are the **same** property, because `1` is converted to the string `"1"`.

Creating a Map

Use `new Map()`. You can fill it at creation time with an array of `[key, value]` pairs:

JAVASCRIPT

```
// Create an empty Map
const myMap = new Map();

// Create a Map with initial values – notice the different key types!
const userMap = new Map([
  ["name", "Alice"], // string key
  [true, "is verified"], // boolean key
  [100, "points"], // number key
]);

console.log(userMap.get(true)); // "is verified"
console.log(userMap.get(100)); // "points"
console.log(userMap.get("100")); // undefined – "100" (string) is not 100
```

Core methods (CRUD)

Method	What it does
<code>.set(key, value)</code>	Adds or updates a pair. Returns the Map, so you can chain.
<code>.get(key)</code>	Gives the value for that key, or <code>undefined</code> if the key doesn't exist.
<code>.has(key)</code>	<code>true</code> / <code>false</code> — does this key exist?
<code>.delete(key)</code>	Removes that pair.
<code>.clear()</code>	Removes all pairs.
<code>.size</code>	Number of pairs (property, no <code>()</code>).

JAVASCRIPT

```
const metadata = new Map();
let user1 = { name: "Alice" };
let user2 = { name: "Bob" };

// Set values using OBJECTS as keys
metadata.set(user1, { lastLogin: "2023-10-27" });
metadata.set(user2, { lastLogin: "2023-10-26" });

// Get a value using the exact same object reference
console.log(metadata.get(user1)); // { lastLogin: '2023-10-27' }
console.log(metadata.has(user2)); // true
console.log(metadata.size);       // 2

console.log(metadata.get({ name: "Alice" })); // undefined - a NEW object!
```

The last line is important: object keys work **by reference**, exactly like the Set in the previous section. You must use the *same* object (`user1`) to get the value back.

Looping over a Map

A Map is iterable too. Each item is a `[key, value]` pair, so we use `for...of` with **destructuring**. Items come out in **insertion order**.

JAVASCRIPT

```
const userMap = new Map([
  ["name", "Alice"],
  ["age", 30],
]);

// Use destructuring to unpack the [key, value] pair
for (const [key, value] of userMap) {
  console.log(`${key} -> ${value}`);
}
// name -> Alice
// age -> 30

console.log([...userMap.keys()]); // [ 'name', 'age' ]
console.log([...userMap.values()]); // [ 'Alice', 30 ]
userMap.forEach((value, key) => console.log(key, value)); // VALUE comes first!
```

Map ≈ std::unordered_map / std::map

C++

```
std::unordered_map<std::string, int> m;
m["age"] = 30; // like m.set("age", 30)
for (auto& [key, value] : m) // C++17 structured bindings
    std::cout << key << " -> " << value << "\n";
```

Two differences to remember:

1. In C++, `m[key]` inserts a default value if the key is missing. In JavaScript, `map.get(key)` just returns `undefined` and inserts nothing.
2. A JavaScript Map keeps insertion order; `std::map` sorts by key and `std::unordered_map` has no order.

⚠ Using `[ ]` on a Map

JAVASCRIPT

```
const m = new Map();
m["name"] = "Rohit"; // ❌ a normal object property, NOT a Map entry!
console.log(m.get("name")); // undefined
console.log(m.size); // 0

m.set("name", "Rohit"); // ✅ the correct way
console.log(m.get("name")); // Rohit
```

Real-world uses of Map

1. **Storing extra data about objects (metadata).** This is the killer use case. You can “attach” data to an object without changing the object itself — great for keeping web-page elements clean:

JAVASCRIPT

```
const elementData = new Map();
const button1 = document.querySelector("#btn1");

// Associate some data with this specific button element
elementData.set(button1, { clicks: 0, lastClickTime: null });

// Now we can store and read data about button1 without ever doing
// button1.mydata = ... (which would pollute the DOM object)
```

2. **Caching (remembering results).** If a function does a slow calculation, store the answer in a Map.

The argument is the key and the result is the value:

JAVASCRIPT

```
const cache = new Map();

function slowSquare(n) {
  if (cache.has(n)) {
    console.log(`From cache: ${n}`);
    return cache.get(n);
  }
  console.log(`Calculating: ${n}`);
  const result = n * n; // imagine a very heavy calculation here
  cache.set(n, result);
  return result;
}

slowSquare(4); // Calculating: 4
slowSquare(4); // From cache: 4 (instant!)
```

3. A “dictionary” with non-string keys. Whenever your keys are not simple strings (objects, numbers you don’t want converted, booleans...), a Map is the correct choice.

Object or Map?

Use...	When you need...
Object	A simple, fixed set of string keys that describe one thing — like a <code>user</code> with <code>name</code> , <code>age</code> , <code>email</code> . Easy to write, and works directly with JSON (Chapter 6).
Map	A collection of key → value pairs where keys can be any type , you add/remove entries often, you need <code>.size</code> , or insertion order matters.

Feature	Object <code>{}</code>	Map
Key types	strings (other types get converted to strings)	anything — objects, numbers, booleans, functions...
Number of entries	<code>Object.keys(obj).length</code>	<code>map.size</code>
Loop directly with <code>for...of</code>	No (use <code>Object.entries(obj)</code>)	Yes
Converts to JSON with <code>JSON.stringify</code>	Yes	Not directly (you get <code>{}</code>)

2.3 Things to Remember

📌 Things to Remember

- `new Set(array)` removes duplicates. `[...new Set(array)]` gives back a **unique array**.
- Set methods: `add` (chainable), `has`, `delete`, `clear`; count with `.size`.
- `Set.has` is fast (hashing, like `unordered_set`); `Array.includes` checks one by one.
- Sets and Maps keep **insertion order** and have **no index**.
- Objects inside a Set / as Map keys are compared **by reference** (same object in memory), not by content.
- In a plain object, keys become **strings** → two object keys collide as `"[object Object]"`. A **Map keeps the key's real type**.
- Map methods: `set` (chainable), `get` (→ `undefined` if missing), `has`, `delete`, `clear`, `.size`.
- Loop a Map with `for (const [key, value] of map)`. Never use `map[key] = value`.

🎯 Quick Quiz

1. What is `new Set([1, 1, 2, 2, 3]).size`?
2. What does `[...new Set("hello")]` give? (Hint: a string is iterable.)
3. What does this print?

JAVASCRIPT

```
const m = new Map();
const k = { id: 1 };
m.set(k, "first");
console.log(m.get(k), m.get({ id: 1 }));
```

4. What does this print? `const o = {}; o[1] = "a"; o["1"] = "b"; console.log(o[1]);`
5. You need to store the number of clicks for each button element on a page. Object or Map? Why?

✓ Answers

1. `3`
2. `['h', 'e', 'l', 'o']` — the second `l` is a duplicate, so it is dropped.
3. `first undefined` — the second `{ id: 1 }` is a brand-new object (a different reference).
4. `b` — the key `1` becomes the string `"1"`, so both lines write the **same** property.
5. A **Map**, because the keys are objects (button elements). In a plain object all of them would become `"[object Object]"`.

CHAPTER 3 Lecture 17 · Day 17 code



The Event Loop: How JavaScript Runs Your Code

IN THIS CHAPTER YOU WILL LEARN

- What **single-threaded** means
- The **call stack** (same idea as in C++)
- Why synchronous code **blocks** the page
- **Web APIs** — the browser's helpers
- The **event loop**, microtasks and macrotasks
- Why "JavaScript never waits"
- What an **API** is
- Every experiment from the **Day 17** code

This is the most important chapter for understanding **asynchronous** JavaScript. Callbacks (Chapter 4), Promises (Chapter 5) and `async` / `await` (Chapter 7) all make sense only after you understand the event loop. Go slowly here.

3.1 What does "single-threaded" mean?

First principle: your JavaScript code runs on **one** call stack. Only **one** thing executes at a time.

JAVASCRIPT

```
console.log('First');  
console.log('Second');  
console.log('Third');
```

OUTPUT

```
First  
Second  
Third
```

This **always** runs in this order. JavaScript can't run `Second` and `Third` at the same time. It is like a **single-lane road** — the cars must go one at a time.

🧠 One thread for your code

In C++ you can create extra threads with `std::thread` and run functions truly in parallel. JavaScript does **not** give your code extra threads — all your code runs on **one main thread**. The good news: no `std::mutex`, no data races on your variables. The challenge: if one piece of code is slow, **everything** else waits.

3.2 The call stack (you already know it from C++)

The **call stack** is exactly the same idea as in C++ — the thing you see when you learn recursion. When a function is **called**, a *frame* is **pushed** on top of the stack. When the function **returns**, its frame is **popped**. JavaScript always runs whatever is on the **top**.

JAVASCRIPT

```
function multiply(a, b) {  
  return a * b;  
}  
function square(n) {  
  return multiply(n, n);  
}  
function printSquare(n) {  
  const result = square(n);  
  console.log(result);  
}  
printSquare(4); // 16
```

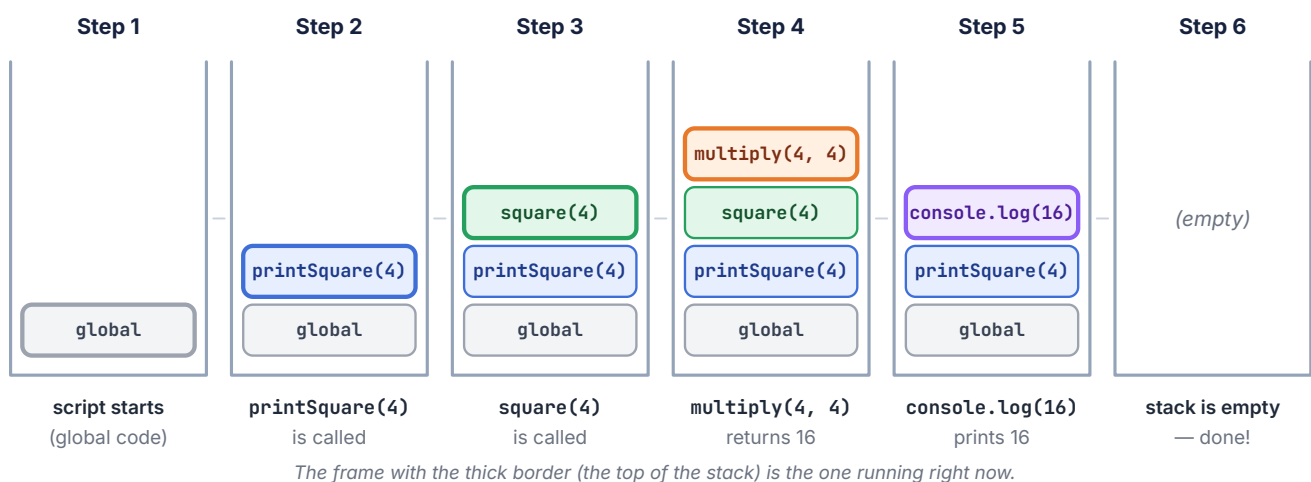


Figure 3.1 The call stack while `printSquare(4)` runs. Frames are pushed when a function is called and popped when it returns.

If a function keeps calling itself forever, the stack overflows — in C++ you get a crash (segmentation fault); in JavaScript you get `RangeError: Maximum call stack size exceeded`. Same idea!

3.3 Synchronous = blocking

First principle: each line **blocks** the next line until it completes.

JAVASCRIPT

```
function slowFunction() {  
  // Simulate slow work  
  let sum = 0;  
  for (let i = 0; i < 3000000000; i++) {  
    sum += i;  
  }  
  return sum;  
}  
  
console.log('Start');  
slowFunction();           // This BLOCKS everything  
console.log('End');       // This waits for slowFunction to finish
```

While `slowFunction()` runs, the page **freezes**: you can't click buttons, you can't scroll, animations stop. Why? Because the one and only thread is busy with the loop. Clicking a button needs the same thread to run the click code — so the click has to wait. **This is the problem** that the rest of this chapter solves.

3.4 Web APIs: JavaScript's helpers

If JavaScript has only one thread, how can `setTimeout` wait 2 seconds without freezing the page?

First principle: the **browser** gives JavaScript extra powers called **Web APIs**, and they run **outside** JavaScript's single thread. JavaScript doesn't do the waiting — **the browser does**.

JAVASCRIPT

```
console.log('1. Start');  
  
// setTimeout is a Web API, NOT part of JavaScript itself  
setTimeout(() => {  
  console.log('2. Inside timeout');  
}, 2000);  
  
console.log('3. End');
```

OUTPUT

```
1. Start  
3. End  
2. Inside timeout ← 2 seconds later
```

What happened?

1. JavaScript calls `setTimeout` and gives it a callback and a time (2000 ms).
2. The **browser** takes over the timer (not JavaScript!).

- JavaScript immediately continues to line 3 and prints `3. End`.
- After 2 seconds the browser says: “Hey, run this callback” — and `2. Inside timeout` is printed.

Common Web APIs (from the lecture’s diagram):

Web API	What the browser does for you	Your callback runs...
<code>setTimeout</code> / <code>setInterval</code>	counts time	after the time is over (once / again and again)
<code>fetch()</code>	sends a network request and waits for the reply	when the response arrives
DOM events: <code>addEventListener('click', ...)</code>	watches the page for clicks, key presses...	every time the event happens
<code>localStorage</code> , <code>location</code>	stores data, reads/changes the URL	(these answer immediately)
<code>console.log</code>	prints to the console	(runs immediately)
Node.js: <code>fs.readFile(...)</code>	reads a file from disk	when the file has been read



`console.log` is given by the browser too — but it is **synchronous**

`console` , `document` , `localStorage` are also provided by the browser (they are not part of the JavaScript language itself). But they do their job **immediately** and return. Only APIs that have to *wait* for something (time, network, disk, the user) use a callback later.



A chef working alone

You are a chef working **alone** in a kitchen (one thread). You put rice in an **automatic rice cooker** (a Web API) and set its buzzer. You don’t stand and stare at the cooker — you keep chopping vegetables. When the buzzer rings, you add “*serve the rice*” to your to-do list (the **queue**). You do it as soon as your hands are free (the **call stack is empty**).

3.5 The event loop — the coordinator

First principle: the **event loop** connects single-threaded JavaScript with the asynchronous Web APIs. Here are all the parts:

- Call Stack** — where your JavaScript runs, one frame at a time.
- Web APIs** — the browser’s helpers (timers, network, DOM events). They work outside the JS thread.
- Task queues** — waiting areas for callbacks that are ready to run:
 - Microtask queue** — Promise callbacks (`.then` , `.catch` , `.finally` , code after `await`). **Higher priority** — the VIP line.
 - Macrotask queue** (also called the *callback queue* or *task queue*) — `setTimeout` , `setInterval` , clicks and other events, I/O. **Lower priority**.

- **Event Loop** — keeps checking: “Is the call stack empty? Then move the next callback from a queue onto the stack.”

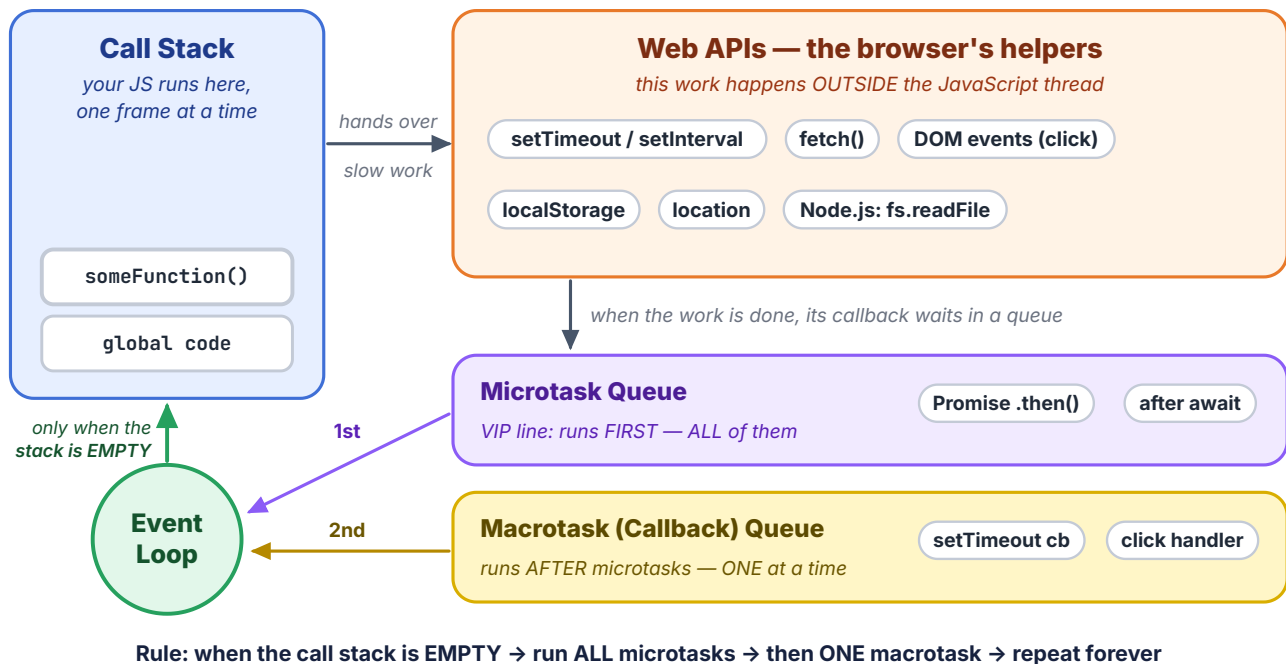


Figure 3.2 The whole machine. Your code runs on the call stack; slow work is handed to Web APIs; finished work waits in a queue; the event loop moves it to the stack when the stack is empty.

The event loop follows one simple rule, forever:

The event loop rule

When the call stack is empty: first run **ALL** the microtasks (Promises), then run **ONE** macrotask (`setTimeout` , click...). Then check the microtasks again, and repeat forever.

The event loop in C++-style pseudo-code

If you have ever written a game loop or a GUI message loop, the event loop is the same idea:

PSEUDO-CODE

```
runWholeScript();                                // 1. your file runs top to bottom (sync code)

while (true) {                                    // the event loop
  while (!microtaskQueue.empty())                // 2. run ALL microtasks (Promise callbacks)
    runAndPop(microtaskQueue);

  if (!macrotaskQueue.empty())                   // 3. run ONE macrotask (setTimeout, click...)
    runAndPop(macrotaskQueue);

  // 4. the browser may repaint the screen here, then loop again
}
```

Notice: a callback can only run when **nothing else** is running. JavaScript never interrupts running code in the middle.

Example: who prints first?

JAVASCRIPT

```
console.log('1. Synchronous');

setTimeout(() => {
  console.log('2. Timeout 0ms');
}, 0);

Promise.resolve().then(() => {
  console.log('3. Promise');
});

console.log('4. Synchronous');
```

OUTPUT

```
1. Synchronous
4. Synchronous
3. Promise
2. Timeout 0ms
```

Why this order? Normal (synchronous) code always runs first — lines 1 and 4. Then the stack is empty, so the event loop runs the **microtasks** (the Promise → **3**). Only after that does it run a **macrotask** (the timeout → **2**), even though the timeout was 0 ms.

3.6 Step-by-step example: A, E, C, D, B

JAVASCRIPT

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve()
  .then(() => console.log('C'))
  .then(() => console.log('D'));
console.log('E');
```

OUTPUT

```
A
E
C
D
B
```

Let's play the event loop ourselves:

Step	What happens	Microtask queue	Macrotask queue	Printed so far
1	Call stack runs <code>console.log('A')</code>	—	—	A
2	<code>setTimeout</code> → handed to the browser's timer. 0 ms passes, so its callback waits in the macrotask queue	—	B	A
3	<code>Promise.resolve().then(C)</code> → the promise is already resolved, so callback <code>C</code> goes to the microtask queue	C	B	A
4	Call stack runs <code>console.log('E')</code> . The script is finished → stack empty	C	B	A E
5	Event loop: microtasks first! <code>C</code> runs. The second <code>.then</code> is now ready → <code>D</code> is added	D	B	A E C
6	Still a microtask left: <code>D</code> runs	—	B	A E C D
7	Microtasks empty → one macrotask: <code>B</code> runs	—	—	A E C D B

Notice step 5: `D` is not in the queue from the beginning. A chained `.then` becomes ready only **after** the previous one has finished.

⚠️ `setTimeout(fn, 0)` does NOT mean “run now”

The time you give to `setTimeout` is the **minimum** waiting time, not an exact time. The callback runs only when (1) the time is over **and** (2) the call stack is empty **and** (3) all microtasks are done.

JAVASCRIPT

```
setTimeout(() => console.log("timer"), 0);
for (let i = 0; i < 1e9; i++) {} // keeps the stack busy for about a second
console.log("loop done");
```

OUTPUT

```
loop done
timer ← only after the loop finished
```

3.7 JavaScript never waits

The lecture's big idea: **JavaScript doesn't know and doesn't care** whether a function is “slow”. It just calls the function, takes whatever the function **returns**, and moves to the next line. The **browser** decides what happens later.

From JavaScript's point of view, **all function calls look the same**:

JAVASCRIPT

```
const result1 = Math.random();           // call function
const result2 = document.getElementById('btn'); // call function
const result3 = setTimeout(() => {}, 1000); // call function
const result4 = fetch('/api/data');       // call function

// JavaScript just:
// 1. calls the function
// 2. gets whatever is returned
// 3. moves to the next line
// JavaScript NEVER waits!
```

The key: return value vs. callback

- A **synchronous** function returns the **actual result** immediately. Example: `window.innerWidth` gives `1920` right away and JavaScript uses it.
- An **asynchronous** function returns a “ticket” immediately (a timer ID, or a Promise). The **real result** comes later through a callback.

JAVASCRIPT

```
console.log('1');

// JavaScript doesn't know this is "async" - it just calls it and gets a return value
const timerId = setTimeout(() => {
  console.log('3');
}, 1000);

console.log('2', timerId); // timerId is returned immediately!
```

OUTPUT

```
1
2 1 ← the ticket: a timer ID (a number in the browser)
3 ← 1 second later
```

JavaScript never stopped! It called `setTimeout`, got the ticket `timerId`, and moved on. (In Node.js the ticket is a `Timeout` object instead of a number — still just a ticket, not the result.)



The token at a food counter

At a busy food court you pay at the counter and **immediately** get a **token number** (the ticket). You don't stand frozen at the counter until the food is ready — you go and sit with your friends. When your number is called (the callback), you collect the food (the result). `setTimeout` gives you a timer ID token; `fetch` gives you a Promise token.

Proof: JavaScript doesn't "wait" even for `fetch`

JAVASCRIPT

```
console.log('A');
fetch('/api/data'); // looks like it should wait... but it doesn't!
console.log('B');   // runs immediately

const promise = fetch('/api/data');
console.log(promise); // Promise { <pending> } - a ticket, not the data
```

OUTPUT

```
A
B
Promise { <pending> }
```

`fetch()` returns a **Promise immediately** — JavaScript got *something* back (the ticket) and moved on. The data will arrive later.

Code	What comes back immediately	Is it the real result?
<code>add(2, 3)</code>	<code>5</code>	yes
<code>Math.random()</code>	<code>0.234...</code>	yes
<code>window.innerWidth</code>	<code>1920</code>	yes
<code>setTimeout(fn, 1000)</code>	a timer ID, e.g. <code>1</code>	no — the work happens later, in <code>fn</code>
<code>fetch('/api')</code>	a Promise (pending)	no — the data comes later, via <code>.then</code>

A visual timeline

JAVASCRIPT

```
console.log('1');
const id = setTimeout(() => console.log('3'), 1000);
console.log('2');
```

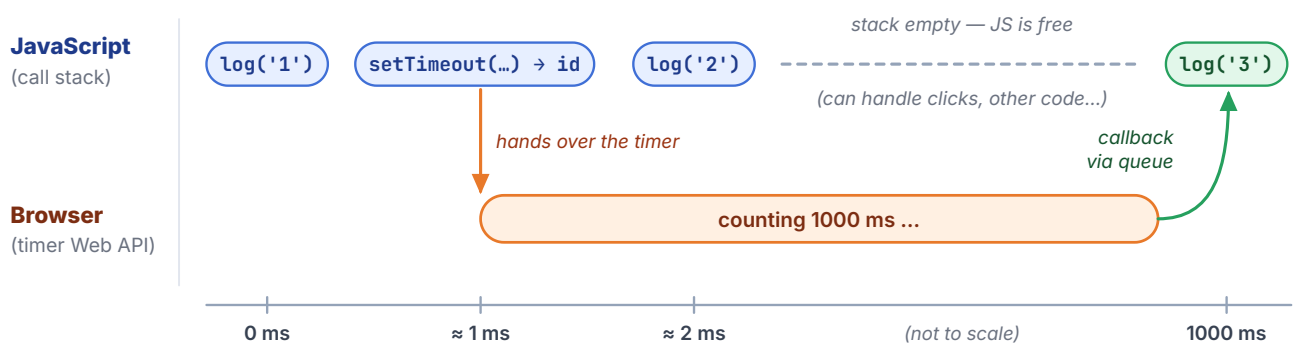


Figure 3.3 Timeline of the code above. JavaScript finishes its own work in a few milliseconds; the browser counts the 1000 ms separately and then hands the callback back.

JavaScript's "contract" with every function

JAVASCRIPT

```
// ALL functions work the same way from JavaScript's perspective:
// 1. Call the function
// 2. Get the return value (might be a number, undefined, a Promise, a timer ID...)
// 3. Continue immediately

function sync() {
  return 42;
}

function async() {
  return new Promise(resolve => {
    setTimeout(() => resolve(42), 1000);
  });
}

const a = sync();    // gets 42
const b = async();   // gets a Promise (not 42!)
// JavaScript calls both the SAME way. It doesn't "know" or "care" which is which.
```

Three ways an async function can give you the result later

JAVASCRIPT

```
// Method 1: Callback - "I'll call YOU when I'm done"
setTimeout(() => {
  console.log('Result!');           // called later
}, 1000);

// Method 2: Promise - "Here's a Promise, use .then()"
const promise = fetch('/api/data');
promise.then(result => {
  console.log('Result!', result); // called later
});

// Method 3: async/await - looks like waiting, but it's just Promise syntax!
async function load() {
  const result = await fetch('/api/data');
  // Only THIS function pauses here. The JavaScript engine keeps doing other work.
}
```

We will study Method 1 in Chapter 4, Method 2 in Chapter 5 and Method 3 in Chapter 7.

3.8 What is an API?

API (Application Programming Interface)

An **API** is any way for code to talk to other code. It has three parts:

- **Interface** — *how* to interact with it (which function to call, which arguments to give).
- **Abstraction** — you **don't need to know how it works inside**.
- **Contract** — *what to send* and *what you'll get back*.

Example: `setTimeout(callback, ms)`. **Interface:** give a function and a number. **Contract:** you get a timer ID now, and your function is called after at least `ms` milliseconds. **Abstraction:** you have no idea how the browser's timer works inside — and you don't need to.

You use APIs in C++ every day

When you call `std::sort(v.begin(), v.end())` you know the **contract** (give two iterators → the range becomes sorted), but not the **internal algorithm** (it's usually introsort). The header file is the **interface**. That is exactly what an API is.

Two meanings you will hear in web development:

- **Web APIs** — features the *browser* gives to your code (`setTimeout` , `fetch` , the DOM).
- **A server's API**, like `https://api.github.com/users` — a URL you *send a request to* over the internet, and it sends data back. Same idea: a contract between two pieces of code.

3.9 Day 17 code, experiment by experiment

Day 17 — `index.html` + `first.js`

The page has three coloured buttons on a black background. The script tag is written **after** the body, so the buttons already exist when `first.js` runs (that's why `getElementById` can find them).

HTML 03JS/Day17/index.html

```
<body style="background-color: black; color: white;">
  <button id="button1" style="background-color: red;">Button1</button>
  <button id="button2" style="background-color: orange;">Button2</button>
  <button id="button3" style="background-color: green;">Button3</button>
</body>
<script src="./first.js"></script>
```

Experiment 1 — a loop that blocks everything

JAVASCRIPT Day 17 · first.js — experiment 1

```
console.log("First");
let sum = 0;
for (let i = 0; i < 3000000000; i++)
  sum += i;

console.log(sum);
console.log("Last");
```

OUTPUT

```
First
4499999997067114000 ← after a few seconds of freezing
Last
```

- 3 billion loop steps take a few seconds (about 4 s on my machine). During that time the page is **frozen** — nothing else can run, because the one thread is busy.
- `"Last"` is printed only after the loop ends: synchronous code **blocks**.

🧠 Why is the number slightly “wrong”?

The exact sum $0 + 1 + \dots + 2,999,999,999$ is 4,499,999,998,500,000,000. JavaScript printed 4,499,999,997,067,114,000. That’s because **every JavaScript number is a 64-bit `double`** (like C++ `double`), which stores integers exactly only up to $2^{53} \approx 9 \times 10^{15}$. After that, each `+=` is rounded a little, and the tiny errors add up. The same loop with a C++ `double` would behave the same way (with `long long` you would get the exact answer).

Experiment 2 — `setTimeout` doesn’t block

JAVASCRIPT Day 17 · first.js — experiment 2

```
console.log("Hello Ji");

setTimeout(() => {
  console.log("Time Out Executed");
}, 5000);

console.log("I am the end");
```

OUTPUT

```
Hello Ji
I am the end
Time Out Executed ← 5 seconds later
```

The timer is handed to the browser (Web API). JavaScript immediately continues and prints `"I am the end"`. After 5 s, the callback goes to the macrotask queue, and the event loop runs it (the stack is empty by then).

Experiment 3 — button click listeners

JAVASCRIPT Day 17 · first.js — experiment 3

```

console.log("Hi Ji");

const button1 = document.getElementById("button1");
button1.addEventListener('click', () => {
  console.log("Button 1 is clicked");
});

const button2 = document.getElementById("button2");
button2.addEventListener('click', () => {
  console.log("Button 2 is clicked");
});

const button3 = document.getElementById("button3");
button3.addEventListener('click', () => {
  console.log("Button 3 is clicked");
});

console.log("I am the end");

```

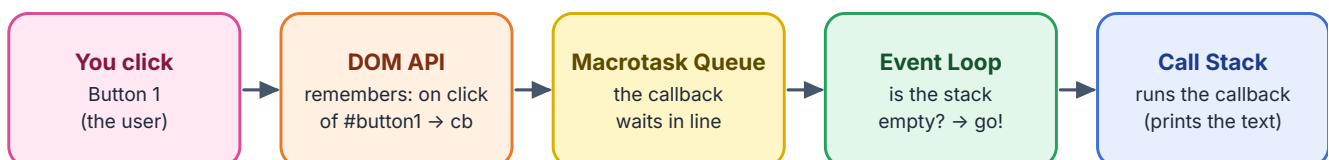
OUTPUT

```

Hi Ji
I am the end
Button 2 is clicked ← only when you click the orange button
Button 1 is clicked ← ...in whatever order you click
Button 1 is clicked ← every click prints again

```

- `document.getElementById("button1")` finds the button element on the page (DOM API).
- `addEventListener('click', callback)` does **not** run the callback. It **registers** it with the browser: “when this button is clicked, call this function”. Then JavaScript moves on — that’s why “I am the end” is printed right away.
- Each time you click, the browser puts the callback into the **macrotask queue**, and the event loop runs it when the stack is empty.
- Unlike `setTimeout` (runs once), a listener runs **every time** the event happens.



Console: "Button 1 is clicked"
...and this happens again for EVERY click.

Figure 3.4 What happens when you click Button 1.

Experiment 4 — the same idea again

JAVASCRIPT Day 17 · first.js — experiment 4

```
console.log("I am first");

setTimeout(() => {
  console.log("I am timeOut");
}, 5000);

console.log("I am end");
```

OUTPUT

```
I am first
I am end
I am timeOut ← 5 seconds later
```

Experiment 5 — **fetch** (the code that is active in the file)

JAVASCRIPT Day 17 · first.js — experiment 5 (not commented out)

```
console.log("Start the Operation");

fetch("https://api.github.com/users").then(() => {
  console.log("Git Hub user info");
})

console.log("end of operation");
```

OUTPUT

```
Start the Operation
end of operation
Git Hub user info ← when GitHub's server replies (a few hundred ms later)
```

- `fetch(url)` asks the **browser** to send a network request to GitHub's server. It immediately returns a Promise (a ticket).
- `.then(callback)` says: "when the response arrives, run this callback".
- JavaScript doesn't wait for the internet — it prints `"end of operation"` first. When the reply arrives, the callback becomes a **microtask** and runs.

The other small experiments at the bottom of the file

JAVASCRIPT Day 17 · first.js — other experiments

```
let arr = [100, 20, 90, 70];
arr.push(10);           // [100, 20, 90, 70, 10] — normal synchronous work
arr.map(x => x * 10);    // creates [1000, 200, 900, 700, 100] (result not stored)

function hello() {
  console.log("Hi ji");
}
hello();                // a frame for hello() is pushed on the call stack, then popped
console.log("Hello Ji");

ahsdjsah("Hello Ji");  // ❌ ReferenceError: ahsdjsah is not defined
```

These show that array work and normal function calls are plain **synchronous** code on the call stack. The last line calls a function that doesn't exist, so JavaScript throws a **ReferenceError** and the script **stops there** — an error in synchronous code stops everything after it.

3.10 Things to Remember

Things to Remember

- JavaScript runs your code on **one thread** with **one call stack** (push on call, pop on return — like C++).
- Synchronous code **blocks**: a long loop freezes the whole page.
- Timers, network requests and DOM events are handled by the **browser (Web APIs)**, outside the JS thread.
- When their work is done, their callbacks wait in a **queue**: Promise callbacks → **microtask queue**; `setTimeout` / `setInterval` / events → **macrotask queue**.
- **Event loop rule**: when the stack is empty → run **all** microtasks → then **one** macrotask → repeat.
- Order to remember: **synchronous code** → **microtasks (Promises)** → **macrotasks (setTimeout)**.
- The `setTimeout` delay is a **minimum**, not an exact time.
- **JavaScript never waits**: an async function returns a *ticket* (timer ID / Promise) immediately; the result comes later via a callback.
- A `setTimeout` callback runs **once**; an event-listener callback runs **every time** the event happens.
- An **API** = interface + abstraction + contract.

 Predict the output!

1. What is printed?

JAVASCRIPT

```
console.log("X");  
setTimeout(() => console.log("Y"), 0);  
console.log("Z");
```

2. What is printed?

JAVASCRIPT

```
setTimeout(() => console.log("timeout"), 0);  
Promise.resolve().then(() => console.log("promise"));  
console.log("sync");
```

3. What is printed?

JAVASCRIPT

```
console.log(1);  
setTimeout(() => console.log(2), 1000);  
setTimeout(() => console.log(3), 0);  
Promise.resolve().then(() => console.log(4));  
console.log(5);
```

4. What does `setTimeout(...)` return: the result of the callback, or something else?

5. A button has a click listener. The user clicks it while a 5-second loop is running. When does the listener run?

 Answers

- `X`, `Z`, `Y`
- `sync`, `promise`, `timeout` — synchronous first, then microtasks, then macrotasks.
- `1`, `5`, `4`, `3`, `2` — sync (1, 5), microtask (4), then timers: the 0 ms timer (3) before the 1000 ms timer (2).
- A **timer ID** (a ticket). The callback's result is not returned to you.
- Only **after the loop finishes** — the click callback waits in the macrotask queue until the call stack is empty.

CHAPTER 4 Lecture 18 · Day 18 code



Callbacks and Callback Hell

IN THIS CHAPTER YOU WILL LEARN

- Why async code needs **callbacks**
- The **Zomato order** story: 4 steps, each takes time
- The full **Day 18** code, line by line, with a timeline
- Why we can't just call the steps one after another
- Why nesting works (Russian dolls)
- The **Pyramid of Doom** and the 8 problems of callback hell

4.1 The problem: some work takes time

JavaScript is **single-threaded** — imagine a single worker who can do only **one** task at a time:

JAVASCRIPT

```
console.log("Task 1");
console.log("Task 2");
console.log("Task 3");
// Executes line by line, top to bottom
```

But some operations take time:

- Making **API calls** (network requests)
- **Reading files**
- **Database queries**
- **Timers**

If JavaScript **waited** for these, everything would freeze:

JAVASCRIPT

```
// ❌ BAD - imaginary blocking function: everything stops here
let data = waitForAPI(); // blocks for 3 seconds
console.log(data);
console.log("This waits 3 seconds too!");
```

The solution: asynchronous programming. Don't wait — instead say “call me back when you're done”:

JAVASCRIPT

```
// ✅ GOOD - non-blocking (fetchAPI is an imaginary async function)
fetchAPI(function () {
  console.log("Data received!");
});
console.log("I run immediately!");
```

OUTPUT

```
I run immediately!  
Data received! ← after the delay
```

This “call me back” function is a **CALLBACK**.

4.2 What exactly is a callback?

Callback

A **callback** is a function that you pass **as an argument** to another function. The other function decides **when** to call it — immediately, or later when some work is finished.

A **synchronous** callback — it is called right away:

JAVASCRIPT

```
function greetUser(name, callback) {  
  console.log(`Hi ${name}`);  
  callback(); // call the function we were given  
}  
  
greetUser("Rohit", () => console.log("Welcome to the course!"));
```

OUTPUT

```
Hi Rohit  
Welcome to the course!
```

An **asynchronous** callback — it is called later:

JAVASCRIPT

```
function afterTwoSeconds(callback) {  
  setTimeout(() => {  
    callback("done!"); // we can also PASS DATA to the callback  
  }, 2000);  
}  
  
afterTwoSeconds(result => console.log("Result:", result));  
console.log("waiting...");
```

OUTPUT

```
waiting...  
Result: done! ← 2 seconds later
```

Notice the second example: the function that **calls** the callback decides **what data to give it** (`"done!"`). The Day 18 code uses exactly this to pass the order details from one step to the next.

Callbacks in C++

A callback in C++ is a **function pointer**, a `std::function`, or a **lambda** passed as an argument. You have used one with `std::sort`:

C++

```
// std::sort calls YOUR lambda back again and again:
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
```

The difference in JavaScript: callbacks are used for **time-based** work too ("call me after 3 seconds", "call me when the server replies"), because JavaScript's single thread must never wait.

4.3 The real-world story: ordering food on Zomato

Think of ordering food online:

1. **Place order** → wait → payment done, order placed
2. **Prepare food** → wait → food ready
3. **Pick up order** → wait → the delivery partner has it
4. **Deliver order** → wait → food delivered 🍕

Each step:

- **takes time** (asynchronous),
- **depends on the previous step** finishing,
- **produces data** needed by the next step.

You can't prepare food before the order is placed! You can't pick up food before it's prepared! This is called **sequential async operations** — and it is the root of callback hell.



Each step: takes time · needs the previous step · adds data for the next step

The SAME orderDetail object is passed along — every step can see what earlier steps added.

Figure 4.1 The four steps. One `orderDetail` object travels through all of them, and each step adds some information to it.

4.4 The Day 18 code, step by step

Day 18 — 03JS/Day18/index.js

This is the teacher's "Zomato application". Each step uses `setTimeout(..., 3000)` to **pretend** that it takes 3 seconds (a real app would talk to a payment gateway, the restaurant, the delivery partner...).

The order object

JAVASCRIPT Day 18 · index.js

```
// zomato application

const orderDetail = {
  orderId: 123123,
  food: ["Pizza", "biryani", "coke"],
  cost: 620,
  customer_name: "Rohit",
  customer_location: "Dwarka",
  restaurant_location: "Delhi"
}
```

This object holds all the order information. It will flow through **every** step, and each step will add a new property to it.

Step 1: place the order (payment)

JAVASCRIPT Day 18 · index.js

```
function placedOrder(orderDetail, Callback) {
  console.log(`${orderDetail.cost} Payment is in progress`);

  setTimeout(() => {
    console.log("Payment is received and order get placed");
    orderDetail.status = true; // ✅ add new data
    Callback(orderDetail);    // 🏃 pass the order to the NEXT step
  }, 3000)
}
```

What happens when this function is called:

1. It **immediately** prints `620 Payment is in progress`.
2. It hands a **3-second timer** to the browser (this simulates the payment taking time) — and the function **returns right away**.
3. After 3 s, the timer's callback runs: it prints `Payment is received and order get placed` and adds `status: true` to the order.
4. It calls `Callback(orderDetail)` — "payment is done, here is the updated order, do the next thing".

`Callback` is just the parameter's name (the teacher wrote it with a capital C). You could call it `next`, `done` or `cb` — it is simply "the function to run after me".

Step 2: prepare the food

JAVASCRIPT Day 18 · index.js

```
function preparingOrder(orderDetail, Callback) {
  console.log(`Your food preparation started of ${orderDetail.food}`);

  setTimeout(() => {
    console.log("Your order is now prepared");
    orderDetail.token = 123; // ✅ add a token number
    Callback(orderDetail)   // 🏹 pass to the next step
  }, 3000);
}
```

Key point: this function receives the `orderDetail` that already has `status: true` from the previous step. And `${orderDetail.food}` prints the array joined with commas: `Pizza,biryani,coke`.

Step 3: pick up the order

JAVASCRIPT Day 18 · index.js

```
function pickupOrder(orderDetail, Callback) {
  console.log(`Delivery boy is on way to pickup order from ` +
    `${orderDetail.restaurant_location} `);

  setTimeout(() => {
    console.log("I have picked up the order");
    orderDetail.received = true; // ✅ mark as picked up
    Callback(orderDetail);      // 🏹 pass to the next step
  }, 3000);
}
```

Now `orderDetail` has `status`, `token` and `received` !

Step 4: deliver the order

JAVASCRIPT Day 18 · index.js

```
function deliverOrder(orderDetail) {
  console.log(`I am on my way to deliver order ${orderDetail.customer_location}`);

  setTimeout(() => {
    console.log("Order delivered succesfully");
    orderDetail.delivery = true; // ✅ final status
  }, 3000)
}
```

This is the **last step**, so it doesn't need a callback — nothing comes after it.

Putting it together: the callback-hell structure

JAVASCRIPT Day 18 · index.js

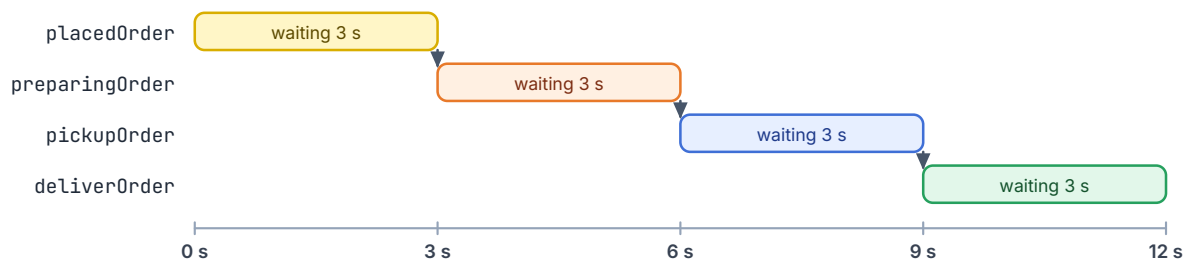
```
placedOrder(orderDetail, (orderDetail) => {           // ↓ Level 1: after payment
  preparingOrder(orderDetail, (orderDetail) => {       // ↓ Level 2: after preparation
    pickupOrder(orderDetail, (orderDetail) => {        // ↓ Level 3: after pickup
      deliverOrder(orderDetail);                      // ↓ Level 4: final delivery
    });
  });
});
});
```

See the pattern? Each step is **nested** inside the previous step's callback.

OUTPUT What you see in the console (verified by running the file)

```
620 Payment is in progress ← 0 s
Payment is received and order get placed ← 3 s
Your food preparation started of Pizza,biryani,coke ← 3 s
Your order is now prepared ← 6 s
Delivery boy is on way to pickup order from Delhi ← 6 s
I have picked up the order ← 9 s
I am on my way to deliver order Dwarka ← 9 s
Order delivered succesfully ← 12 s
```

✓ Nested callbacks (Day 18 code)



✗ Called one after another (wrong)



Figure 4.2 Top: the nested version — each step starts only when the previous step calls its callback. Bottom: calling the functions one after another (next section) — everything starts at 0 s.

🧠 One object, shared by all steps (like a pointer)

Remember from Chapter 0: objects are passed **by reference**. Every step receives the **same** `orderDetail` object — like passing the same pointer `Order*` to four functions in C++. That's why a property added in step 1 (`status`) is visible in step 2, 3 and 4.

The arrow functions' parameter is also named `orderDetail`. Inside each arrow function, this parameter **shadows** (hides) the outer `const orderDetail` — just like an inner-scope variable with the same name in C++. Here it doesn't matter, because they all point to the same object anyway.

4.5 Why can't we just call them one after another?

It looks simpler to write:

JAVASCRIPT

```
placedOrder(orderDetail, preparingOrder);
preparingOrder(orderDetail, pickupOrder);
pickupOrder(orderDetail, deliverOrder);
```

Problem: these three lines all execute **immediately**, one after another, **without waiting**. JavaScript doesn't know it should wait for `placedOrder` to finish before calling `preparingOrder` — `placedOrder` returned as soon as it handed its timer to the browser!

OUTPUT

Real output of the wrong version

```
620 Payment is in progress ← 0 s
Your food preparation started of Pizza,biryani,coke ← 0 s — before payment!
Delivery boy is on way to pickup order from Delhi ← 0 s — before cooking!
Payment is received and order get placed ← 3 s
Your food preparation started of Pizza,biryani,coke ← 3 s — AGAIN
...
Uncaught TypeError: Callback is not a function ← 6 s — crash!
```

Everything starts at 0 s, some steps run **twice** (once directly, once as a callback), and then it **crashes**: when `placedOrder` calls `preparingOrder(orderDetail)` as its callback, it passes only **one** argument — so inside `preparingOrder`, `Callback` is `undefined`, and calling `undefined(...)` is a `TypeError`.

4.6 Why nesting works

JAVASCRIPT

```
placedOrder(orderDetail, (orderDetail) => {  
  // This function doesn't run immediately.  
  // It WAITS for placedOrder to finish, THEN it runs with the updated orderDetail.  
  preparingOrder(orderDetail, (orderDetail) => {  
    // This WAITS for preparingOrder to finish, then runs with the updated orderDetail.  
    pickupOrder(orderDetail, (orderDetail) => {  
      // And so on...  
      deliverOrder(orderDetail);  
    });  
  });  
});
```

The intuition:

- Each callback is a **“what to do NEXT”** instruction.
- It is wrapped **inside** the previous step, so it waits.
- It is a chain of *“when you’re done, do this”*.

💡 Russian nesting dolls 🍷

Open doll 1 → find doll 2 inside. Open doll 2 → find doll 3 inside. Open doll 3 → find doll 4 inside. Each step **reveals** the next step. Nested callbacks work the same way: finishing step 1 is what “opens” step 2.

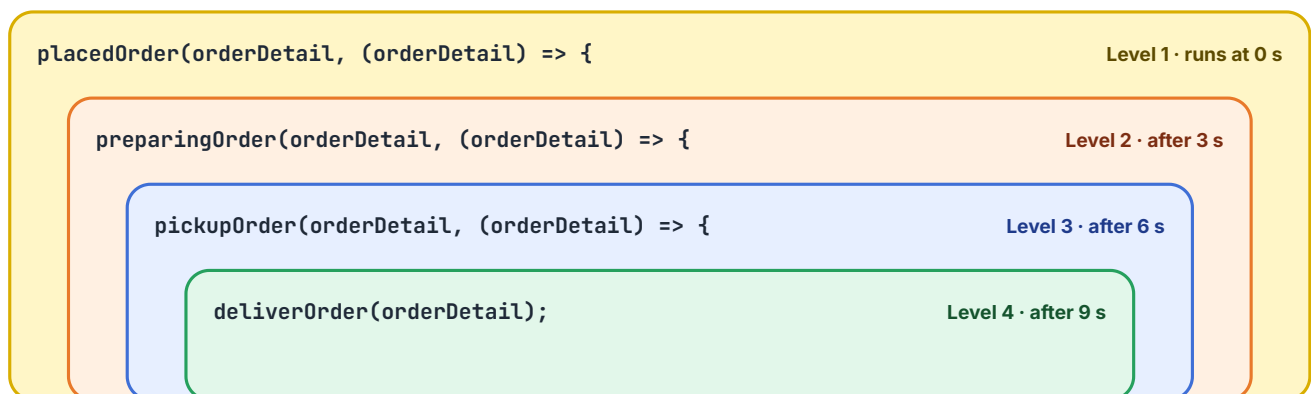


Figure 4.3 Every callback lives inside the previous one — that’s why the code keeps moving to the right.

4.7 The problems with callback hell

1. Readability — the Pyramid of Doom

JAVASCRIPT

```
step1((data) => {  
  step2(data, (data) => {  
    step3(data, (data) => {  
      step4(data, (data) => {  
        step5(data, (data) => {  
          step6(data, (data) => {  
            // Code keeps moving right →→→  
          });  
        });  
      });  
    });  
  });  
});
```

- The code doesn't read naturally from top to bottom.
- It is hard to see the flow.
- The indentation grows until it is unmanageable.

2. Error handling nightmare

What if the payment fails? What if the kitchen is closed? What if the driver cancels? You must check for errors **at every level**:

JAVASCRIPT

```
placedOrder(orderDetail, (orderDetail, error) => {
  if (error) {
    console.log("Payment failed:", error);
    return; // ❌ but what about refund? notification?
  }
  preparingOrder(orderDetail, (orderDetail, error) => {
    if (error) {
      console.log("Kitchen error:", error);
      return; // ❌ need to cancel order, refund payment
    }
    pickupOrder(orderDetail, (orderDetail, error) => {
      if (error) {
        console.log("Pickup failed:", error);
        return; // ❌ food is ready but stuck!
      }
      deliverOrder(orderDetail, (error) => {
        if (error) {
          console.log("Delivery failed:", error); // ❌ customer charged, food gone
        }
      });
    });
  });
});
```

Problems: error handling at **every** level, lots of repeated code, proper error recovery is hard, and one error can leave the system in an inconsistent state.

3. Hard to modify

Want to add a “Send SMS notification” step between pickup and delivery?

JAVASCRIPT

```
placedOrder(orderDetail, (orderDetail) => {
  preparingOrder(orderDetail, (orderDetail) => {
    pickupOrder(orderDetail, (orderDetail) => {
      // 🆕 NEW STEP – must break and re-nest everything below it
      sendSMS(orderDetail, (orderDetail) => {
        deliverOrder(orderDetail);
      });
    });
  });
});
```

You have to break the chain, re-indent everything, and it's easy to introduce bugs — touching old code is risky.

4. You can't use normal control flow (try / catch, return)

try / catch doesn't work:

JAVASCRIPT

```
try {
  placedOrder(orderDetail, (orderDetail) => {
    throw new Error("Payment failed"); // ❌ thrown LATER, after try/catch is done
  });
} catch (error) {
  console.log("Won't catch it!"); // this never runs
}
```

Why? Think about the call stack (Chapter 3). The `try` block only calls `placedOrder`, which starts a timer and returns immediately — so the `try / catch` is **finished and gone** from the stack. Three seconds later, the callback runs from the event loop with a fresh, empty stack. There is no `try` around it anymore.

return doesn't work as expected:

JAVASCRIPT

```
function processOrder() {
  placedOrder(orderDetail, (orderDetail) => {
    if (orderDetail.cost > 1000) {
      return "Too expensive"; // ❌ this only exits the callback, not processOrder
    }
    preparingOrder(orderDetail, (orderDetail) => {
      // This still runs even though we "returned"!
    });
  });
  return "Order processed"; // ❌ this returns IMMEDIATELY, before the order even starts
}
```

5. Variable scope confusion

JAVASCRIPT

```
let finalStatus;

placedOrder(orderDetail, (orderDetail) => {
  preparingOrder(orderDetail, (orderDetail) => {
    pickupOrder(orderDetail, (orderDetail) => {
      deliverOrder(orderDetail);
      finalStatus = "Delivered"; // set it here...
    });
  });
});

console.log(finalStatus); // ❌ undefined! The callbacks haven't run yet
```

It is hard to get data **out** of a callback chain — the last line runs at 0 s, long before the callbacks.

6. Debugging is painful

When an error happens deep inside the chain, the stack trace looks like this:

ERROR

```
Error: Delivery failed
  at anonymous (line 45)
  at anonymous (line 38)
  at anonymous (line 31)
  at anonymous (line 24)
```

All functions show as **“anonymous”**, it’s hard to trace which step failed, and the stack trace doesn’t show the flow.

7. Testing is difficult

You can’t easily test `preparingOrder` alone — it is buried inside `placedOrder`’s callback. You can’t mock just one step, and you can’t easily test error scenarios.

8. Parallel operations are messy

What if you want to **prepare food AND assign a driver at the same time**, and continue only when **both** are done?

JAVASCRIPT

```
// With callbacks, you need manual coordination:
let foodReady = false;
let driverAssigned = false;

prepareFood(() => {
  foodReady = true;
  if (driverAssigned) startDelivery();
});

assignDriver(() => {
  driverAssigned = true;
  if (foodReady) startDelivery();
});
// ❌ Messy! Easy to get wrong! Duplicate code!
```

(In Chapter 7 we’ll see `Promise.all`, which solves this in one line.)

Summary table

Aspect	Problem	Impact
Readability	Pyramid shape, deep nesting	Hard to understand the flow
Error handling	Must repeat at every level	Repetitive, easy to miss
Maintainability	Hard to add/remove steps	Risky to modify
Control flow	Can't use <code>try / catch</code> , <code>return</code>	Can't write normal code
Debugging	Anonymous function traces	Hard to find bugs
Testing	Steps are tightly coupled	Can't test in isolation
Parallelism	Manual coordination needed	Complex, error-prone
Scope	Variables trapped in closures	Data access issues

4.8 Key takeaways

1. **Callback hell exists because:** sequential async operations need to wait for each other, each step needs data from the previous step, and callbacks are the only way to say *"do this next"*.
2. **The nesting happens because:** JavaScript doesn't wait for async operations, so we wrap the next step **inside** the previous step's callback to get the right order.
3. **Why it's called "hell":** unreadable and unmaintainable code, error handling becomes a nightmare, normal JavaScript features (`try / catch`, `return`) don't work, and debugging/testing is very hard.
4. **The fundamental insight:** we are trying to express **sequential** logic ($A \rightarrow B \rightarrow C \rightarrow D$) using **nested** functions (which represent *scope*, not *sequence*). This mismatch creates all the problems.

✓ What's next?

Modern JavaScript has solutions. With **Promises** (Chapter 5) the same order flow becomes a flat, readable chain with **one** place for errors:

JAVASCRIPT

```
placedOrder(orderDetail)
  .then(preparingOrder)
  .then(pickupOrder)
  .then(deliverOrder)
  .catch(error => console.log("Error:", error));
```

And with `async / await` (Chapter 7) it looks almost like normal synchronous code.

4.9 Things to Remember

📌 Things to Remember

- A **callback** = a function passed to another function, to be called now or later. The caller can pass data into it: `Callback(orderDetail)`.
- Async steps that depend on each other must **start inside the previous step's callback** — otherwise they all start at 0 s.
- In the Day 18 code, the **same** `orderDetail` object (a reference) travels through all 4 steps; each step adds a property (`status` , `token` , `received` , `delivery`).
- `${array}` inside a template literal joins the items with commas (`Pizza,biryani,coke`).
- Nesting → **Pyramid of Doom**: hard to read, change, debug and test.
- `try` / `catch` around async code **cannot** catch errors thrown later in callbacks; `return` inside a callback only leaves that callback.
- The fix: **Promises** and **async/await**.

🎯 Quick Quiz

1. In one sentence, what is a callback?
2. In the Day 18 code, why is `deliverOrder` the only function without a `Callback` parameter?
3. What is printed, and when?

JAVASCRIPT

```
function task(name, ms, cb) {
  setTimeout(() => { console.log(name); cb(); }, ms);
}
task("A", 300, () => task("B", 100, () => console.log("done")));
console.log("start");
```

4. Will `"Caught!"` be printed?

JAVASCRIPT

```
try {
  setTimeout(() => { throw new Error("Oops"); }, 0);
} catch (e) {
  console.log("Caught!");
}
```

5. How long does the whole Day 18 order take, and why not 3 seconds?

✓ Answers

1. A function passed as an argument to another function, which calls it back (now or later).
2. Because it is the **last** step — nothing needs to happen after delivery.
3. `start` (0 ms), `A` (≈300 ms), `B` (≈400 ms), `done` (right after B). Task B starts only inside A's callback.
4. **No**. The error is thrown later from the event loop, after the `try / catch` has already finished. It becomes an *uncaught* error.
5. About **12 seconds** (4 steps × 3 s), because each step starts only after the previous step's callback is called.

CHAPTER 5 Lecture 19 · Day 19 code



Promises

IN THIS CHAPTER YOU WILL LEARN

- What a **Promise** is (a placeholder for a future value)
- The 3 states: **pending**, **fulfilled**, **rejected**
- Creating promises with `new Promise((resolve, reject) => ...)`
- Consuming them with `.then()`, `.catch()`, `.finally()`
- **Promise chaining** — the cure for callback hell
- `fetch()`, the `Response` object and `response.json()`
- The **Day 19** code: GitHub avatars + Zomato with Promises

5.1 What is a Promise?

 Promise

The `Promise` object represents the **eventual completion (or failure)** of an asynchronous operation and its resulting value.

In simple words: a Promise is a **placeholder** — an object that says *"I don't have the value yet, but I will give it to you later (or tell you why I couldn't)"*.

Promise characteristics:

1. **A Promise is an object** — you can store it in a variable, pass it around, return it from a function.
2. **It represents a future value** — the value isn't available yet, but will be.
3. **It has states** — it changes state over time (pending → fulfilled or rejected).
4. **One-time use** — once it is settled, it never changes again.

 Tracking an online order

When you order something online, you **immediately** get an **Order ID** — that's the Promise. At first the status is *Pending* (being packed, shipped...). Later it becomes either **Delivered** (you get the parcel = the **value**) or **Cancelled** (you get a **reason**, like "out of stock"). Once it is delivered or cancelled, the status never changes again.

🧠 C++ has promises too: `std::promise` / `std::future`

C++

```
std::promise<int> p;
std::future<int> f = p.get_future();

std::thread worker([&p] {
    std::this_thread::sleep_for(std::chrono::seconds(1));
    p.set_value(42);           // like resolve(42) (set_exception is like reject)
});

int x = f.get();              // 🛑 BLOCKS this thread until the value arrives
worker.join();
```

JAVASCRIPT

```
const p = new Promise(resolve => setTimeout(() => resolve(42), 1000));
p.then(x => console.log(x));           // ✅ does NOT block — "call this later"
console.log("I am not blocked!");     // printed first, then 42
```

Same idea — a *producer* sets the value later, a *consumer* reads it. The **big difference**: C++ `future.get()` **blocks** the thread. JavaScript has no `.get()` — you give a callback to `.then()`, and the single thread keeps working.

5.2 The three states of a Promise

A Promise is in **exactly one** of three states at any time:

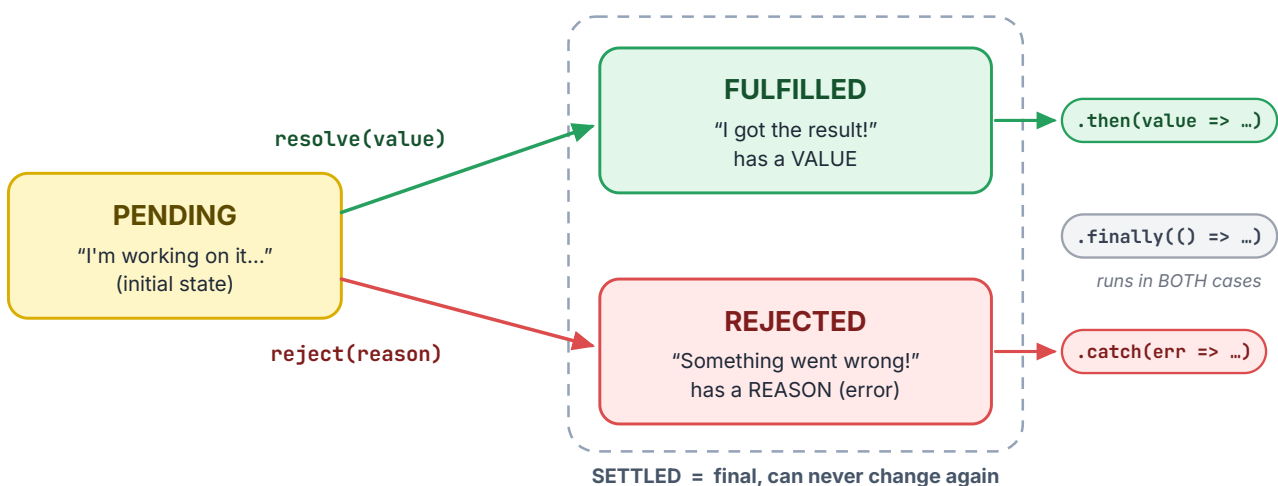


Figure 5.1 The life of a Promise. It starts **pending**, and settles **once** — either **fulfilled** (with a value) or **rejected** (with a reason).

State	Meaning	Caused by	Which handler runs
Pending	"I'm working on it..." (initial state)	—	none yet
Fulfilled	"I got the result!"	calling <code>resolve(value)</code>	<code>.then(...)</code>
Rejected	"Something went wrong!"	calling <code>reject(reason)</code> (or an error is thrown)	<code>.catch(...)</code>

Once a promise is **fulfilled or rejected**, we say it is **settled** — final, it can never change again.

JAVASCRIPT

```
// Create a promise
const myPromise = new Promise((resolve, reject) => {
  // At this moment: the promise is PENDING
  setTimeout(() => {
    const success = true;
    if (success) {
      resolve("Success!"); // → the promise becomes FULFILLED
    } else {
      reject("Failed!"); // → the promise becomes REJECTED
    }
  }, 2000);
});
// myPromise is PENDING for 2 seconds, then becomes FULFILLED with the value "Success!"
```

A promise can settle only ONCE

JAVASCRIPT

```
const promise = new Promise((resolve, reject) => {
  console.log("State: PENDING");
  setTimeout(() => {
    resolve("Done");
    console.log("State: FULFILLED");
    // ✗ These do nothing – the promise is already settled!
    reject("Error"); // ignored!
    resolve("Again"); // ignored!
  }, 1000);
});

promise.then(value => console.log("Value:", value));
```

OUTPUT

```
State: PENDING
State: FULFILLED ← 1 second later
Value: Done
```

Key point: only the **first** `resolve` or `reject` counts. Everything after it is ignored.

5.3 Creating a Promise

JAVASCRIPT

```
const promise = new Promise((resolve, reject) => {  
  // this function is called the "executor"  
  // it runs IMMEDIATELY when the promise is created  
});
```

The **executor** function receives two functions from JavaScript:

- **resolve(value)** — call it when the operation **succeeds**.
- **reject(reason)** — call it when the operation **fails**.

! The executor runs immediately (synchronously)!

Many students think the code inside `new Promise(...)` runs "later". It doesn't — it runs **right away**. Only the `.then` callbacks run later.

JAVASCRIPT

```
console.log("1. before");  
const p = new Promise((resolve) => {  
  console.log("2. inside the executor - runs immediately!");  
  resolve("done");  
});  
console.log("3. after");  
p.then(value => console.log("4. then:", value));
```

OUTPUT

```
1. before  
2. inside the executor - runs immediately!  
3. after  
4. then: done
```

Five small examples (from the lecture)

JAVASCRIPT

```
// Example 1: a promise that immediately resolves with "Hello!"  
const simplePromise = new Promise((resolve, reject) => {  
  resolve("Hello!");  
});  
  
// Example 2: a promise with setTimeout  
const delayedPromise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve("Done after 2 seconds");  
  }, 2000);  
});
```

JAVASCRIPT

```
// Example 3: a promise with conditional logic
function checkAge(age) {
  return new Promise((resolve, reject) => {
    if (age >= 18) {
      resolve("Access granted");
    } else {
      reject("Access denied - Too young");
    }
  });
}

const ageCheck = checkAge(20); // returns a promise
```

JAVASCRIPT

```
// Example 4: real-world style - simulating an API call
function fetchUserData(userId) {
  return new Promise((resolve, reject) => {
    console.log(`Fetching user ${userId}...`);
    setTimeout(() => {
      // Simulate a successful API response
      const user = { id: userId, name: "John Doe", email: "john@example.com" };
      resolve(user);
    }, 2000);
  });
}

// Example 5: a promise with error handling
function divideNumbers(a, b) {
  return new Promise((resolve, reject) => {
    if (b === 0) {
      reject("Cannot divide by zero!");
    } else {
      resolve(a / b);
    }
  });
}
```

Notice the **most common pattern** (Examples 3, 4, 5): a normal function that returns a new *Promise*. The Day 19 Zomato code uses exactly this pattern.

5.4 Consuming a Promise: `.then()`, `.catch()`, `.finally()`

Once you have a promise, you need to **consume** it to get the result.

JAVASCRIPT

```
promise.then((result) => {  
  // runs when the promise is FULFILLED  
  console.log(result);  
});  
  
promise.catch((error) => {  
  // runs when the promise is REJECTED  
  console.error(error);  
});  
  
promise.finally(() => {  
  // runs ALWAYS – whether fulfilled or rejected  
  console.log("Promise settled");  
});
```

Complete example

JAVASCRIPT

```
function fetchData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      const success = Math.random() > 0.5; // 50% chance  
      if (success) {  
        resolve({ data: "Some data" });  
      } else {  
        reject("Network error");  
      }  
    }, 1000);  
  });  
}  
  
// Consuming the promise:  
fetchData()  
  .then((result) => {  
    console.log("Success:", result);  
  })  
  .catch((error) => {  
    console.error("Error:", error);  
  })  
  .finally(() => {  
    console.log("Request completed");  
  });
```

OUTPUT If it succeeds

```
Success: { data: 'Some data' }  
Request completed
```

OUTPUT If it fails

```
Error: Network error  
Request completed
```

`Math.random()` gives a random decimal between 0 (included) and 1 (not included), so `Math.random() > 0.5` is `true` about half of the time.

Using `checkAge` from Example 3:

JAVASCRIPT

```
checkAge(20)
  .then(message => console.log(message))    // Access granted
  .catch(error => console.log(error));

checkAge(15)
  .then(message => console.log(message))
  .catch(error => console.log(error));    // Access denied - Too young
```

.then() can take TWO functions**JAVASCRIPT**

```
promise.then(
  (result) => { console.log("Success:", result); },    // success handler
  (error) => { console.error("Error:", error); }        // error handler (optional)
);

// But it's better (clearer) to use .catch() for errors:
promise
  .then((result) => { console.log("Success:", result); })
  .catch((error) => { console.error("Error:", error); });
```

5.5 Promise chaining — the power of Promises

This is where promises truly shine: **flat, sequential async operations** — no pyramid!

 The 4 rules of chaining

1. `.then()` always returns a **NEW** promise. That's why you can write `.then(...).then(...)`.
2. Whatever you **return** from a `.then` callback becomes the value for the next `.then`.
3. If you return a **Promise**, the next `.then` **waits** for it to settle and receives its value.
4. If anything **throws** (or a promise **rejects**), JavaScript **skips** the following `.then` s and jumps to the nearest `.catch()`.

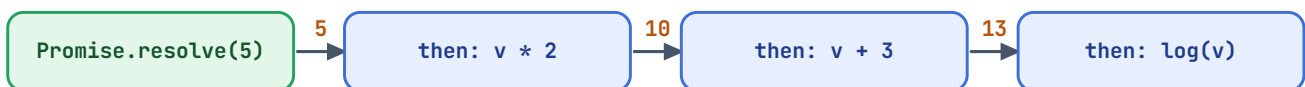
Example 1: basic chaining

JAVASCRIPT

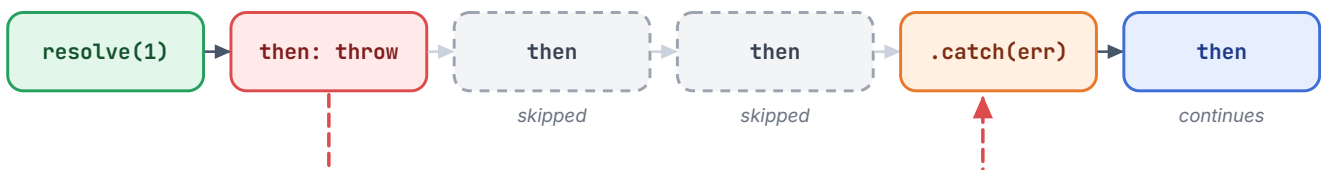
```
Promise.resolve(5)
  .then((value) => {
    console.log(value); // 5
    return value * 2;
  })
  .then((value) => {
    console.log(value); // 10
    return value + 3;
  })
  .then((value) => {
    console.log(value); // 13
  });
```

`Promise.resolve(5)` is a shortcut for “a promise that is already fulfilled with 5” (and `Promise.reject(x)` makes an already-rejected one).

Values flow down the chain



An error jumps straight to .catch()



the error skips every `.then()` until it finds a `.catch()`

After `.catch()` handles it, the chain continues normally.

Figure 5.2 Top: each `.then` receives what the previous one returned. Bottom: when a step throws, the error skips all the remaining `.then`s and lands in `.catch`.

Example 2: chaining real async steps

JAVASCRIPT

```
function step1() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 1 complete");
      resolve("Result from step 1");
    }, 1000);
  });
}

function step2(previousResult) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 2 complete, got:", previousResult);
      resolve("Result from step 2");
    }, 1000);
  });
}
```

```

    }, 1000);
  });
}
function step3(previousResult) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 3 complete, got:", previousResult);
      resolve("Final result");
    }, 1000);
  });
}

// ✅ Clean chain (no nesting!)
step1()
  .then((result1) => {
    return step2(result1);    // returning a promise → the next .then waits for it
  })
  .then((result2) => {
    return step3(result2);
  })
  .then((finalResult) => {
    console.log("All done:", finalResult);
  })
  .catch((error) => {
    console.error("Something failed:", error);
  });

```

OUTPUT

```

Step 1 complete ← 1 s
Step 2 complete, got: Result from step 1 ← 2 s
Step 3 complete, got: Result from step 2 ← 3 s
All done: Final result

```

Even cleaner — you can pass the function itself, because `.then(step2)` will call `step2(result)` for you:

JAVASCRIPT

```

step1()
  .then(step2)
  .then(step3)
  .then((finalResult) => console.log("All done:", finalResult))
  .catch((error) => console.error("Something failed:", error));

```

Errors jump to `.catch()`

JAVASCRIPT

```
Promise.resolve(1)
  .then(x => { console.log("step 1:", x); return x + 1; })
  .then(x => { throw new Error("Boom at step 2"); })
  .then(x => { console.log("step 3 (skipped!)"); })
  .catch(err => console.log("Caught:", err.message))
  .then(() => console.log("the chain continues after catch"));
```

OUTPUT

```
step 1: 1
Caught: Boom at step 2
the chain continues after catch
```

One `.catch()` at the end handles errors from **every** step above it. Compare that with callback hell, where we had to check for errors at every level!

! Forgetting to `return` inside `.then`

JAVASCRIPT

```
fetch(url)
  .then(response => { response.json(); }) // ✗ braces but no return!
  .then(data => console.log(data));      // undefined
```

Fix: `.then(response => response.json())` (no braces) or
`.then(response => { return response.json(); })`.

5.6 The `.finally()` method

`.finally()` runs **regardless** of whether the promise succeeds or fails. Use it for **cleanup code** that must always run:

- hide loading spinners
- close database connections
- re-enable buttons
- stop timers
- release resources

JAVASCRIPT

```
// Example 1: loading spinner
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = Math.random() > 0.5;
      if (success) {
        resolve({ data: "Data loaded" });
      } else {
        reject("Failed to load");
      }
    }, 2000);
  });
}

// Show spinner
console.log("🔄 Loading...");
let isLoading = true;

fetchData()
  .then((result) => {
    console.log("✅ Success:", result);
  })
  .catch((error) => {
    console.error("❌ Error:", error);
  })
  .finally(() => {
    // Always runs - hide the spinner
    isLoading = false;
    console.log("🛑 Loading stopped");
  });
```

JAVASCRIPT

```
// Example 2: database connection (connectToDatabase is imaginary)
function queryDatabase(query) {
  let connection;
  return connectToDatabase()
    .then((conn) => {
      connection = conn;
      return connection.execute(query);
    })
    .then((results) => {
      console.log("Query results:", results);
      return results;
    })
    .catch((error) => {
      console.error("Query failed:", error);
      throw error;
    })
    .finally(() => {
      // Always close the connection
      if (connection) {
        connection.close();
        console.log("Connection closed");
      }
    });
}
```

Key characteristics of **.finally()**

1. It doesn't receive any arguments — it doesn't know if the promise succeeded or failed.

JAVASCRIPT

```
promise
  .then((result) => console.log("Result:", result)) // has the result
  .catch((error) => console.log("Error:", error))   // has the error
  .finally(() => console.log("Done"));              // no arguments!
```

2. It doesn't change the promise's value — the value passes through it.

JAVASCRIPT

```
Promise.resolve("Original")
  .finally(() => {
    return "Modified"; // ignored!
  })
  .then((value) => {
    console.log(value); // "Original"
  });
```

3. But if `.finally()` throws, that error propagates.

JAVASCRIPT

```
Promise.resolve("Success")
  .finally(() => {
    throw new Error("Cleanup failed");
  })
  .then((value) => {
    console.log(value);           // SKIPPED
  })
  .catch((error) => {
    console.error(error.message); // "Cleanup failed"
  });
```

5.7 Promises and the event loop

Remember Chapter 3: `.then`, `.catch` and `.finally` callbacks are **microtasks**. They go into the **microtask queue**, which the event loop empties **before** it runs any `setTimeout` callback. That's why `Promise.resolve().then(...)` prints before `setTimeout(..., 0)`.

Also remember: a `.then` callback **never** runs immediately, even if the promise is already fulfilled. It always waits until the current synchronous code has finished.

5.8 `fetch()` — a real Promise from the browser

`fetch(url)` asks the browser to send a network request. It **immediately** returns a Promise, which later fulfills with a `Response` object.

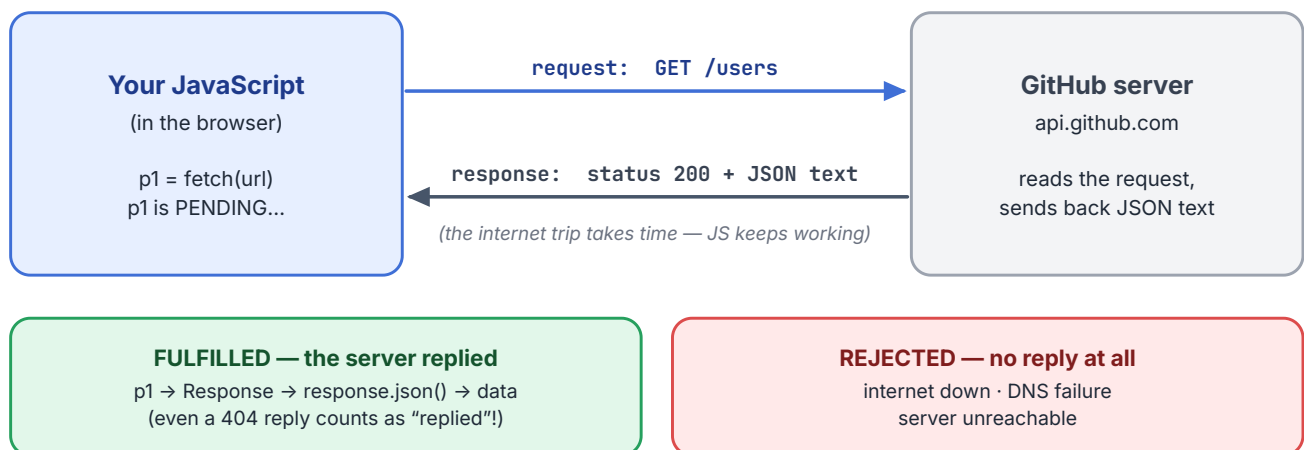


Figure 5.3 Your code talks to GitHub's server. `p1` stays pending until the server answers (fulfilled) — or until it's clear the request can't be completed at all (rejected).

The `Response` object is **not** your data yet. The body arrives as a stream of text, and reading + parsing it also takes time — so `response.json()` returns another Promise. That's why you always see two `.then` steps with `fetch`:

JAVASCRIPT

```
fetch("https://api.github.com/users")
  .then(response => {
    // response is a Response object: status, ok, headers, body...
    console.log(typeof response); // "object"
    return response.json();       // read the body + parse JSON → a Promise
  })
  .then(data => {
    // Now 'data' is a real JavaScript array of user objects
    console.log(data[0].login);    // "mojombo" (the first GitHub user)
  });
```

Useful parts of a `Response`:

Property / method	What it gives you
<code>response.ok</code>	<code>true</code> if the status is 200–299 (success), otherwise <code>false</code>
<code>response.status</code>	the HTTP status code: <code>200</code> , <code>404</code> (not found), <code>500</code> (server error)...
<code>response.json()</code>	a Promise → the body parsed as JSON (a JS object/array)
<code>response.text()</code>	a Promise → the body as a plain string

⚠ `fetch()` does NOT reject on 404 or 500!

`fetch()` only rejects on **network failures** (no internet, DNS failure, server unreachable). If the server **answers** with an error status like 404 or 500, the promise is still **fulfilled** — with `response.ok === false`. You must check it yourself and `throw`:

JAVASCRIPT

```
fetch(url)
  .then((response) => {
    if (!response.ok) {
      // → jumps to .catch:
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    return response.json();
  })
  .then((data) => console.log(data))
  .catch((error) => console.log(error.message));
```

The Day 19 code does exactly this.

5.9 Day 19 code, experiment by experiment

 **Day 19** — 03JS/Day19/index.html + index.js

The page is just an empty box where the code will put the GitHub avatars:

```
HTML 03JS/Day19/index.html
<body>
  <div id="first">
  </div>
</body>
<script src="./index.js"></script>
```

Experiment 1 — **fetch** with two separate promises

JAVASCRIPT Day 19 · index.js — experiment 1

```
const p1 = fetch("https://api.github.com/users");

// fulfilled , reject

const p2 = p1.then((response) => {
  return response.json();
});

p2.then((data) => {
  console.log(data);
});
```

- **p1** is the Promise from **fetch**. It will be **fulfilled** with a **Response** (or **rejected** if there's no network — that's what the teacher's comment **// fulfilled , reject** reminds us).
- **p1.then(...)** returns a **new** promise, **p2**. Because the callback **returns** **response.json()** (itself a promise), **p2** waits for the JSON parsing and is fulfilled with the **data**.
- **p2.then(...)** finally prints the data: an array of 30 GitHub users.

CONSOLE (BROWSER)

```
(30) [{...}, {...}, {...}, ...]
0: {login: 'mojombo', id: 1, avatar_url: '...', html_url: '...', ...}
1: {login: 'defunkt', id: 2, avatar_url: '...', html_url: '...', ...}
...
```

Experiment 2 — the same thing, chained

JAVASCRIPT Day 19 · index.js — experiment 2

```
fetch("https://api.github.com/users")
  .then((response) => {
    return response.json();
  })
  .then((data) => {
    console.log(data);
  })
```

Same result, but without the extra variables `p1` and `p2`. This is how you will normally write it.

Experiment 3 — making our own Promise

JAVASCRIPT Day 19 · index.js — experiment 3

```
const p1 = new Promise((resolve, reject) => {

  resolve({
    name: "Rohit",
    age: 30,
  });
});

p1.then((response) => {
  console.log(response);
}).catch((error) => {
  console.log(error);
})
```

OUTPUT

```
{ name: 'Rohit', age: 30 }
```

The executor calls `resolve` with an object, so the promise is **fulfilled** and `.then` receives that object. `.catch` doesn't run. Try changing `resolve({...})` to `reject("Something went wrong")` — now only `.catch` runs and prints the message.

Experiment 4 — show GitHub avatars on the page

JAVASCRIPT Day 19 · index.js — experiment 4

```

fetch("https://api.github.com/users")
  .then((response) => {

    console.log(response);
    if (!response.ok) {
      throw new Error("Data is not present in server");
    }
    return response.json();
  })
  .then((data) => {
    // console.log(data);

    const parent = document.getElementById("first");

    for (let i = 0; i < data.length; i++) {
      const image = document.createElement('img');
      image.src = data[i].avatar_url;
      image.style.height = "40px";
      image.style.width = "40px";

      parent.append(image);
    }
  })
  .catch((error) => {
    const parent = document.getElementById("first");
    parent.textContent = error.message;
  })

```

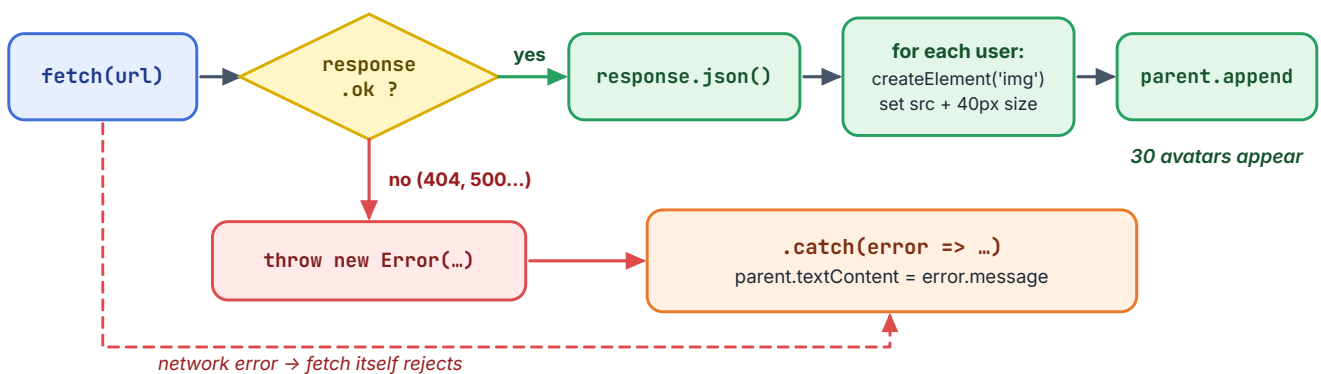


Figure 5.4 The path through experiment 4: success draws 30 avatars; an HTTP error or a network error ends in `.catch`, which writes the message on the page.

What each DOM line does:

Code	Meaning
<code>document.getElementById("first")</code>	find the <code><div id="first"></code> on the page
<code>document.createElement('img')</code>	create a brand-new <code></code> element (not on the page yet)
<code>image.src = data[i].avatar_url</code>	which picture to show (each user's avatar URL)
<code>image.style.height = "40px"</code>	set CSS directly on the element (40 × 40 pixels)
<code>parent.append(image)</code>	put the image inside the div → now it appears on the page
<code>parent.textContent = error.message</code>	replace everything in the div with the error text

Result: 30 small avatars appear in a row. Now try these experiments:

- Change the URL to `https://api.github.com/userss` → GitHub answers **404** → `response.ok` is `false` → we **throw** → `.catch` puts **"Data is not persent in server"** on the page.
- Turn off your internet → `fetch` **rejects** → `.catch` shows the browser's message, e.g. **"Failed to fetch"**.

The line `console.log(response)` shows the Response object, something like

```
Response {type: 'cors', url: 'https://api.github.com/users', status: 200, ok: true, ...}.
```

Experiment 5 — JSON

The next part of the file (`JSON.stringify` / `JSON.parse`) is explained in **Chapter 6**.

Experiment 6 — the Zomato app, rebuilt with Promises

This is the Day 18 app again, but now **each step returns a Promise** instead of taking a callback. There is one more new idea: steps can now **fail**. `Math.random()` is used to simulate failures.

JAVASCRIPT Day 19 · index.js — experiment 6 (the active code)

```
const orderDetail = {
  orderId: 123123,
  food: ["Pizza", "biryani", "coke"],
  cost: 620,
  customer_name: "Rohit",
  customer_location: "Dwarka",
  restaurant_location: "Delhi"
}

function placedOrder(orderDetail) {
  console.log(`${orderDetail.cost} Payment is in progress`);

  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() > 0.1) { // 90% chance of success
        console.log("Payment is received and order get placed");
        orderDetail.status = true;
        resolve(orderDetail); // success: pass the order on
      }
    }, 1000);
  });
}
```

```

    }
    else {
        reject("Payment is failed");           // failure: give a reason
    }
}, 3000)
})
}

function preparingOrder(orderDetail) {
    console.log(`Your food preparation started of ${orderDetail.food}`);

    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (Math.random() > 0.05) {         // 95% chance of success
                console.log("Your order is now prepared");
                orderDetail.token = 123;
                resolve(orderDetail);
            }
            else {
                reject("Food item is not present at restaurant");
            }
        }, 3000);
    })
}

```

JAVASCRIPT Day 19 · index.js — experiment 6 (continued)

```

function pickupOrder(orderDetail) {
    console.log(`Delivery boy is on way to pickup order from ` +
        `${orderDetail.restaurant_location} `);

    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (Math.random() > 0.05) {         // 95% chance of success
                console.log("I have picked up the order");
                orderDetail.received = true;
                resolve(orderDetail);
            }
            else {
                reject("Delivery boy Unable to reach restaurant")
            }
        }, 3000);
    })
}

function deliverOrder(orderDetail) {
    console.log(`I am on my way to deliver order ${orderDetail.customer_location}`);

    return new Promise((resolve, reject) => {
        setTimeout(() => {
            console.log("Order delivered succesfully");
            orderDetail.delivery = true;
            resolve(orderDetail);               // this step never fails
        }, 3000)
    })
}

```

```
placedOrder(orderDetail)
  .then((orderDetail) => preparingOrder(orderDetail))
  .then((orderDetail) => pickupOrder(orderDetail))
  .then((orderDetail) => deliverOrder(orderDetail))
  .then((orderDetail) => {
    console.log(orderDetail);
  })
  .catch((error) => {
    console.log("Error: ", error);
  })
  .finally(() => {
    console.log("I am doing cleanup");
  })
```

How it works:

- Each function prints its first message **immediately**, then returns a **new Promise** that settles after 3 s.
- `.then((orderDetail) => preparingOrder(orderDetail))` **returns** the next step's promise, so the next `.then` **waits** for it (rule 3 of chaining). You could also write it as `.then(preparingOrder)`.
- If **any** step calls `reject(...)`, all the remaining `.then` s are **skipped** and `.catch` prints the reason.
- `.finally` runs at the very end in **both** cases.
- The strange-looking `}).` + new line + `finally(` is fine: JavaScript doesn't care that the dot is at the end of the line.

OUTPUT When everything succeeds (verified by running the file)

```
620 Payment is in progress ← 0 s
Payment is received and order get placed ← 3 s
Your food preparation started of Pizza,biryani,coke ← 3 s
Your order is now prepared ← 6 s
Delivery boy is on way to pickup order from Delhi ← 6 s
I have picked up the order ← 9 s
I am on my way to deliver order Dwarka ← 9 s
Order delivered succesfully ← 12 s
{
  orderId: 123123,
  food: [ 'Pizza', 'biryani', 'coke' ],
  cost: 620,
  customer_name: 'Rohit',
  customer_location: 'Dwarka',
  restaurant_location: 'Delhi',
  status: true,
  token: 123,
  received: true,
  delivery: true
}
I am doing cleanup
```

OUTPUT If the restaurant fails (at 6 s)

```
620 Payment is in progress
Payment is received and order get placed
Your food preparation started of Pizza,biryani,coke
Error: Food item is not persent at restaurant
I am doing cleanup
```

OUTPUT If the payment fails (at 3 s)

```
620 Payment is in progress
Error: Payment is failed
I am doing cleanup
```

(There are **two** spaces after `Error:` because the string `"Error: "` already ends with a space, and `console.log` adds another space between its arguments.)

How often does the whole order succeed? $0.9 \times 0.95 \times 0.95 \approx 0.81$, so about **81 %** of the time. Run the file several times and you'll see different endings!

Callbacks (Day 18) vs Promises (Day 19)

	Day 18 — callbacks	Day 19 — Promises
Function signature	<code>placedOrder(orderDetail, Callback)</code>	<code>placedOrder(orderDetail)</code> → returns a Promise
"I'm done, here's the data"	<code>Callback(orderDetail)</code>	<code>resolve(orderDetail)</code>
"I failed, here's why"	no clean way	<code>reject("Payment is failed")</code>
Shape of the code	nested pyramid	flat chain of <code>.then</code>
Error handling	at every level	one <code>.catch</code> at the end
Cleanup	manual	<code>.finally</code>

5.10 Things to Remember

📌 Things to Remember

- A **Promise** is an object = a placeholder for a future value. States: **pending** → **fulfilled** (`resolve`) or **rejected** (`reject`). Settled = final.
- Only the **first** `resolve` / `reject` counts.
- The **executor** inside `new Promise(...)` runs **immediately**; `.then/.catch/.finally` callbacks run **later** (as microtasks).
- `.then` for success, `.catch` for failure, `.finally` for cleanup (no arguments, value passes through).
- `.then()` **returns a new promise**. Return a value → next `.then` gets it. Return a promise → next `.then` waits for it.
- A thrown error or rejection **skips** to the nearest `.catch()`. One `.catch` at the end handles the whole chain.
- **Always** `return` inside `.then` when the next step needs the value.
- `fetch()` → Promise of a `Response` ; `response.json()` → another Promise → the data.
- `fetch()` **doesn't reject on 404/500** — check `response.ok` and `throw` .
- Unlike C++ `future.get()` , nothing in a Promise **blocks** the thread.

 Quick Quiz

1. What is printed?

JAVASCRIPT

```
const p = new Promise((resolve, reject) => {
  resolve("A");
  reject("B");
  resolve("C");
});
p.then(v => console.log(v)).catch(e => console.log(e));
```

2. What is printed?

JAVASCRIPT

```
Promise.resolve(2)
  .then(x => x * 3)
  .then(x => { console.log(x); })
  .then(x => console.log(x));
```

3. In what order are these printed?

JAVASCRIPT

```
console.log("start");
new Promise(r => { console.log("executor"); r(); })
  .then(() => console.log("then"));
console.log("end");
```

4. The server answers `404 Not Found`. Does `fetch()` reject? What should you do?

5. In Day 19's Zomato code, the payment fails. Which messages are printed after

`620 Payment is in progress` ? Answers

1. `A` — only the first `resolve` / `reject` counts.
2. `6`, then `undefined` — the second `.then` has `{ }` and no `return`.
3. `start`, `executor`, `end`, `then` — the executor runs immediately; `.then` is a microtask.
4. **No**, the promise is fulfilled with `response.ok === false`. Check `response.ok` and `throw` an error yourself.
5. `Error: Payment is failed` and then `I am doing cleanup` — all the `.then` steps are skipped.

CHAPTER 6 Lecture 19 · Day 19 code



JSON vs JavaScript Objects

IN THIS CHAPTER YOU WILL LEARN

- What **JSON** is and why every language understands it
- The rules that make JSON different from a JS object
- `JSON.stringify()` — object → text
- `JSON.parse()` — text → object
- What JSON **drops** (functions, `undefined`)
- Why `fetch()` gives you JSON, and what `response.json()` does
- The Day 19 JSON experiments

6.1 What is JSON, and why do we need it?

JSON = JavaScript Object Notation. It is a **text format** for storing and sending data. It *looks* like a JavaScript object, but it is really just a **string** that follows strict rules.

Why do we need a text format at all? When your browser talks to a server, it cannot send a JavaScript object “as it is” — an object lives in your computer’s memory, and the server may be written in a completely different language (C++, Python, Java, Go...). So both sides agree on a **common text format**. JSON is that format — the “language” different systems use to talk to each other.

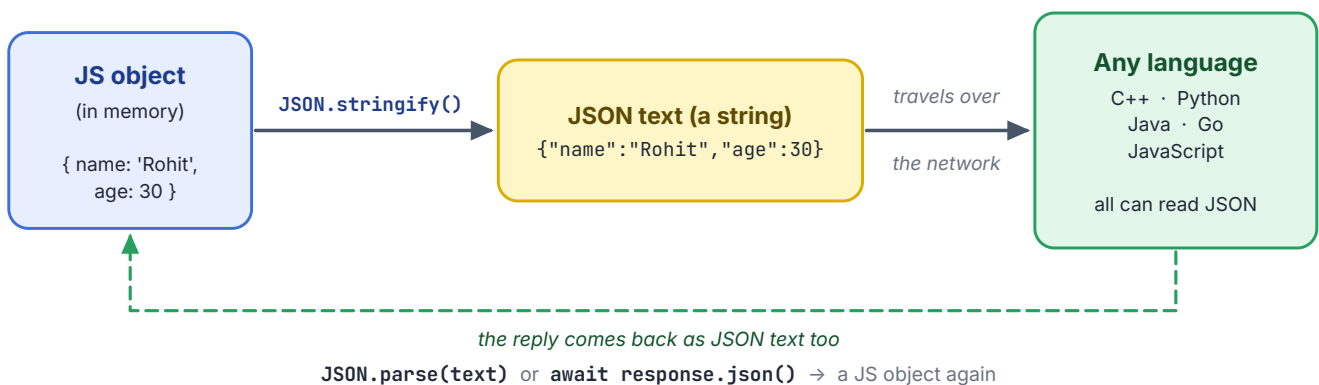


Figure 6.1 A JavaScript object is turned into JSON text, travels over the network, and any language can turn the text back into its own kind of object.

6.2 Quick visual comparison

JAVASCRIPT

```
// JavaScript Object – lives in memory, used by your code
const jsObject = {
  name: "John",           // no quotes needed around the key
  age: 30,                // number
  active: true,           // boolean
  greet: function () {   // can have functions
    console.log("Hi!");
  },
  date: new Date(),       // can have Date objects
  score: undefined,       // can have undefined
};

// JSON – just a STRING with strict rules
const jsonString = `{
  "name": "John",
  "age": 30,
  "active": true
}`;

// Note: must use double quotes – no functions, no undefined, etc.
```

6.3 Key differences

Feature	JavaScript Object	JSON
Type	a JavaScript data structure (in memory)	text / a string
Keys	can be unquoted: <code>name</code>	must use double quotes: <code>"name"</code>
Strings	single <code>'...'</code> or double <code>"..."</code> quotes	only double quotes <code>"..."</code>
Data types	string, number, boolean, array, object, function, Date, <code>undefined</code> , <code>null</code> , Symbol...	string, number, boolean, array, object, <code>null</code> — only these
Functions	✓ allowed	✗ not allowed
<code>undefined</code>	✓ allowed	✗ not allowed (<code>null</code> is fine)
Date objects	✓ allowed	✗ not allowed (stored as a string)
Comments	✓ allowed	✗ not allowed
Trailing commas	✓ allowed	✗ not allowed

Valid or not? Test yourself

This text...	Valid JSON?	Why
<code>{"name": "John", "age": 30}</code>	yes	keys and strings in double quotes
<code>{name: "John"}</code>	no	the key is not in double quotes
<code>{'name': 'John'}</code>	no	single quotes are not allowed
<code>{"age": 30,}</code>	no	trailing comma after the last value
<code>{"city": null}</code>	yes	<code>null</code> is a valid JSON value
<code>{"age": undefined}</code>	no	<code>undefined</code> doesn't exist in JSON
<code>{"greet": function() {}}</code>	no	functions are not data
<code>{"a": 1} // note</code>	no	comments are not allowed
<code>[1, "two", true, null, {"x": [2, 3]}]</code>	yes	arrays and nesting are fine

6.4 Converting between JSON and JavaScript objects

JavaScript object → JSON string: `JSON.stringify()`

JAVASCRIPT

```
const user = { name: "John", age: 30, active: true };

// Convert to a JSON string
const jsonString = JSON.stringify(user);
console.log(jsonString);           // {"name":"John","age":30,"active":true}
console.log(typeof jsonString);    // "string"
```

Want it readable? Pass `null, 2` to indent with 2 spaces:

JAVASCRIPT

```
console.log(JSON.stringify(user, null, 2));
```

OUTPUT

```
{
  "name": "John",
  "age": 30,
  "active": true
}
```

JSON string → JavaScript object: `JSON.parse()`

JAVASCRIPT

```
const jsonString = '{"name":"John","age":30,"active":true}';

// Convert to a JavaScript object
const user = JSON.parse(jsonString);
console.log(user);           // { name: 'John', age: 30, active: true }
console.log(typeof user);    // "object"
console.log(user.name);      // "John"
```

An easy way to remember the names

stringify = make it a **string** (object → text). **parse** = read the text and build the object (text → object). Parse means “read and understand” — like a compiler *parses* your C++ source code.

What happens to things JSON can't store?

JAVASCRIPT

```
const data = {
  name: "Alice",
  greet() { return "hi"; },           // function
  age: undefined,                     // undefined
  born: new Date("2000-01-01T00:00:00Z"), // Date
};

console.log(JSON.stringify(data));
// {"name":"Alice","born":"2000-01-01T00:00:00.000Z"}
```

- The **function** and the **undefined** property are **silently dropped**.
- The **Date** becomes a plain **string**. (After `JSON.parse` it stays a string — not a **Date** object.)

Invalid JSON makes `JSON.parse` throw

JAVASCRIPT

```
try {
  JSON.parse("{name: 'Rohit'}"); // ❌ not valid JSON (no double quotes)
} catch (error) {
  console.log(error.name);        // SyntaxError
}
```

If the text comes from outside (a server, a file, the user), wrap `JSON.parse` in `try / catch`.

6.5 Why does `fetch()` give us JSON?

JAVASCRIPT

```
fetch("https://api.github.com/users")
  .then(response => {
    // response.body is a stream of JSON text
    console.log(typeof response); // "object" (a Response object)
    return response.json();       // parses the JSON string → a JS object
  })
  .then(data => {
    // Now 'data' is a JavaScript array of objects
    console.log(typeof data);     // "object"
    console.log(data[0].login);   // you can access properties
  });
```

The flow:

1. The server sends the data as a **JSON string** (text).
2. `fetch()` receives it as a **Response object**.
3. `.json()` parses the JSON string → **JavaScript object** (it's like `JSON.parse`, but async, because the body may still be arriving).
4. Now you can use it like a normal JS object: `data[0].login`.

6.6 Practical example: sending and receiving

JAVASCRIPT

```
// Creating a user object in JavaScript
const user = {
  name: "Alice",
  age: 25,
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  },
};
user.greet(); // Works fine: Hi, I'm Alice

// Sending to a server (must convert to JSON)
const jsonToSend = JSON.stringify(user);
console.log(jsonToSend); // {"name":"Alice","age":25}
// Notice: the greet() function is gone! JSON can't store functions.

// Receiving from a server
const jsonReceived = '{"name":"Bob","age":30}';
const userFromServer = JSON.parse(jsonReceived);
console.log(userFromServer.name); // "Bob"
// userFromServer.greet();        // ❌ TypeError: userFromServer.greet is not a function
```

JSON carries only **data**, never **behaviour** (functions). That's why a parsed object has no methods.

6.7 Day 19 code: the JSON experiments

Day 19 — `index.js`, the JSON part

These two small experiments sit between the `fetch` experiments and the Zomato code in the Day 19 file.

Object → JSON

JAVASCRIPT Day 19 · `index.js` — JSON experiment 1

```
const j1 = {
  name: "Rohit",
  age: 30,
  address: "dwarka",
}

// convert to json

const jsonFormat = JSON.stringify(j1);

console.log(jsonFormat);
```

OUTPUT

```
{"name":"Rohit","age":30,"address":"dwarka"}
```

Look at the differences: the keys now have **double quotes**, there are **no spaces**, and the **trailing comma** after `"dwarka"` (allowed in a JS object) is **gone** — it's not allowed in JSON.

JSON → object

JAVASCRIPT Day 19 · `index.js` — JSON experiment 2

```
const jsonFormat = `{
  "name":"Rohit",
  "age": 30,
  "address": "dwarka"
}`;

// java script object

const JsObject = JSON.parse(jsonFormat);

console.log(JsObject);
```

OUTPUT

```
{ name: 'Rohit', age: 30, address: 'dwarka' }
```

- The JSON text is written with **backticks** because a template literal can span **multiple lines**.
- After `JSON.parse`, it is a real object: `JsObject.name` is `"Rohit"`, `JsObject.age + 1` is `31`.

⚠ Running both experiments at the same time

Both experiments declare `const jsonFormat`. If you un-comment both together you get `SyntaxError: Identifier 'jsonFormat' has already been declared` — exactly like a C++ *redefinition* error. That's why the teacher keeps one of them commented out.

🧠 JSON in C++ (serialization)

Turning an object into text and back is called **serialization** / **deserialization**. C++ has no built-in JSON, but libraries like *nlohmann/json* do the same job:

C++

```
#include <nlohmann/json.hpp>
using json = nlohmann::json;

json j = { {"name", "Rohit"}, {"age", 30} };
std::string text = j.dump();           // like JSON.stringify → {"age":30,"name":"Rohit"}
json back = json::parse(text);        // like JSON.parse
std::cout << back["name"];            // "Rohit"
```

Because JSON is plain text with fixed rules, a **C++ server** and a **JavaScript browser** can exchange data without knowing anything about each other's memory layout.

6.8 Summary

- **JavaScript object**: the native JS data structure; can hold functions, dates, `undefined`, etc.
- **JSON**: a text format for data exchange; strict rules; only basic data types.
- `JSON.stringify()`: JS object → JSON string.
- `JSON.parse()`: JSON string → JS object.
- `fetch().json()`: automatically parses a JSON response → JS object (returns a Promise).

Think of **JSON** as the language different systems use to talk to each other, while **JavaScript objects** are what you actually work with in your code!

6.9 Things to Remember

📌 Things to Remember

- JSON is **text** (a string). A JS object is a **data structure in memory**.
- JSON rules: **double quotes** for keys and strings, **no functions**, **no** `undefined`, **no** comments, **no** trailing commas. `null` is allowed.
- `JSON.stringify(obj)` → string. `JSON.stringify(obj, null, 2)` → pretty string.
- `JSON.parse(text)` → object. Invalid text → `SyntaxError` (use `try / catch`).
- `stringify` **drops** functions and `undefined` properties; Dates become strings.
- `response.json()` = “read the body and `JSON.parse` it” — and it returns a **Promise**.

🎯 Quick Quiz

1. What is `typeof JSON.stringify({ a: 1 })` ?
2. What does `JSON.stringify({ a: 1, b: undefined, c: () => 1 })` return?
3. Is `{'name': 'Rohit'}` valid JSON? Why?
4. After `const o = JSON.parse('{ "x": 5 }')`, what is `o.x * 2` ?
5. Why does `response.json()` return a Promise instead of the data directly?

✓ Answers

1. `"string"`
2. `'{"a":1}'` — `undefined` and functions are dropped.
3. **No** — JSON allows only double quotes.
4. `10` — after parsing, `o.x` is a real number.
5. Because reading the whole response body (and parsing it) takes time — it's asynchronous work.

CHAPTER 7 Lecture 20 · Day 20 code



async / await

IN THIS CHAPTER YOU WILL LEARN

- `async` functions **always return a Promise**
- `await` pauses **only this function** — not the program
- Error handling with `try` / `catch`
- The important rules (and the most common mistakes)
- **Sequential vs parallel** — `Promise.all`
- The **Day 20** code: GitHub user cards, Zomato with `async` / `await`
- Callbacks vs Promises vs `async` / `await` — the same app 3 ways

7.1 What is `async` / `await` ?

`async` / `await` is a modern JavaScript feature (ES2017) that gives us a **cleaner way to work with promises**. Instead of chaining `.then()` calls, you can write asynchronous code that **reads like synchronous code** — top to bottom.

It is **not** a new mechanism. Underneath, it is still Promises and the event loop. That's why people call it “**syntactic sugar**” over Promises: the same thing, but sweeter to write.

7.2 `async` functions always return a Promise

Put the word `async` before a function and it will **always return a Promise** — even if you return a normal value:

JAVASCRIPT

```
async function myFunction() {
  return "Hello";
}

// is equivalent to:
function myFunction() {
  return Promise.resolve("Hello");
}
```

- `return value` inside an `async` function → the promise is **fulfilled** with `value`.
- `throw error` inside an `async` function → the promise is **rejected** with `error`.

 Day 20 — experiment 1: `greet()`**JAVASCRIPT** Day 20 · index.js — experiment 1

```
// async await
// async function always return a promise
async function greet() {
  return "Rohit";

  // return new Promise((resolve, reject) => {
  //   reject("Rohit");
  // })
}

const response = greet();
// console.log(response);
response.then((data) => console.log(data))
  .catch((error) => {
    console.log("Error:", error);
  })
```

OUTPUT

Rohit

- `greet()` returns a **Promise**, not the string. That's why we need `.then` to get `"Rohit"` out of it.
- If you un-comment `console.log(response)`, Chrome shows `Promise {<fulfilled>: 'Rohit'}` (Node.js shows `Promise { 'Rohit' }`).
- If you replace `return "Rohit"` with the commented code (a promise that **rejects**), the async function's promise rejects too, and the output becomes `Error: Rohit`.

7.3 `await` — “pause this function until the promise settles”

`await` can be written in front of a promise **inside an async function**. It pauses **that function** until the promise settles:

- If the promise is **fulfilled** → `await` gives you its **value**.
- If the promise is **rejected** → `await` **throws** the error (so you can catch it with `try / catch`).

JAVASCRIPT

```
async function fetchData() {
  const response = await fetch('https://api.example.com/data'); // wait for the Response
  const data = await response.json(); // wait for the body
  return data;
}
```

Compare with Chapter 5: no `.then`, no nested callbacks — just normal-looking lines.

`await` pauses ONLY this function — the rest of the program keeps running

This is the most important thing to understand:

JAVASCRIPT

```
const wait = (ms) => new Promise(resolve => setTimeout(resolve, ms));

async function demo() {
  console.log("2. demo started");
  await wait(1000);           // demo() pauses here for 1 second...
  console.log("4. demo resumed"); // ...and continues from here later
}

console.log("1. before calling demo");
demo();
console.log("3. after calling demo — main code is NOT blocked");
```

OUTPUT

```
1. before calling demo
2. demo started
3. after calling demo — main code is NOT blocked
4. demo resumed ← 1 second later
```

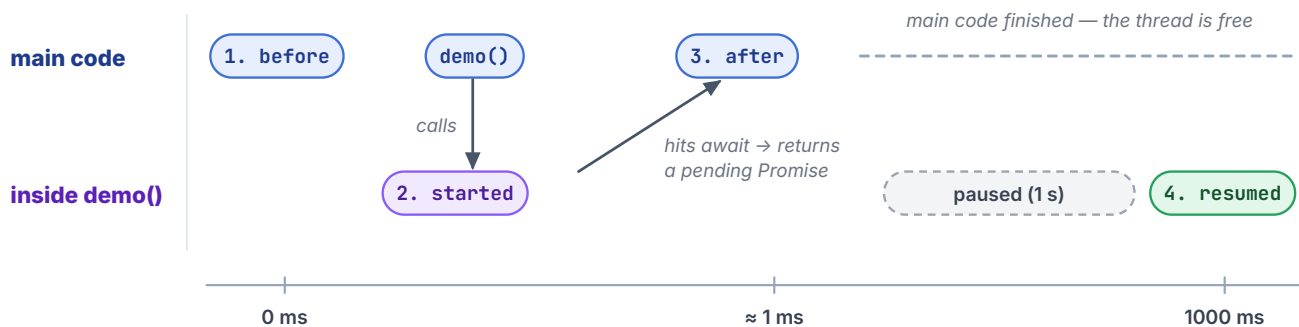


Figure 7.1 `demo()` runs until its first `await`, then steps aside. The main code continues; one second later the rest of `demo()` is resumed (as a microtask).

`future.get()` blocks — `await` doesn't

In C++, `int x = f.get();` **blocks the whole thread** until the value is ready. `await` is different: it **suspends only this function** and gives the thread back to the event loop, which can run other code (clicks, timers, other functions) in the meantime. If you have heard of C++20 coroutines, `co_await` is the same idea.

Under the hood, JavaScript turns “the rest of the function after `await`” into a `.then()` callback:

JAVASCRIPT

```
// This async function...
async function getUsername() {
  const response = await fetch("/api/user");
  const user = await response.json();
  console.log(user.name);
}

// ...does the same as this promise chain:
function getUsername() {
  return fetch("/api/user")
    .then(response => response.json())
    .then(user => { console.log(user.name); });
}
```

7.4 Promises vs `async` / `await` — side by side

With Promises:

JAVASCRIPT

```
function getUser() {
  return fetch('/api/user')
    .then(response => response.json())
    .then(user => {
      return fetch(`/api/posts/${user.id}`);
    })
    .then(response => response.json())
    .then(posts => {
      console.log(posts);
      return posts;
    })
    .catch(error => console.error(error));
}
```

With `async / await`:

JAVASCRIPT

```
async function getUser() {
  try {
    const response = await fetch('/api/user');
    const user = await response.json();
    const postsResponse = await fetch(`/api/posts/${user.id}`);
    const posts = await postsResponse.json();
    console.log(posts);
    return posts;
  } catch (error) {
    console.error(error);
  }
}
```

The second version reads like a normal recipe. And notice another win: `user` is a normal variable, so every later line can use it — no more “how do I get data out of the callback?” problem.

7.5 Error handling with `try / catch`

Use `try / catch` blocks for error handling — this feels more natural than `.catch()`, and it’s the same `try / catch` you know from C++:

JAVASCRIPT

```
async function riskyOperation() {
  try {
    const result = await mightFail();
    return result;
  } catch (error) {
    // Handle specific errors
    if (error.code === 'NETWORK_ERROR') {
      console.error('Network failed');
    }
    throw error; // re-throw if needed
  }
}
```

What gets caught?

When you use `try / catch` with `async / await`, the `catch` block handles:

1. **Rejected promises** — when a promise you `await` rejects.
2. **Thrown errors** — anything thrown with `throw` inside the `try` block.
3. **Synchronous errors** — runtime errors like `TypeError`, `ReferenceError`, etc.

JAVASCRIPT

```
// 1. Rejected promise
async function example1() {
  try {
    const result = await Promise.reject(new Error('Promise rejected!'));
  } catch (error) {
    console.log(error.message); // "Promise rejected!"
  }
}

// 2. Thrown error
async function example2() {
  try {
    throw new Error('Something went wrong');
  } catch (error) {
    console.log(error.message); // "Something went wrong"
  }
}

// 3. Runtime error
async function example3() {
  try {
    const obj = null;
    console.log(obj.property); // TypeError
  } catch (error) {
    console.log(error.message); // "Cannot read properties of null (reading 'property')"
  }
}
```

(Older browsers word the last message a bit differently, e.g. *"Cannot read property 'property' of null"* — same error.)

Real-world example with `fetch`

JAVASCRIPT

```
async function fetchUser(id) {
  try {
    const response = await fetch(`/api/user/${id}`);

    // fetch doesn't reject on HTTP errors (404, 500, etc.)
    // So you need to check manually:
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const user = await response.json();
    return user;
  } catch (error) {
    // This catches:
    // - network failures (promise rejection from fetch)
    // - the thrown error for a bad HTTP status
    // - JSON parsing errors
    console.error('Failed to fetch user:', error);
  } finally {
    console.log("done"); // like .finally() - runs in every case
  }
}
```



Important note about `fetch()`

A common gotcha: `fetch()` only rejects on **network failures**, not on HTTP error statuses like 404 or 500. You need to check `response.ok` manually and `throw` an error if needed. And yes — rejected promises go into the `catch` block, along with any other errors thrown during the `try` block!

7.6 Important rules

1. `await` only works inside `async` functions (or at the top level of ES modules). In a normal function it's an error:
`SyntaxError: await is only valid in async functions and the top level bodies of modules`.
2. `await` pauses execution of *that* function, but it doesn't block the entire program.
3. You await promises. If you `await` a normal value, it is simply wrapped in a resolved promise:
`await 5` gives `5`.

! Forgetting `await`**JAVASCRIPT**

```

async function getData() {
  const response = fetch("/api/data"); // ❌ forgot await → response is a Promise!
  console.log(response.ok);           // undefined
  const data = response.json();       // ❌ TypeError: not a function
}

```

Without `await` you get the **ticket** (the Promise), not the result. Fix:

```
const response = await fetch("/api/data"); .
```

7.7 Sequential vs parallel

Sequential (slower):

JAVASCRIPT

```

const user = await fetchUser(); // wait...
const posts = await fetchPosts(); // ...then wait again

```

Parallel (faster):

JAVASCRIPT

```

const [user, posts] = await Promise.all([
  fetchUser(),
  fetchPosts(),
]);

```

In the sequential version, `fetchPosts()` doesn't even **start** until `fetchUser()` has finished. In the parallel version, **both start immediately**, and we wait for both together.

Sequential — two awaits**Parallel — `Promise.all([ ... ])`**

Figure 7.2 If each request takes 2 s: sequential = 2 + 2 = 4 s; parallel = about 2 s.

 `Promise.all([p1, p2, ...])`

- Takes an **array of promises** and returns **one** promise.
- It **fulfills** when **all** of them are fulfilled — with an **array of results**, in the **same order** as the input (not the order in which they finished).
- It **rejects** as soon as **any one** of them rejects (with that error) — “fail fast”.

You can measure the difference yourself:

JAVASCRIPT

```
const wait = (ms, value) => new Promise(resolve => setTimeout(() => resolve(value), ms));

async function sequential() {
  console.time("sequential");
  const a = await wait(1000, "A");
  const b = await wait(1000, "B");
  console.timeEnd("sequential"); // sequential: ~2000 ms
}

async function parallel() {
  console.time("parallel");
  const [a, b] = await Promise.all([wait(1000, "A"), wait(1000, "B")]);
  console.timeEnd("parallel"); // parallel: ~1000 ms
}
```

When to use which? Use **sequential** when a step **needs the result** of the previous one (like the Zomato order: you can't cook before payment). Use **parallel** when the tasks are **independent** (like loading a user's comments, photos and chats). This also solves “Problem 8” of callback hell from Chapter 4.

7.8 Common patterns

Handling multiple operations:

JAVASCRIPT

```
async function loadDashboard() {
  try {
    const [user, notifications, stats] = await Promise.all([
      getUser(),
      getNotifications(),
      getStats(),
    ]);
    return { user, notifications, stats };
  } catch (error) {
    console.error('Failed to load dashboard:', error);
  }
}
```

`{ user, notifications, stats }` is a shortcut for
`{ user: user, notifications: notifications, stats: stats } .)`

Conditional awaits:

JAVASCRIPT

```
async function getData(useCache) {  
  if (useCache) {  
    return getCacheData();  
  }  
  return await fetchFreshData();  
}
```

The beauty of `async / await`: asynchronous code becomes much easier to read and reason about, especially when you have complex logic with multiple asynchronous operations.

7.9 Day 20 code, experiment by experiment

 **Day 20** — `index.html` + `style.css` + `index.js`

The page has a heading and an empty container. The CSS turns every user into a **card**:

HTML 03JS/Day20/index.html

```
<head>  
  <link rel="stylesheet" href="./style.css">  
</head>  
<body>  
  <h1>Github User</h1>  
  <div id="first"></div>  
</body>  
<script src="./index.js"></script>
```

The important CSS rules (from `style.css`):

CSS 03JS/Day20/style.css (important parts)

```
body { height: 100vh; background-color: black; color: white; }
h1   { font-size: 40px; text-align: center; margin: 20px 0; }

#first {
    /* the container of all cards */
    display: flex;           /* put cards in a row... */
    flex-wrap: wrap;        /* ...and wrap to new lines */
    justify-content: center;
    gap: 30px;
    width: 80%;
    margin: auto;
}

.user {
    /* one card */
    width: 200px;
    text-align: center;
    padding: 10px;
    border: 2px solid white;
    border-radius: 5px;
    transition: transform 1s ease;
}
.user:hover { transform: scale(1.2); } /* grows 1.2x when the mouse is on it */

img { width: 100%; height: 200px; object-fit: cover; }
a    { text-decoration: none; color: orange; }
```

Experiment 1 — `greet()`

Explained in section 7.2 above.

Experiment 2 — GitHub user cards with `async` / `await`

JAVASCRIPT Day 20 · index.js — experiment 2

```
async function github() {

    try {
        const response = await fetch("https://api.github.com/users");
        if (!response.ok) {
            throw new Error("Data is not present");
        }

        const data = await response.json();
        // console.log(data);

        const parent = document.getElementById("first");

        for (let user of data) {

            const element = document.createElement("div");
            element.classList.add("user");
```

```

    const image = document.createElement('img');
    image.src = user.avatar_url;

    const userName = document.createElement('h2');
    userName.textContent = user.login;

    const anchor = document.createElement('a');
    anchor.href = user.html_url;
    anchor.textContent = "Visit Profile";

    element.append(image, userName, anchor);
    parent.append(element);
  }
}
catch (error) {
  console.log("error");
}
}

github();

console.log("Hello Ji kaise ho");

```

Step by step:

1. `await fetch(...)` — pause `github()` until GitHub answers; `response` is the `Response` object.
2. `if (!response.ok) throw ...` — turn an HTTP error (404, 500...) into a real error → jumps to `catch`.
3. `await response.json()` — pause again until the body is parsed; `data` is an array of users.
4. `for (let user of data)` — for each user object (a `for...of` loop, like C++ range-for):
 - create a `<div>` and give it the CSS class `user` (`classList.add`) → it becomes a card;
 - create an `<img>` with the user's avatar, an `<h2>` with their `login` name, and an `<a>` link to their GitHub profile (`html_url`) with the text "Visit Profile";
 - `element.append(image, userName, anchor)` — `append` can add **several** children at once;
 - `parent.append(element)` — put the finished card inside `#first`.
5. Any problem (network error, bad status, parsing error) jumps to `catch`, which prints `"error"`.

OUTPUT

Hello Ji kaise ho ← *printed FIRST!*

Why is `"Hello Ji kaise ho"` printed first? `github()` runs until its first `await`, then **pauses** and returns a (pending) promise. The main code continues and prints the message. When GitHub replies, `github()` resumes and builds the cards. `await` pauses the **function**, not the program.

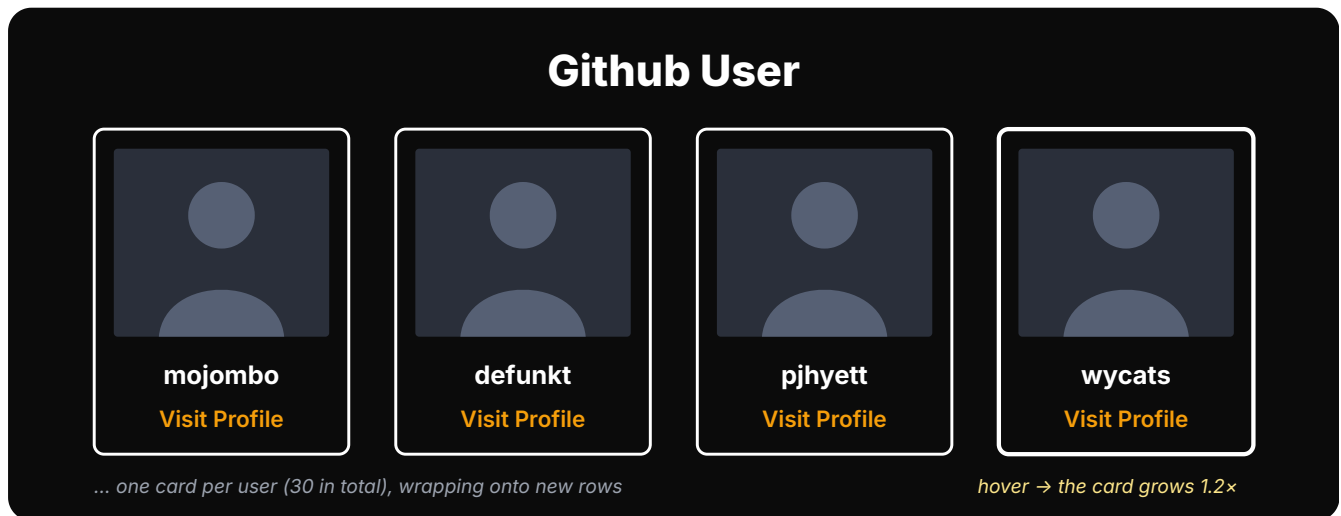


Figure 7.3 Roughly what the page looks like: a centred heading and one card per user, wrapping onto new rows. The user names shown are examples.

Experiment 3 — the Zomato app with `async / await`

The four step functions (`placedOrder`, `preparingOrder`, `pickupOrder`, `deliverOrder`) are **exactly the same** as in Day 19 — each one returns a Promise. Only the code that **uses** them changes:

JAVASCRIPT Day 20 · index.js — experiment 3

```
async function ordering() {

  try {
    const response1 = await placedOrder(orderDetail);
    const response2 = await preparingOrder(response1);
    const response3 = await pickupOrder(response2);
    const response4 = await deliverOrder(response3);

    console.log(response4);
  }
  catch (error) {
    console.log("Error: ", error);
  }
}

ordering();
```

- Each `await` waits for one step's promise and gives us its value (the updated `orderDetail`).
- The steps run **in order** because each line waits for the previous one — exactly what we need (sequential!).
- If any step **rejects**, `await` **throws**, the remaining lines are skipped, and `catch` prints the reason — e.g. `Error: Payment is failed`.
- The output is the same as the Day 19 version, just without the `"I am doing cleanup"` line (there is no `finally` here — you could add `finally { console.log("I am doing cleanup"); }`).

Experiment 4 — `Promise.all` (the active code in the file)

JAVASCRIPT Day 20 · index.js — experiment 4

```
async function userDetails(params) {

  // const comment = await fetch("userComment");
  // const photos = await fetch("userPhoto");
  // const chat = await fetch("chat");

  const [comment, photos, chat] = await Promise.all([
    fetch("userComment"),
    fetch("photo"),
    fetch("chat"),
  ]);
}
```

- The commented lines are the **sequential** way: the photo request would start only after the comment request finished, and so on.
- `Promise.all` starts **all three requests at the same time** and waits until all three have finished. The three results come back in an array, and **array destructuring** puts them into `comment`, `photos` and `chat` (in the same order as the input array).
- `"userComment"`, `"photo"` and `"chat"` are just **example addresses** to show the idea (they don't exist on a real server). The function is also never called in the file, and `params` isn't used — it's a demo of the pattern.

7.10 One app, three styles

	Callbacks (Day 18)	Promises (Day 19)	<code>async</code> / <code>await</code> (Day 20)
Step function	takes a <code>Callback</code>	returns a Promise	returns a Promise (same as Day 19)
Running the steps	nested callbacks	<code>.then().then()...</code> chain	<code>await</code> line by line
Shape of the code	pyramid →	flat chain	looks like normal sync code
Handling errors	at every level (hard)	one <code>.catch()</code>	one <code>try</code> / <code>catch</code>
Cleanup	manual	<code>.finally()</code>	<code>finally { }</code>
Getting data out	hard (scope problem)	inside <code>.then</code>	normal variables (<code>response1</code> ...)

JAVASCRIPT

```
// 1. Callbacks (Day 18) – nested
placedOrder(orderDetail, (o) =>
  preparingOrder(o, (o) =>
    pickupOrder(o, (o) =>
      deliverOrder(o))));

// 2. Promises (Day 19) – a flat chain
placedOrder(orderDetail)
  .then(preparingOrder)
  .then(pickupOrder)
  .then(deliverOrder);

// 3. async / await (Day 20) – reads like normal code (inside an async function)
const o1 = await placedOrder(orderDetail);
const o2 = await preparingOrder(o1);
const o3 = await pickupOrder(o2);
await deliverOrder(o3);
```

7.11 Things to Remember

Things to Remember

- `async` function → **always returns a Promise**. `return x` → fulfilled with `x`; `throw e` → rejected with `e`.
- `await promise` → pauses **this function** until the promise settles: gives the **value**, or **throws** the error.
- `await` does **not** block the program: code after the call to an async function runs **before** the code after its first `await`.
- Use `try` / `catch` / `finally` for errors and cleanup. It catches rejected promises, thrown errors and runtime errors.
- `await` works only inside `async` functions (or top-level modules). Forgetting `await` gives you a **Promise** instead of the value.
- `fetch` doesn't reject on 404/500 → check `response.ok`.
- **Sequential** awaits for dependent steps; `Promise.all` for independent ones (faster, results in input order, fails fast).
- `async` / `await` is just **nicer syntax for Promises** — the event loop works exactly the same.

 Quick Quiz

1. What does `async function f() { return 10; }` return when you call `f()` ?
2. What is printed, and in which order?

JAVASCRIPT

```
async function f() {  
  console.log("B");  
  await null;  
  console.log("D");  
}  
console.log("A");  
f();  
console.log("C");
```

3. Two independent requests take 3 s each. How long do sequential awaits take? And `Promise.all` ?
4. In Day 20's `ordering()`, the restaurant rejects with `"Food item is not persent at restaurant"`. What is printed after `Your food preparation started of Pizza,biryani,coke` ?
5. Why does `"Hello Ji kaise ho"` appear before the GitHub cards in Day 20?

 Answers

1. A **Promise** that is fulfilled with `10` (use `await f()` or `f().then(...)` to get 10).
2. `A`, `B`, `C`, `D` — `f` runs synchronously until `await`, then the main code continues; `D` runs later as a microtask.
3. Sequential $\approx$ 6 s; `Promise.all` $\approx$ 3 s.
4. `Error: Food item is not persent at restaurant` — `await` throws, the rest of the `try` is skipped, `catch` runs.
5. `github()` pauses at its first `await`; meanwhile the main code prints the message. The cards are built later, when the data arrives.

CHAPTER 8 Lecture 21 · Day 21 code



Prototypes and Classes

IN THIS CHAPTER YOU WILL LEARN

- The **problem**: copying the same method into every object
- The **prototype chain**: objects linked to other objects
- `[[Prototype]]`, `__proto__`, `Object.create()`, `hasOwnProperty`
- **Constructor functions** and what `new` really does
- The two “prototypes”: `obj.__proto__` vs `Func.prototype`
- `class`, `constructor`, `extends`, `super` — and why they are “syntactic sugar”
- `this` inside a constructor
- Every experiment from the **Day 21** code

By the end of this chapter you won't just know **how** to use prototypes and classes — you will understand **what they are**, so you can *reason* about object behaviour instead of guessing. We go step by step, exactly like the lecture: **Part 1** the problem → **Part 2** the prototype chain → **Part 3** constructor functions → **Part 4** classes.

8.1 Part 1 — the problem of repetition

The atom of JavaScript is the **object**: a simple bag of key → value pairs. Imagine we are making a small game with users:

JAVASCRIPT

```
let user1 = {
  name: "Alice",
  score: 100,
  sayHi: function () {
    console.log("Hi, I'm " + this.name);
  },
};

let user2 = {
  name: "Bob",
  score: 50,
  sayHi: function () {
    console.log("Hi, I'm " + this.name);
  },
};

user1.sayHi(); // Hi, I'm Alice
user2.sayHi(); // Hi, I'm Bob
```

It works — but look at it from first principles. **What's the problem?**

1. **Inefficiency (memory):** the `sayHi` function is identical for both users, but JavaScript creates and stores **two separate copies** of it. With 1,000,000 users we'd have 1,000,000 copies of the same function — a huge waste. You can even prove they are different: `user1.sayHi === user2.sayHi` is `false`.
2. **Inefficiency (maintenance):** want to add an `increaseScore()` method to all users? You have to go back and add it to `user1`, `user2` and every other object. Want to fix a bug in `sayHi`? Fix it in every single object. A nightmare.

The fundamental question

How can many objects *share* one method without each owning a copy of it?

The answer to this question is the reason prototypes exist: we need a **central place** to store the shared stuff, and a **way for each object to find it**.

In C++ you never had this problem

In C++, a member function is stored **once** in the program's code. Each object only stores its **data members** — `sizeof(User)` doesn't grow when you add methods. When you call `user1.sayHi()`, the compiler just passes `&user1` as the hidden `this` pointer.

A JavaScript object literal is different: `sayHi: function () {...}` creates a **new function object** every time the literal runs. Prototypes are how JavaScript gets the C++ behaviour: *methods stored once, shared by all objects*.

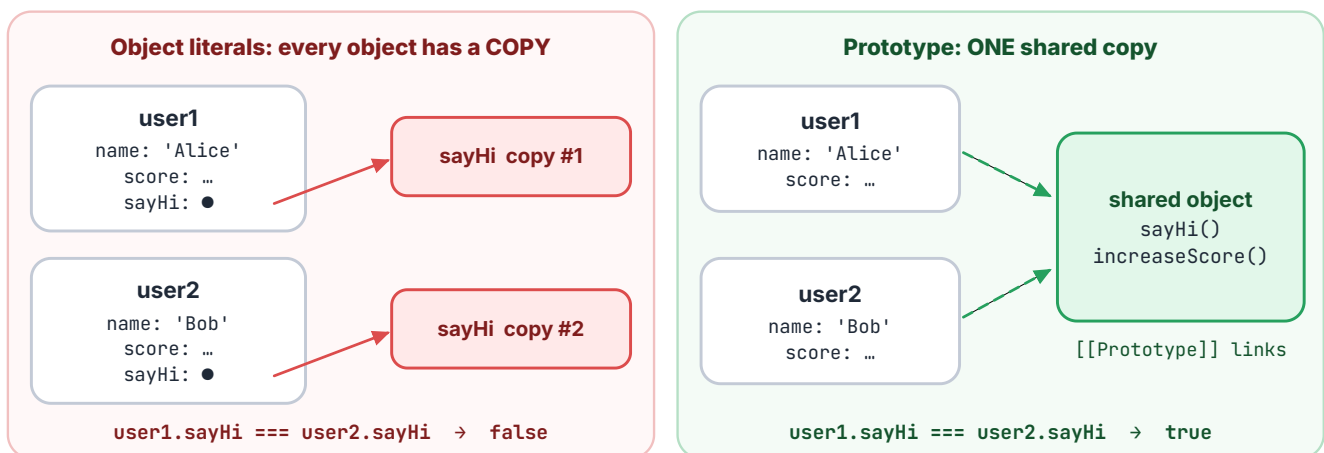


Figure 8.1 Left: with object literals, each object carries its own copy of `sayHi`. Right: with a prototype, the data stays in each object but the method lives in ONE shared place.

8.2 Part 2 — the solution: the prototype chain

The creators of JavaScript solved the problem with one simple, powerful idea: **objects can be linked to other objects**.

Let's make a central object for all the shared methods and call it `userFunctions`:

JAVASCRIPT

```
// A central place for all shared methods
const userFunctions = {
  sayHi: function () {
    console.log("Hi, I'm " + this.name);
  },
  increaseScore: function () {
    this.score++;
  },
};
```

How do our user objects find `userFunctions`? Through a secret link. **Every object in JavaScript has a hidden internal property called `[[Prototype]]`**. Think of it as a secret “parent” or “fallback” object.

When you try to read a property, JavaScript does this:

1. Does the **object itself** have this property? (`user1.name` → yes, `"Alice"` .)
2. If not, does its `[[Prototype]]` object have it?
3. If not, does **that** object's `[[Prototype]]` have it?
4. ...and so on, until it reaches a `[[Prototype]]` that is `null` — then the answer is `undefined` .

This sequence of linked objects is called the **prototype chain**.

Creating the link with `Object.create()`

`Object.create(proto)` creates a **new empty object** whose `[[Prototype]]` is `proto`:

JAVASCRIPT

```
// Shared methods
const userFunctions = {
  sayHi: function () {
    console.log("Hi, I'm " + this.name);
  },
};

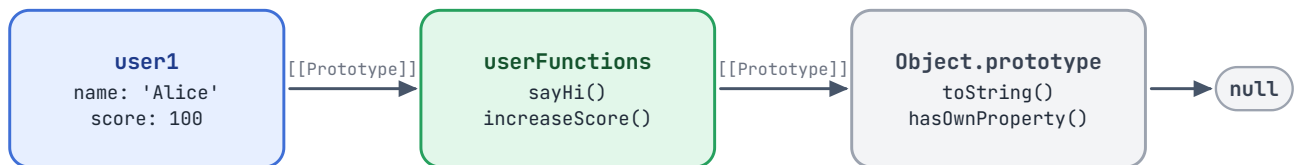
// Create a new user, but tell it to use userFunctions as its fallback/prototype
let user1 = Object.create(userFunctions);
user1.name = "Alice";
user1.score = 100;

let user2 = Object.create(userFunctions);
user2.name = "Bob";
user2.score = 50;

// LET'S TEST IT
user1.sayHi(); // "Hi, I'm Alice"
```

What just happened when we called `user1.sayHi()`?

1. JavaScript looked for `sayHi` on the `user1` object. It couldn't find it — `user1` only has `name` and `score`.
2. JavaScript didn't give up. It followed the hidden `[[Prototype]]` link from `user1` to the `userFunctions` object.
3. It looked for `sayHi` on `userFunctions`. **Found it!**
4. It executed the function. Crucially, the value of `this` inside the function was still `user1` — the object that *started* the call (the one before the dot).



`user1.sayHi()` ① not on `user1` → ② found on `userFunctions` ✓

`user1.toString()` → found two steps up, on `Object.prototype` ✓

`user1.fly` → not found anywhere, reached `null` → `undefined`

Figure 8.2 Looking up `user1.sayHi`: not on `user1` → follow the link → found on `userFunctions`. `toString` would be found one step higher, on `Object.prototype`. Past `null`, the answer is `undefined`.

We have solved the problem! There is **one** copy of `sayHi`, and both `user1` and `user2` can use it (`user1.sayHi === user2.sayHi` is now `true`). This is **prototypal inheritance** in its purest form. Let that sink in — this is the bedrock. Everything else in this chapter is just a more convenient way of creating this link.

🧠 The prototype link ≈ the hidden vtable pointer

In C++, an object of a class with `virtual` functions secretly contains a `vp`tr — a pointer to a shared **vtable** of functions. JavaScript's `[[Prototype]]` is a similar hidden pointer to a shared object. Two big differences:

- The prototype is a **normal object** — you can look at it and even change it while the program runs.
- Lookup happens **by name, at run time**, walking the chain like a **linked list** (`node = node->next` until `nullptr`). C++ resolves member names at **compile time**.

8.3 Every object already has a prototype

You have been using prototypes all along without knowing it! A normal object literal is automatically linked to `Object.prototype`, which contains methods like `toString` and `hasOwnProperty`. Arrays are linked to `Array.prototype` (which has `push`, `map`, `filter` ...), which is linked to `Object.prototype`.

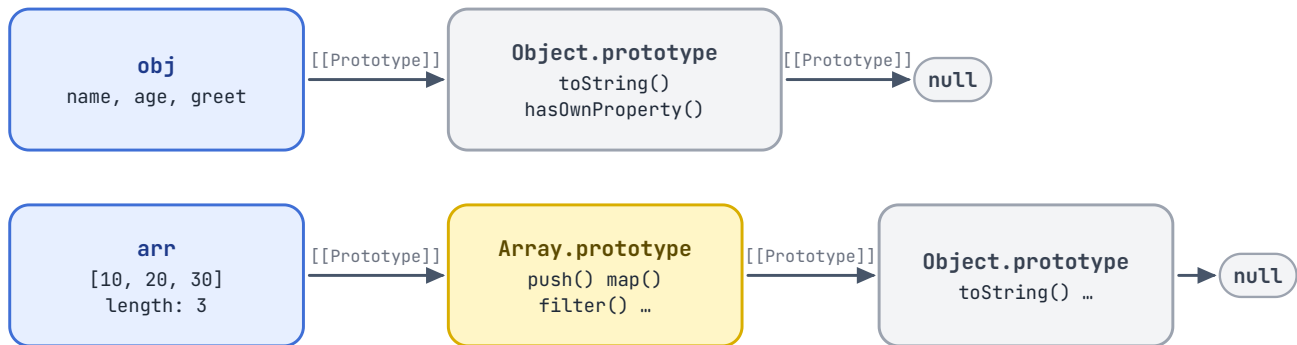


Figure 8.3 The chains you get for free. This is why a plain object can call `toString()` and an array can call `map()` — the methods live on the prototypes, not on your object.

 Day 21 — experiment 1: methods you never wrote**JAVASCRIPT** Day 21 · index.js — experiment 1

```
const obj = {
  name: "Rohit",
  age: 38,
  greet: function () {
    console.log("Hello Ji")
  }
};

console.log(obj.greet());           // prints "Hello Ji", then undefined
obj.greet()                       // prints "Hello Ji"

console.log(obj.hasOwnProperty("names")); // false
console.log(obj.toString());           // [object Object]

const arr = [10, 20, 30];
console.log(arr.length)              // 3
```

OUTPUT

```
Hello Ji
undefined
Hello Ji
false
[object Object]
3
```

- `console.log(obj.greet())` prints **two** lines: first `greet()` itself prints `Hello Ji`, then `console.log` prints what `greet()` **returned** — and it returns nothing, i.e. `undefined`.
- `obj.hasOwnProperty("names")` → `false`: `obj` has no **own** property called `names` (there's a typo on purpose — it's `name`).
- Where do `hasOwnProperty` and `toString` come from? We never wrote them! JavaScript didn't find them on `obj`, followed `[[Prototype]]`, and found them on `Object.prototype`.
- `arr.length` is the array's **own** property; `arr.push`, `arr.map` ... come from `Array.prototype`.

 hasOwnProperty(key)

Returns `true` only if the property is **directly on the object itself** — not somewhere up the prototype chain. So `obj.hasOwnProperty("name")` is `true`, but `obj.hasOwnProperty("toString")` is `false` (even though `obj.toString()` works).

__proto__ — seeing and changing the link

The hidden `[[Prototype]]` can be read and written through the old accessor `__proto__` (two underscores on each side). Browsers show it in the console as `[[Prototype]]`.

Day 21 — experiment 2: linking two objects by hand**JAVASCRIPT** Day 21 · index.js — experiment 2

```
const obj2 = {
  account: 30
}

obj2.__proto__ = obj;    // obj2's parent/fallback is now obj

console.log(obj.hasOwnProperty("name")) // true
```

After `obj2.__proto__ = obj`:

- `obj2.account` → `30` (its own property)
- `obj2.name` → `"Rohit"` — not on `obj2`, found on `obj` through the chain
- `obj2.greet()` → prints `Hello Ji`
- `obj.hasOwnProperty("name")` → `true` (`obj` itself has `name`), but
`obj2.hasOwnProperty("name")` → `false`

✓ Modern, cleaner ways

`__proto__` is old and mostly kept for compatibility. Prefer `Object.create(proto)` to create a linked object, and `Object.getPrototypeOf(obj)` to read the link: `Object.getPrototypeOf(obj2) === obj` → `true`.

8.4 Part 3 — constructor functions and `new` (the old way)

Creating objects with `Object.create()` and then adding properties one by one works, but it's tedious. For years, JavaScript developers used a standard pattern with normal functions.

Any function can become a “constructor function” if you call it with the `new` keyword. By convention we capitalise their names, to signal they’re meant to be used this way.

JAVASCRIPT

```
function User(name, score) {
  this.name = name;
  this.score = score;
}

// Where do we put the shared methods?
// Every function automatically gets a special public property called 'prototype'.
// This is NOT the hidden [[Prototype]]. It is a plain object.
User.prototype.sayHi = function () {
  console.log("Hi, I'm " + this.name);
};
User.prototype.increaseScore = function () {
  this.score++;
};

// Now we use the 'new' keyword
const user1 = new User("Alice", 100);
const user2 = new User("Bob", 50);

user1.sayHi();           // "Hi, I'm Alice"
user2.increaseScore();
console.log(user2.score); // 51
```

This looks very different, but it achieves the **exact same result** as Part 2. The `new` keyword is the key.

Demystifying `new`

When you call `new User("Alice", 100)`, **four things happen automatically** behind the scenes:

1. **Create an empty object:** a brand-new object is created, like `{}`.
2. **Link the prototype:** the new object’s hidden `[[Prototype]]` is set to the constructor’s `prototype` object (`User.prototype`). This is the most critical step — this is how `user1` gets connected to the shared methods.
3. **Execute the constructor:** the `User` function is called, and `this` is set to the new object. So `this.name = name;` really means `newObject.name = "Alice";`.
4. **Return `this`:** the function automatically returns the new object.

`new User("Alice", 100)` does four things:



(no magic: it's just an automated recipe)

Figure 8.4 The four steps of `new User("Alice", 100)`.

So `new` is just an **automated recipe** for creating an object and setting up its prototype link for us.

Write your own `new` (to prove there's no magic)

JAVASCRIPT

```
function myNew(Constructor, ...args) {
  const obj = Object.create(Constructor.prototype); // steps 1 + 2
  Constructor.apply(obj, args); // step 3: run it with this = obj
  return obj; // step 4
}

const u = myNew(User, "Zed", 1);
u.sayHi(); // Hi, I'm Zed
```

(`apply` calls a function with a chosen `this` — Chapter 10 explains it. `...args` collects all the other arguments into an array.)

`new` in C++ vs `new` in JavaScript

C++ `new User("Alice", 100)`: allocate memory → run the constructor with `this` pointing to the new memory → return a **pointer**. JavaScript's `new` is the same recipe (create → link prototype → run constructor with `this` → return the object reference). The extra step is **linking the prototype**, which in C++ happens “for free” because methods belong to the class at compile time.

! Arrow functions can't be constructors

`const Arrow = () => {}; new Arrow();` → `TypeError: Arrow is not a constructor`. Arrow functions have no `prototype` property and no own `this` (Chapter 10), so `new` can't use them.

8.5 The two “prototypes” (the most confusing part!)

	What it is	Where it lives
<code>user1.[[Prototype]]</code> — read it with <code>user1.__proto__</code> or <code>Object.getPrototypeOf(user1)</code>	the actual hidden link from an object to its parent / fallback	on every object
<code>User.prototype</code>	a regular object that sits on a constructor function; it will become the <code>[[Prototype]]</code> of every instance created with <code>new User()</code>	on functions (constructors and classes)

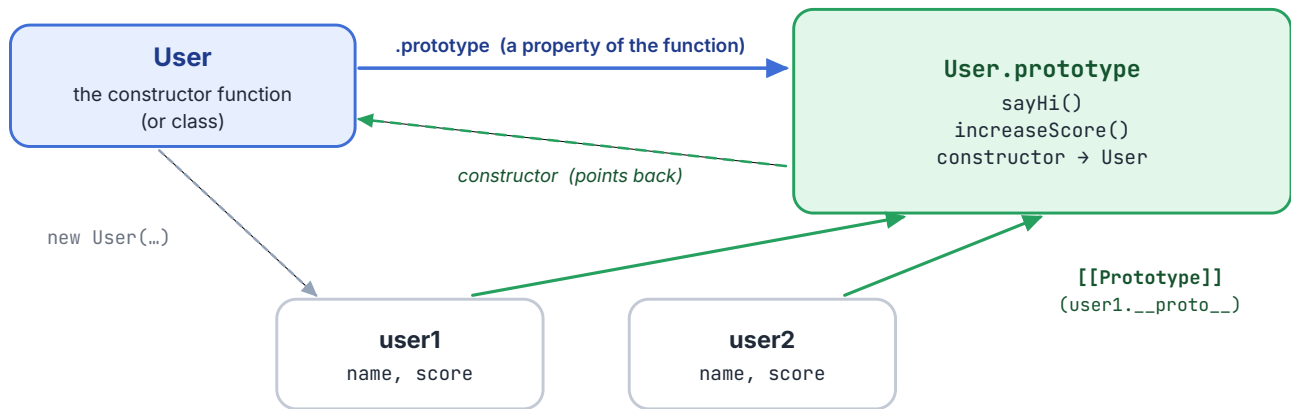


Figure 8.5 `User.prototype` is an object hanging off the function. `new User()` makes each instance's hidden `[[Prototype]]` point to it.

JAVASCRIPT

```

console.log(Object.getPrototypeOf(user1) === User.prototype); // true
console.log(user1.__proto__ === User.prototype);             // true
console.log(User.prototype.constructor === User);           // true (points back)
console.log(user1.prototype); // undefined – plain objects have no .prototype
  
```

Understanding this distinction is the key to mastering prototypes. This pattern worked for over a decade, but it is a bit weird: the methods are defined **outside** the function body, and the `prototype` property is confusing. Programmers coming from Java, C++ or Python wanted a cleaner syntax...

8.6 Part 4 — the modern abstraction: `class`

The `class` keyword was introduced in ES2015 to make the Part 3 process look nicer and more familiar.

Let me be clear

`class` does not introduce a new object model to JavaScript. It is “syntactic sugar” over the existing prototypal inheritance system. It does the **exact same thing** as the constructor-function pattern, just with a cleaner syntax.

Let's rewrite `User` from Part 3 using `class`:

JAVASCRIPT

```
class User {
  constructor(name, score) {
    this.name = name;
    this.score = score;
  }

  // Methods defined here are automatically put on User.prototype!
  sayHi() {
    console.log("Hi, I'm " + this.name);
  }

  increaseScore() {
    this.score++;
  }
}

const user1 = new User("Alice", 100);
const user2 = new User("Bob", 50);

user1.sayHi();           // "Hi, I'm Alice"
console.log(user2.score); // 50
user2.increaseScore();
console.log(user2.score); // 51
```

Let's break it down and prove it's the same thing:

- The `constructor` method is the same as our old `function User(...)`. It's what gets called when you use `new`.
- Any other methods you write inside the class (`sayHi`, `increaseScore`) are **automatically placed on `User.prototype`** for you. No more typing `User.prototype.whatever = ...`.

The proof:

JAVASCRIPT

```
// A class is really just a special kind of function
console.log(typeof User);           // "function"

// The methods are on the prototype, not on the instance
console.log(user1.hasOwnProperty('sayHi')); // false
console.log(User.prototype.hasOwnProperty('sayHi')); // true

// The instance's [[Prototype]] is linked to the class's .prototype
console.log(Object.getPrototypeOf(user1) === User.prototype); // true
```

The evidence is clear. The `class` syntax is a clean, modern wrapper around the constructor function and prototype mechanism we built from first principles.

Small differences from a plain constructor function:

- A class **must** be called with `new`: `User("x", 1)` → `TypeError: Class constructor User cannot be invoked without 'new'`.
- The code inside a class body always runs in **strict mode** (Chapter 9).

C++ class vs JavaScript class

	C++	JavaScript
Constructor	same name as the class: <code>User(...)</code>	always called <code>constructor(...)</code>
Data members	declared with types in the class	created by assigning <code>this.name = ...</code> in the constructor
Methods stored	once, in the code	once, on <code>User.prototype</code>
Create an object	<code>User u("Alice", 100);</code> or <code>new User(...)</code> (pointer)	always <code>new User("Alice", 100)</code> (a reference)
Inheritance	<code>class Customer : public Person</code>	<code>class Customer extends Person</code>
Call parent constructor	initializer list <code>: Person(name, age)</code>	<code>super(name, age)</code>
Access the object	<code>this->name</code> (a pointer)	<code>this.name</code>
private / protected	keywords	everything public by default (<code>#field</code> makes it private)

8.7 Inheritance with `extends` and `super`

`extends` makes one class's prototype link to another's — something that used to be a very messy manual process. Here is the Day 21 example:

Day 21 — experiments 4 & 5: Person and Customer

JAVASCRIPT Day 21 · index.js

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}

class Customer extends Person {
  constructor(name, age, account, balance) {
    super(name, age); // call Person's constructor FIRST
    this.account = account;
    this.balance = balance;
  }

  checkBalance() {
    return this.balance;
  }
}

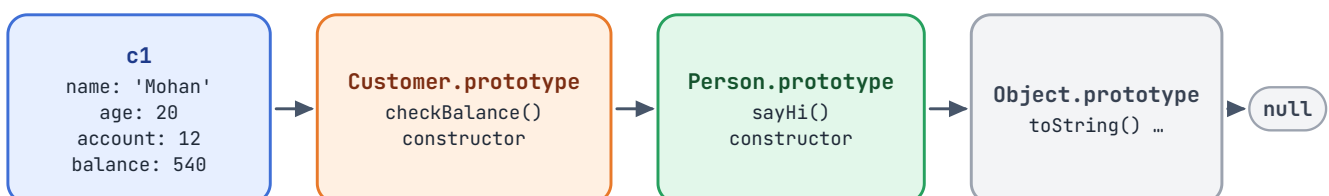
const c1 = new Customer("Mohan", 20, 12, 540);

console.log(c1.checkBalance()); // 540
```

OUTPUT

540

- `class Customer extends Person` — every Customer is a Person, plus more.
- `super(name, age)` calls the **parent's constructor**, which sets `this.name` and `this.age`. Then the child adds its own properties.
- `c1.checkBalance()` is found on `Customer.prototype`. And `c1.sayHi()` works too (prints `Hi Mohan`) — it is found one step higher, on `Person.prototype`.



`c1.checkBalance()` → found on `Customer.prototype` → 540

`c1.sayHi()` → not on `Customer.prototype` → found on `Person.prototype` (inherited!)

Figure 8.6 The prototype chain of `c1`. Data lives on the object; methods are found by walking up the chain.

C++

```
// The same thing in C++
class Person {
public:
    std::string name; int age;
    Person(std::string name, int age) : name(name), age(age) {}
    void sayHi() { std::cout << "Hi " << name << "\n"; }
};

class Customer : public Person {
public:
    int account; double balance;
    Customer(std::string name, int age, int account, double balance)
        : Person(name, age), account(account), balance(balance) {}
    double checkBalance() { return balance; }
};
```

⚠ Using `this` before `super()`

In a child class constructor you **must** call `super(...)` before you touch `this`:

JAVASCRIPT

```
class Customer extends Person {
    constructor(name, age) {
        this.account = 10; // ❌ too early!
        super(name, age);
    }
}

new Customer("A", 1);
// ReferenceError: Must call super constructor in derived class
// before accessing 'this' or returning from derived constructor
```

Think of it like C++: the base part of the object must be constructed before the derived part.

8.8 `this` inside a constructor

The `this` keyword is a special identifier whose value is decided by **how a function is called** (the whole story is in Chapter 10). In a **constructor**, `this` has a very specific and crucial job.

💡 The blank nametag

Imagine a conference registration desk:

1. The `constructor` is the **registration process**.
2. The parameters (`name` , `score`) are the **details you write on the registration form**.
3. The `new` keyword hands the constructor a **brand-new, blank nametag**. This blank nametag is `this`.

The constructor's job is to copy the details from the form onto that blank nametag:

- `this.name = name;` means "take the `name` ('Alice') from the form and write it in the Name field of this specific nametag".
- `this.score = score;` means "take the `score` (100) from the form and write it in the Score field of this specific nametag".

At the end, the constructor hands back the filled-out nametag — a complete object.

Technically, when you call `new User("Alice", 100)`:

1. Creates a brand-new, empty object — call it `newInstance = {}`.
2. Links the prototype — `newInstance`'s prototype is set to `User.prototype`.
3. Calls the constructor — and (the most important part) sets `this` to be `newInstance`. Inside the constructor, `this` is the empty object created in step 1.
4. Returns `this` — the `new` expression gives back `newInstance`, now filled with properties.

So `this` is the mechanism that lets the constructor **populate the new object**.

What would happen without `this`?

JAVASCRIPT

```
class User {
  constructor(name, score) {
    // This is WRONG and does not work.
    // 'name' here just refers to the local parameter, not a property.
    name = name;
    score = score;
    // We never attached anything to the new object being created.
  }
}

const user1 = new User("Alice", 100);
console.log(user1.name);    // undefined
console.log(user1.score);   // undefined
```

The `user1` object is created, but it's **empty**, because we never used `this` to attach `name` and `score` to it. We only had local variables that disappeared when the constructor finished — like assigning to a parameter in a C++ constructor instead of to a member.

8.9 Day 21 code: the other experiments

Experiment 3 — the repetition problem, live

JAVASCRIPT Day 21 · index.js — experiment 3

```
const obj1 = {
  name: "Rohit",
  age: 30,
  greet: function () {
    console.log(`Hello ${this.name}`);
  }
}

const obj2 = {
  name: "Mohit",
  age: 20,
  greet: function () {
    console.log(`Hello ${this.name}`);
  }
}

const obj3 = {
  name: "Mohan",
  age: 10,
  greet: function () {
    console.log(`Hello ${this.name}`);
  }
}
```

Three objects, **three copies** of the same `greet` function — exactly the Part 1 problem. In the Day 22 file the teacher writes a comment about exactly this: *"100 user: greet function: 100*memory, code copy paste"*. This is what motivates prototypes and classes.

Experiment 4 — a class instance vs an object literal in the console

JAVASCRIPT Day 21 · index.js — experiment 4

```

class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  sayHi() {
    console.log(`Hi ${this.name}`);
  }
}

const person1 = new Person("Rohit", 20);
const person2 = new Person("Mohit", 10);
console.log(person1);

const ob1 = {
  name: "Mohan",
  age: 20,
  greet: function () {
  }
};
console.log(ob1);

```

CONSOLE (BROWSER)

```

Person {name: 'Rohit', age: 20}
{name: 'Mohan', age: 20, greet: f}

```

Look closely: `person1` shows **only data** (`name`, `age`) — `sayHi` is not inside it, because it lives on `Person.prototype` (expand `[[Prototype]]` in the console to see it). But `ob1` shows `greet: f` as its **own** property — every such literal carries its own copy. (Node.js prints `Person { name: 'Rohit', age: 20 }` and `greet: [Function: greet] .`)

Experiment 6 — `Object.create()` (the active code in the file)

JAVASCRIPT Day 21 · index.js — experiment 6

```

const obj = {
  name: "Rohit",
  age: 20
}

const obj2 = Object.create(obj);
obj2.account = 10;

console.log(obj2.account);

```

OUTPUT

10

`Object.create(obj)` makes a new, empty object whose `[[Prototype]]` is `obj`. We give it one own property, `account`. So `obj2.account` → `10` (own property), and — even though `obj2` looks like `{ account: 10 }` — `obj2.name` → `"Rohit"`, found through the chain.

8.10 Final summary from first principles

1. **Problem:** creating many similar objects is inefficient because you duplicate shared functions in memory.
2. **Fundamental solution:** create a link (`[[Prototype]]`) from each instance object to a shared "prototype" object.
3. **Mechanism:** when you access a property, JavaScript checks the instance first, then follows the `[[Prototype]]` links up the chain until it finds the property or the chain ends (`null`).
4. **Old implementation:** a **constructor function** with the `new` keyword, which automates creating a new object and linking its `[[Prototype]]` to the function's `.prototype` property.
5. **Modern abstraction:** the `class` keyword — a much cleaner syntax for doing the exact same thing.

8.11 Things to Remember



Things to Remember

- Every object has a hidden `[[Prototype]]` link (read it with `Object.getPrototypeOf(obj)` or `obj.__proto__`).
- Property lookup: **own property** → **prototype** → **prototype's prototype** → ... → `null` → `undefined`.
- `this` inside a method found on the prototype is still **the object before the dot**.
- `Object.create(proto)` → a new empty object linked to `proto`.
- `hasOwnProperty(key)` → `true` only for the object's **own** properties.
- `new F()` : ① create `{}` ② link it to `F.prototype` ③ run `F` with `this` = the new object ④ return it.
- `F.prototype` (on functions) ≠ `obj.[[Prototype]]` (on every object). `new` connects them.
- `class` = syntactic sugar: `typeof MyClass === "function"`; methods go on `MyClass.prototype`; must use `new`; strict mode inside.
- `extends` links the prototypes; in a child constructor call `super(...)` **before using** `this`.
- In a constructor, `this` is the new object being built; without `this.x = ...` nothing is stored on it.

 Quick Quiz

1. What is printed?

JAVASCRIPT

```
const parent = { greet() { return "hi"; } };
const child = Object.create(parent);
console.log(child.greet(), child.hasOwnProperty("greet"));
```

2. Name the four things `new` does.

3. What is `typeof class A {}`?

4. What is wrong here, and what error do you get when you run `new B()`?

JAVASCRIPT

```
class B extends A {
  constructor() {
    this.x = 1;
    super();
  }
}
```

5. Why does `console.log(person1)` in Day 21 not show `sayHi`?

6. `const arr = [1, 2]`. Is `push` an own property of `arr`? Where is it?

 Answers

1. `hi false` — `greet` is found on the prototype, not on `child` itself.

2. Create an empty object → link its `[[Prototype]]` to `Constructor.prototype` → run the constructor with `this` = the new object → return it.

3. `"function"` — a class is a special function.

4. `this` is used before `super()` →

`ReferenceError: Must call super constructor in derived class before accessing 'this'...`. Call `super()` first.

5. Because `sayHi` is stored on `Person.prototype`, not on the object itself. The object only holds its data.

6. No (`arr.hasOwnProperty("push")` is `false`). It lives on `Array.prototype`.

CHAPTER 9 Lecture 22 · Day 22 code



Strict Mode

IN THIS CHAPTER YOU WILL LEARN

- Why strict mode exists: **silent errors** → **loud errors**
- How to switch it on with `'use strict'`
- The 4 big differences (+ 2 small ones)
- Why modern code is already strict (modules, classes)

9.1 The first principle: turning silent errors into loud errors

Early versions of JavaScript were designed to be **extremely forgiving**. The philosophy was: *“Don’t stop the code from running, even if something looks wrong. Just make a guess and keep going.”* This is **non-strict mode**, often called **“sloppy mode”**.

The problem with this forgiveness is that it **hides bugs**. Your code might run, but it might be doing something completely different from what you intended. These are called **silent errors**.

Strict mode was introduced in **ES5 (2009)** as a way to opt in to a less forgiving, more secure and more sensible version of the language.

The first principle of strict mode

Its purpose is to change “sloppy” JavaScript — code with silent errors or “bad syntax” — into code that **throws immediate, obvious errors**. It helps you write better, safer code by forcing you to be more explicit.

The lenient parent vs. the strict teacher

- **Non-strict mode (the lenient parent):** you make a mess in your room (write bad code). The lenient parent just shrugs and cleans it up for you — the code runs, but maybe not how you expect. You might not even realise you made a mess.
- **Strict mode (the strict teacher):** you make a mess in your homework. The strict teacher immediately gives you an “F” and tells you **exactly** what you did wrong. It’s harsh, but you learn from your mistake and don’t make it again.

Like compiling with `-Wall -Werror`

C++ compilers can warn you about suspicious code, and `-Werror` turns those warnings into **errors** that stop the build. Strict mode is JavaScript’s version of that: code that was *silently accepted* now **fails loudly**. (And some things strict mode forbids — like using a variable you never declared — are compile errors in C++ anyway!)

9.2 How to use it

Put this string at the **very top** of a file, or at the very top of a function:

JAVASCRIPT

```
'use strict';
```

JAVASCRIPT

```
// File-level strict mode (recommended) – must be the FIRST statement of the file
'use strict';

function myStrictFunction() {
  // This whole function runs in strict mode
}
```

JAVASCRIPT

```
// Function-level strict mode
function mySloppyFunction() {
  // This function is in non-strict mode
}

function myInnerStrictFunction() {
  'use strict';
  // Only this function is in strict mode
}
```

⚠ Spelling matters — and so does position

- It must be exactly `'use strict'` (with a **space**). A version with a hyphen, `'use-strict'`, is just an ordinary string that does **nothing** — the code stays sloppy. (Watch out: one of the lecture examples has this typo.)
- It must be the **first statement** of the file or function. If any other code comes before it, it is ignored.

9.3 The core differences in detail

1. Prevents accidental global variables

This is the **biggest and most important** feature. In non-strict mode, if you assign a value to a variable that was never declared, JavaScript “helpfully” **creates a global variable** for you (on `window` in the browser). This is a huge source of bugs — for example, a simple typo in a variable name silently creates a new global.

JAVASCRIPT

```
// Non-strict mode (sloppy)
function createMistake() {
  // Forgot to use let, const or var
  mistake = "I am now a global variable!"; // no error!
}
createMistake();
console.log(window.mistake); // "I am now a global variable!"
```

JAVASCRIPT

```
// Strict mode (safe)
'use strict';
function avoidMistake() {
  mistake = "This will not work"; // loud error!
}
avoidMistake(); // ❌ ReferenceError: mistake is not defined
```

2. Changes the behaviour of `this`

In a **simple function call** (no object before the dot), `this` behaves differently. (Chapter 10 explains `this` completely.)

JAVASCRIPT

```
// Non-strict mode: this defaults to the global object (window in the browser)
function logThis() {
  console.log(this);
}
logThis(); // logs the window object
```

JAVASCRIPT

```
// Strict mode: this is undefined
'use strict';
function logThis() {
  console.log(this);
}
logThis(); // logs undefined
```

This is safer: it prevents you from accidentally reading or modifying the global object.

3. Prohibits duplicate parameter names

This is just bad practice, and strict mode forbids it.

JAVASCRIPT

```
// Non-strict mode: the last parameter wins, and there is no error
function sloppy(param1, param1) {
  console.log(param1); // the value of the SECOND param1 is used
}
sloppy(10, 20); // logs 20
```

JAVASCRIPT

```
// Strict mode: this is a SyntaxError – the code won't even run
'use strict';
function strict(param1, param1) { }
// ❌ SyntaxError: Duplicate parameter name not allowed in this context
```

4. Makes `eval()` safer

`eval()` is powerful and dangerous, because it can run any **string** as code. Strict mode puts a cage around it.

JAVASCRIPT

```
// Non-strict mode: eval() can create new variables in the surrounding scope
eval("var x = 2;");
console.log(x); // 2
```

JAVASCRIPT

```
// Strict mode: variables created inside eval() stay inside eval()'s own scope
'use strict';
eval("var x = 2;");
console.log(x); // ❌ ReferenceError: x is not defined
```

(In real code, avoid `eval` completely.)

Summary table

Feature / behaviour	Non-strict mode (sloppy) 🚩	Strict mode ✅
Undeclared variables	creates a global variable	throws a <code>ReferenceError</code>
<code>this</code> in simple calls	points to the global object (<code>window</code>)	is <code>undefined</code>
Duplicate parameters	allowed, the last one wins	<code>SyntaxError</code>
<code>eval()</code> scope	can create variables in the surrounding scope	variables stay inside <code>eval()</code>
Deleting a variable (<code>delete myVar</code>)	silently fails	<code>SyntaxError</code>
Octal literals (<code>010</code>)	<code>010</code> is 8 (surprise!)	<code>SyntaxError</code> (write <code>0o10</code> if you really mean octal)

9.4 Is this still relevant today?

Yes, more than ever! The good news is that you often get strict mode **for free**:

- JavaScript modules (files using `import` / `export`) are **always** in strict mode.
- Code inside a `class` body is **always** in strict mode.

Because modern JavaScript development (React, Node.js projects...) is almost entirely based on modules, most of the code you write is already running in strict mode by default.

Recommendation: always write in strict mode. Put `'use strict';` at the top of any standalone script file that isn't a module, to make sure you are writing safe, modern, robust code.

📁 Day 22 — strict mode in the code

The Day 22 file uses `'use strict'` many times (commented in and out) to compare behaviours. The teacher's notes at the top of the file summarise it:

JAVASCRIPT Day 22 · index.js (the teacher's notes)

```
// strict mode vs non strict mode
// 'use strict'
// normal function: non strict mode, this will point to global object
// in strict mode: It will point to undefined
```

We'll run all those experiments in the next chapter, together with the `this` keyword.

9.5 Things to Remember

📌 Things to Remember

- Strict mode turns **silent errors into loud errors**. Enable it with `'use strict';` as the **first statement** of a file or function (spelled with a space!).
- Assigning to an undeclared variable → `ReferenceError` (instead of silently creating a global).
- In a plain function call, `this` is `undefined` (instead of `window` / `global`).
- Duplicate parameter names, `delete` of a variable, and `010` octal literals → `SyntaxError`.
- `eval` can't leak variables into the surrounding scope.
- **Modules and class bodies are always strict.**

🎯 Quick Quiz

1. What happens in sloppy mode vs strict mode? `function f() { total = 5; } f();`
2. Which one enables strict mode: `'use-strict'` or `'use strict'`?
3. In strict mode, what does `function show() { return this; } show()` return?
4. Name two places where code is automatically strict.

✓ Answers

1. Sloppy: a **global variable** `total` is silently created. Strict: `ReferenceError: total is not defined`.
2. `'use strict'` (with a space). The hyphen version does nothing.
3. `undefined`.
4. ES modules (`import` / `export` files) and `class` bodies.

CHAPTER 10 Lecture 22 · Day 22 code



The this Keyword, call / apply / bind, and Arrow Functions

IN THIS CHAPTER YOU WILL LEARN

- The first principle: `this` is decided by **how** a function is **called**
- The global object: `window`, `global`, `globalThis`
- The 4 rules: **method**, **simple**, **explicit** (`call` / `apply` / `bind`), **constructor** (`new`)
- The big exception: **arrow functions** take `this` from outside
- `this` at the top level: browser vs Node.js
- The Stopwatch problem, `that = this`, `.bind(this)` and arrows
- When NOT to use arrow functions
- Every experiment from the **Day 22** code

10.1 The idea: “You are wearing a red shirt”

Imagine I write a sentence on a piece of paper: “You are wearing a red shirt.”

Who is “you”? You can’t know by just looking at the paper. The meaning of “you” depends entirely on **who I am speaking to when I say the sentence**:

- If I say it to Alice, “you” is Alice.
- If I say it to Bob, “you” is Bob.
- If I shout it into an empty room, “you” is... nobody? Or the room itself? (This is where it gets interesting.)

The sentence (the **function**) is the same. The **context of the call** (*how I say it*) determines the meaning of “you” (`this`).

In C++, `this` is simple — in JavaScript it’s flexible

In C++, `this` is a **pointer to the object the member function was called on**: `alice.speak()` → `this == &alice`; `bob.speak()` → `this == &bob`. And a member function **can’t** be called without an object.

JavaScript’s **Rule 1** (below) is exactly the same as C++. The difference: in JavaScript a function is a normal **value**. You can copy it out of an object, pass it as a callback, call it on its own, or force it onto another object. **Each way of calling gives a different `this`**. That’s why we need rules.

10.2 The first principle: the call-site

This leads us to the single most important principle for understanding `this`:

 The first principle

The value of `this` is **not** determined by **where** a function is written. It is determined by **HOW** that function is **called**.

That's it — this is the bedrock. To know what `this` is, you don't look at the function's code; you find the line where the function is being invoked (the **call-site**) and look at **how** it's being called. (Arrow functions are the one big exception — section 10.8.)

10.3 The global object: `window`, `global`, `globalThis`

Before the rules, meet the “ultimate boss” object that appears in some of them — the **global object**:

- In the **browser** it is `window` (it holds `document`, `setTimeout`, `console` ...).
- In **Node.js** it is `global`.
- `globalThis` is the modern, standard name that points to the global object in **any** environment.

 Day 22 — experiment 1: the global object

JAVASCRIPT Day 22 · index.js — experiment 1

```
var a = 10;
var b = 20;

console.log(a, b);

function greet(name1, name2) {
  console.log(name1, name2);
}

greet("Rohit", "Mohit");

document.getElementById("first");
console.log(globalThis);
```

CONSOLE (BROWSER)

```
10 20
Rohit Mohit
Window {window: Window, self: Window, document: document, name: '', ...}
```

- `globalThis` in the browser is `window`.
- `var` at the top level of a normal browser script becomes a **property of** `window`: `window.a` → `10` (`let` / `const` don't do this).
- `document.getElementById(...)` is really `window.document.getElementById(...)` — `document` lives on the global object.
- In Node.js, `console.log(globalThis)` prints the `global` object instead (and there is no `document`).

10.4 Rule 1 — the method call (the most intuitive)

When a function is called on an object, using a dot.

- **How it's called:** `object.myFunction()`
- **Intuition:** the object is the one “in charge” of the call.
- **this** is: the object to the left of the dot.

JAVASCRIPT

```
const person = {
  name: "Alice",
  speak: function () {
    // Call-site: person.speak()
    // The object to the left of the dot is 'person'. So, `this` is `person`.
    console.log(`My name is ${this.name}`);
  },
};

person.speak(); // My name is Alice
```

! Losing this by taking the method out

JAVASCRIPT

```
const speak = person.speak; // copy the function out of the object
speak();                     // ✗ no dot anymore → Rule 2 → this is undefined/window
```

The function is the same, but the **call-site** changed. This is exactly what happens when you pass `person.speak` as a **callback** (e.g. to `setTimeout`) — see the Stopwatch problem in 10.10.

10.5 Rule 2 — the simple call (the most confusing)

When a function is called **by itself**, with **no object on the left**.

- **How it's called:** `myFunction()`
- **Intuition:** no one is in charge of the call. It's just... called. This is the **default** case.
- **this** is: it depends on the **strictness** of the code (Chapter 9).

Case 2a — non-strict mode (the “sloppy” old way). If nobody is in charge, the ultimate boss — the **global object** — takes over. In a browser that's `window`; in Node.js it's `global`.

JAVASCRIPT

```
// Running in a browser, in non-strict mode
function whoAmI() {
  // Call-site: whoAmI() — no object on the left. Non-strict mode.
  // `this` defaults to the global object.
  console.log(this === window); // true
}

whoAmI();
```

Case 2b — strict mode (the modern, safe way). If nobody is in charge, then **nobody** is in charge: **this** is **undefined**. This is much safer, because it prevents you from accidentally modifying the global object — your code throws an error immediately if you try, which is a good thing!

JAVASCRIPT

```
'use strict';
function whoAmI() {
  // Call-site: whoAmI() — no object on the left. Strict mode is on.
  // `this` is undefined.
  console.log(this === undefined); // true
  // console.log(this.name);        // this would throw a clear error!
}
whoAmI();
```

 Day 22 — experiment 4: a plain function call

JAVASCRIPT Day 22 · index.js — experiment 4

```
// 'use strict'

function greet() {
  console.log(this);
}

greet();
```

Where you run it	Output
Browser, without <code>'use strict'</code>	Window {...}
Browser, with <code>'use strict'</code>	undefined
Node.js, without <code>'use strict'</code>	<ref *1> Object [global] {...} (the global object)
Node.js, with <code>'use strict'</code>	undefined

10.6 Rule 3 — the explicit call: **call**, **apply**, **bind** (the “bossy” way)

When you want to **force** a specific value for **this**, no matter how the function is called.

- **How it's called:** `myFunction.call(...)`, `myFunction.apply(...)` or `myFunction.bind(...)`
- **Intuition:** you are explicitly telling the function: “Run now, and when you do, **this** must be this specific object I'm giving you.”
- **this** is: the **first argument** you pass.

JAVASCRIPT

```
function speak() {
  console.log(`My name is ${this.name}`);
}

const person1 = { name: "Alice" };
const person2 = { name: "Bob" };

speak.call(person1); // We force `this` to be person1 → My name is Alice
speak.call(person2); // Now we force `this` to be person2 → My name is Bob
```

Why is this useful? Remember the Day 22 comment: “100 users → greet function → 100 × memory, code copy-paste”. With `call`, you write **one** function and **borrow** it for any object — no copies.

call vs **apply** vs **bind**

	Syntax	Runs the function?	Other arguments
call	<code>fn.call(obj, a, b)</code>	yes, right now	one by one, with commas
apply	<code>fn.apply(obj, [a, b])</code>	yes, right now	all together, in an array
bind	<code>fn.bind(obj, a, b)</code>	no — it returns a new function whose <code>this</code> is permanently <code>obj</code>	pre-filled, one by one

✓ **A trick to remember**

Call = Commas. Apply = Array. Bind = Book it for later (you get a new function back).

Day 22 — experiment 3: `call` , `apply` and `bind`

JAVASCRIPT Day 22 · index.js — experiment 3

```
function greet() {
  console.log(`hi ${this.name}`);
}

function incrementAge(value, name) {
  this.age += value;
  this.name = name;
  console.log(this.age);
  console.log(this.name)
}

const user = {
  name: "Rohit",
  age: 30,
}

const user2 = {
  name: "Mohit",
  age: 10
}

greet.call(user);           // hi Rohit
greet.call(user2);         // hi Mohit

incrementAge.call(user2, 10, "Mohan"); // 20, Mohan
// incrementAge.apply(user2, [10, "Mohan"]); // same, arguments in an array
// const incr = incrementAge.bind(user2, 10, "Mohan");
// incr(); // same, but called later
```

- `greet.call(user)` → inside `greet` , `this` is `user` → prints `hi Rohit` . Then `this` is `user2` → `hi Mohit` . **One** function, used by **two** objects.
- `incrementAge.call(user2, 10, "Mohan")` → `this` is `user2` ; `value` is `10` , `name` is `"Mohan"` . So `user2.age` becomes `10 + 10 = 20` and `user2.name` becomes `"Mohan"` → prints `20` and `Mohan` .
- `apply` does the same, but the arguments go in an **array**: `[10, "Mohan"]` .
- `bind` **doesn't run anything** — it returns a new function `incr` with `this` locked to `user2` and the arguments pre-filled. Calling `incr()` later prints `20` and `Mohan` .

(In the file the teacher runs only one of them at a time. If you ran `call` , then `apply` , then `bind` + `incr()` one after another on the same `user2` , you'd see the age grow: 20, 30, 40.)

10.7 Rule 4 — the constructor call with `new` (the "factory" way)

When you call a function using the `new` keyword.

- **How it's called:** `new MyFunction()` (or `new MyClass()`)
- **Intuition:** `new` is a special instruction to **build a brand-new object**.

- **this** is: the brand-new, empty object that was just created. The constructor's job is to populate it (Chapter 8).

JAVASCRIPT

```
class Person {
  constructor(name) {
    // Call-site: new Person(...)
    // `new` creates an empty object and makes `this` point to it.
    this.name = name; // populating the new object
  }
}

const person1 = new Person("Alice");
// `this` inside the constructor was the object that became person1
console.log(person1); // { name: 'Alice' } (the console shows: Person {name: 'Alice'})
```

Day 22 — experiment 5: **this** in a class constructor

JAVASCRIPT Day 22 · index.js — experiment 5

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

// this = { name: "Rohit", age: 20 } ← the teacher's comment: the new object

const p1 = new Person("Rohit", 20);

console.log(p1);
```

CONSOLE (BROWSER)

```
Person {name: 'Rohit', age: 20}
```

1. Method call

```
obj.fn()
this = obj
```

2. Simple call

```
fn()
strict: undefined
sloppy: window
```

3. Explicit

```
fn.call(x)
this = x
(also apply, bind)
```

4. Constructor

```
new Fn()
this = the new
object
```

The exception: arrow functions `() => { ... }`

have NO own this — they use the this of the place where they are WRITTEN

(the four rules above don't apply to them; call / apply / bind can't change it)

Figure 10.1 The four ways to call a regular function — and the arrow-function exception.

10.8 The big exception — arrow functions

Arrow functions (`=>`) are special. They were designed to solve common frustrations with `this`, and they **break the first principle on purpose**:

Arrow functions and `this`

Arrow functions **do not have their own** `this`. They inherit `this` from their parent scope, just like any other variable. They don't care **how** they are called — they only care **where they are written**. This is called **lexical** `this`.

The loyal assistant

An arrow function is like a **loyal assistant**. If you ask the assistant “Who is your boss?”, they don't say “Me!”. They point to **their boss** — the `this` of the function they were written inside.

JAVASCRIPT

```
const user = {
  name: "Alice",
  hobbies: ['reading', 'coding'],
  printHobbies: function () {
    // For this regular function, `this` is `user` (Rule 1).
    this.hobbies.forEach(hobby => {
      // This is an arrow function. It doesn't have its own `this`.
      // It inherits `this` from its parent, printHobbies. So `this` here is also `user`.
      console.log(`${this.name} likes ${hobby}`);
    });
  },
};

user.printHobbies();
```

OUTPUT

```
Alice likes reading
Alice likes coding
```

Because arrow functions ignore the call-site, `call`, `apply` and `bind` **cannot change** an arrow function's `this` either.

10.9 `this` outside of any function (the global context)

What is `this` when it's not inside any function at all?

Intuition: if you're not in any specific room (a function), you must be in the main building (the **global scope**). So `this` refers to the “main building” object.

- In a browser (a regular, non-module `<script>` tag): the main building is the `window` object. `console.log(this === window) → true`. This is true **even with** `'use strict'` — strict mode only changes `this` inside plain function calls.
- In Node.js — the tricky one. **Every file in Node.js is its own module**. Node wraps your file's code in a function, and because of how Node calls that function, `this` at the top level is `module.exports`, which is an **empty object** `{}` by default — **not** the `global` object. (This is for normal `.js` files. In ES-module files — `.mjs`, or `"type": "module"` — top-level `this` is `undefined`.)

JAVASCRIPT

```
// In a Node.js file
console.log(this === global); // false
console.log(this);           // {}
console.log(this === module.exports); // true
```

📅 Day 22 — experiments 2, 6 and 10: `this` at the top level, and a top-level arrow

JAVASCRIPT Day 22 · index.js — experiments 2 / 6 / 10

```
// 'use strict'

console.log(this);

const greet = () => {
  console.log(this);
}

greet();
```

	Browser	Node.js
<code>console.log(this)</code> at the top	<code>Window {...}</code>	<code>{}</code>
... with <code>'use strict'</code>	still <code>Window {...}</code>	still <code>{}</code>
<code>greet()</code> (arrow, written at the top level)	<code>Window {...}</code> — it copies the top-level <code>this</code>	<code>{}</code>

The teacher's summary comment says exactly this: *"this keyword in global scope: NodeJS (Empty Object), in browser it will point to global Object"* and *"arrow function take this keyword from its lexical environment scope"*.

10.10 Arrow functions in detail: the problem they were born to solve

Part 1 — the classic problem

Before arrow functions, a very common and frustrating bug appeared in JavaScript. To understand the genius of arrow functions, you must first **feel the pain** of the problem they solve. Let's build a simple

`Stopwatch` object:

JAVASCRIPT

```
// The Old, Painful Way
function Stopwatch() {
  this.seconds = 0;
  this.start = function () {
    // We want to increment `this.seconds` every second.
    setInterval(function () {
      // Uh oh. We have a problem here.
      // What is `this` inside THIS function?
      console.log(this.seconds);
      this.seconds++;
    }, 1000);
  };
}

const myWatch = new Stopwatch();
myWatch.start();
```

When you run `myWatch.start()`, you don't get 0, 1, 2... Instead the first tick prints `undefined`, and after that `NaN` (Not a Number) every second.

Why did this fail? Apply the first principle — the value of `this` is determined by how the function is called:

1. When we call `myWatch.start()`, the `this` inside `start` is correctly `myWatch` (Rule 1: method call).
2. But the function we pass to `setInterval` is a **regular function**, and `setInterval` calls it later **without** `myWatch.` in front of it — so it is **not** a method call on `myWatch`.
3. So `this` inside the callback is **not** `myWatch`. The browser calls timer callbacks with `this` set to the global object `window` — and it does this even in strict mode, because the timer passes `window` on purpose. (In Node.js, `this` there is a `Timeout` object.)
4. So inside our callback we have **lost the context of** `myWatch`. We are doing `window.seconds++` → `undefined + 1` → `NaN`.

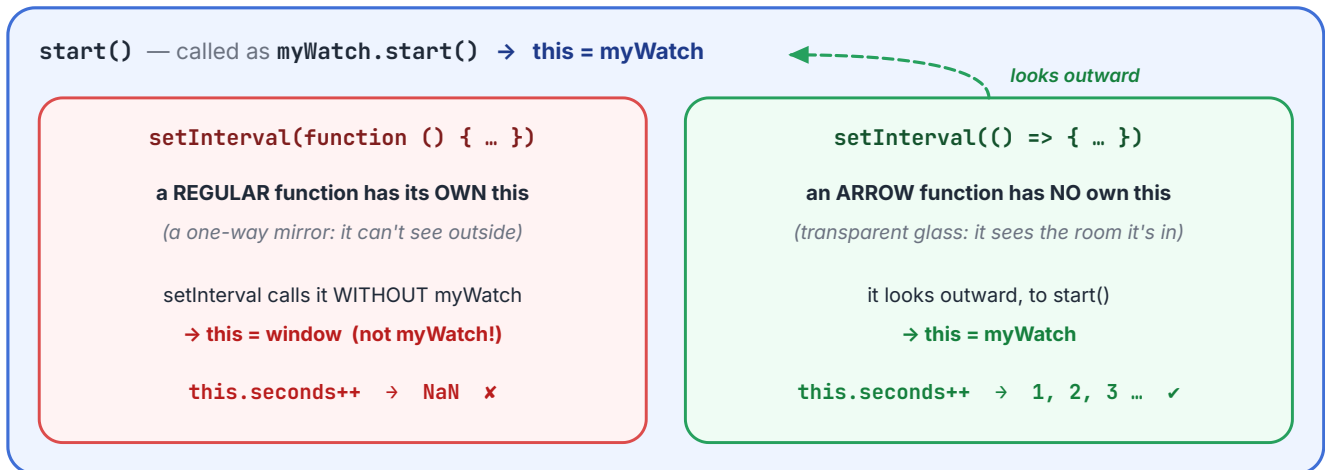


Figure 10.2 A regular function creates its own `this` (a one-way mirror). An arrow function is transparent glass: it sees the `this` of the room it was written in.

The old, clumsy workarounds

Workaround 1: `that = this` (the closure trick). Save the correct `this` in a normal variable **before** you lose it:

JAVASCRIPT

```
function Stopwatch() {
  this.seconds = 0;
  const that = this; // capture the correct `this`
  this.start = function () {
    setInterval(function () {
      // Use the captured variable instead of the wrong `this`
      console.log(that.seconds);
      that.seconds++;
    }, 1000);
  };
}
```

Workaround 2: `.bind(this)` (the explicit way). Create a new function whose `this` is permanently locked to the correct value:

JAVASCRIPT

```
function Stopwatch() {
  this.seconds = 0;
  this.start = function () {
    setInterval(function () {
      console.log(this.seconds);
      this.seconds++;
    }.bind(this), 1000); // bind the callback to the correct `this`
  };
}
```

Both of these work, but they are verbose and add extra noise to the code. We are fighting against the language's own rules.

Part 2 — the solution: the arrow function

The designers of ES6 saw this common problem and created a new kind of function with one special, game-changing rule: **an arrow function does not have its own `this` ; it lexically inherits `this` from its parent scope.**

- “Does not have its own `this` ”: it completely ignores the four rules of `this` (method call, simple call, etc.). They do not apply to it.
- “Lexically inherits”: this is the key. It doesn’t care how it’s **called**. It only cares where it is physically **written** in the code. It looks outside itself to the surrounding code block and uses whatever `this` is in there.

💡 One-way mirror vs. transparent glass

Think of a **regular function** as a **one-way mirror**. You can’t see the `this` of the room outside — it creates its own reflection, its own `this`.

An **arrow function** is like a sheet of **perfectly transparent glass**. It doesn’t have its own reflection. It just lets you see the `this` of the room it’s in.

JAVASCRIPT

```
function Stopwatch() {
  this.seconds = 0;
  this.start = function () {
    // The `this` in this scope correctly points to the stopwatch instance.
    setInterval(() => {
      // This is an arrow function. It has no `this` of its own.
      // It inherits `this` from its parent, the `start` function.
      // Therefore `this` here is the stopwatch instance. It just works.
      console.log(this.seconds);
      this.seconds++;
    }, 1000);
  };
}

const myWatch = new Stopwatch();
myWatch.start(); // 0, 1, 2, 3 ... - this works perfectly!
```

The code is clean, intuitive, and does exactly what we want without any workarounds. The context is preserved automatically.

Day 22 — experiments 7, 8 and 9: `that = this` and the arrow stopwatch

JAVASCRIPT Day 22 · index.js — experiment 7: an inner function loses this

```
const user = {
  name: "Rohit",
  greet: function () {
    // console.log(this);           // here `this` is user (method call)
    const that = this;           // save it!
    function meet() {
      console.log(that);          // inner function: use `that`, not `this`
    }
    meet();                       // plain call: inside meet, `this` is NOT user
  }
}

user.greet();                     // {name: 'Rohit', greet: f}
```

JAVASCRIPT Day 22 · index.js — experiment 8: stopwatch with that = this

```
const stopWatch = {
  second: 0,
  start: function () {
    const that = this;
    setInterval(function () {
      that.second++;
      console.log(that.second);
    }, 1000);
  }
}
```

JAVASCRIPT Day 22 · index.js — experiment 9: stopwatch with an arrow function

```
const stopWatch = {
  second: 0,
  start: function () {
    console.log(this);           // {second: 0, start: f}
    setInterval(() => {
      this.second++;             // arrow → `this` from start() → stopWatch
      console.log(this.second);
    }, 1000)
  }
};

stopWatch.start();
```

OUTPUT OF EXPERIMENT 9 (EXPERIMENT 8 PRINTS THE SAME NUMBERS, WITHOUT THE FIRST LINE)

```
{second: 0, start: f}
1 ← after 1 s
2 ← after 2 s
3 ← after 3 s ... and so on, forever
```

If you wrote experiment 8 with `this` instead of `that` inside the regular callback, `this` would be `window` → `this.second++` makes `NaN`, and you would see `NaN` every second.

Part 3 — the consequences: when NOT to use arrow functions

Because arrow functions **always** inherit `this`, they are the **wrong tool** when you **want** your function to have its own `this` based on how it's called.

1. Object methods (the most common mistake). If you define a method on an object using an arrow function, it inherits `this` from the **outside** (the global scope), which is not what you want:

JAVASCRIPT

```
const person = {
  name: "Alice",

  // WRONG WAY — using an arrow function for a method
  sayHi: () => {
    // This `this` is inherited from the global scope (where the object is written).
    // In Node it's module.exports ({}). In a browser it's `window`.
    console.log(`Hi, my name is ${this.name}`);
  },

  // CORRECT WAY — using a regular function
  speak: function () {
    // This `this` is decided by the call-site (person.speak()), so `this` is person.
    console.log(`Hi, my name is ${this.name}`);
  },
};

person.sayHi(); // Hi, my name is undefined (Node.js)
person.speak(); // Hi, my name is Alice
```



Why might the browser show an empty name?

In a browser, `person.sayHi()` prints `Hi, my name is` with an **empty** name, not `undefined`. That's because `window` happens to have a built-in property called `name` whose value is `""`. Either way, it is **not** `"Alice"` — the arrow method did not get `person` as `this`.

2. Event listeners in HTML. When you add an event listener, you often want `this` to refer to the **element that was clicked**. A regular function does this automatically (the browser calls it with `this` = the element). An arrow function does not.

JAVASCRIPT

```
const button = document.getElementById('myButton');

// CORRECT WAY - regular function: `this` is the button element itself
button.addEventListener('click', function () {
  console.log(this);           // <button id="myButton">...</button>
  this.textContent = 'Clicked!';
});

// WRONG WAY - arrow function: `this` comes from the outside (window)
button.addEventListener('click', () => {
  console.log(this);           // Window {...}
  // this.textContent = 'Clicked!'; // would NOT change the button
});
```

 Day 22 — experiments 11 and 12: arrow method and arrow listener**JAVASCRIPT** Day 22 · index.js — experiment 11

```
// 'use strict'
console.log(this);

const user = {
  name: "Rohit",
  greet: () => {
    console.log(this);
  }
}

user.greet();
```

Browser: both lines print `Window {...}` (Node.js: both print `{}`). The dot in `user.greet()` doesn't matter — **arrow functions ignore the call-site**.

JAVASCRIPT Day 22 · index.js — experiment 12

```
const button = document.getElementById("first");

button.addEventListener('click', () => {
  console.log(this);
})
```

Clicking the red square button prints `Window {...}`, not the button — because the arrow function took `this` from the top level of the script. Change it to `function () { console.log(this); }` and it prints the **button element**.

10.11 The final intuitive summary — how to find `this`

Ask these questions in **this order** (the order matters when several apply):

1. Is it an arrow function? → `this` is whatever `this` was in the **parent scope** (where it was written).
Stop here.
2. Is the `new` keyword used? (`new Person()`) → `this` is the **brand-new object** being created.
3. Is `.call()`, `.apply()` or `.bind()` used? (`speak.call(person)`) → `this` is the **object passed as the first argument**.
4. Is the function called with a dot? (`person.speak()`) → `this` is the **object to the left of the dot**.
5. None of the above — a simple call like `speak()` :
 - in `'use strict'` mode → `this` is **undefined** ;
 - in sloppy mode → `this` is the **global object** (`window` in the browser, `global` in Node.js).

And **outside any function**: browser → `window` ; Node.js → `{}` (`module.exports`).

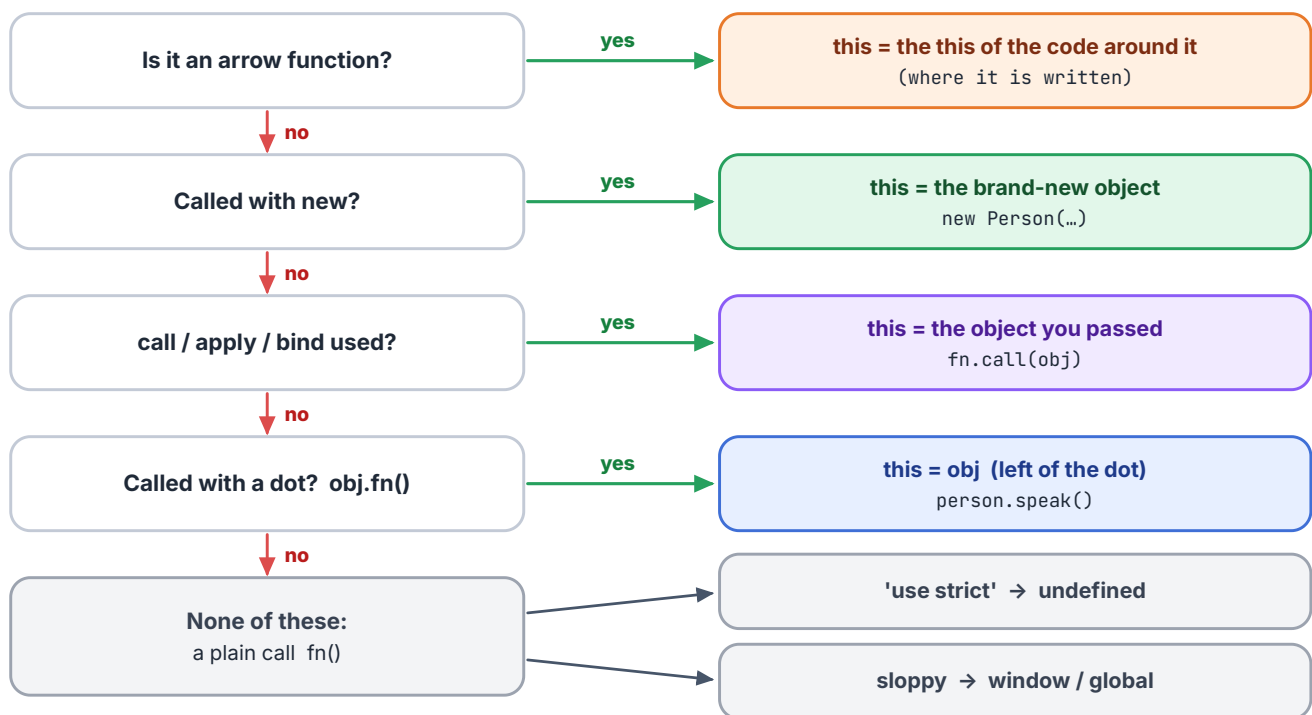


Figure 10.3 Follow the questions from top to bottom. The first “yes” gives you the answer.

10.12 All Day 22 experiments at a glance

Experiment	Code	Browser	Node.js
Global object	<code>globalThis</code>	<code>Window {...}</code>	<code>Object [global] {...}</code>
<code>var</code> at the top	<code>var a = 10</code>	<code>window.a</code> is 10	not on <code>global</code>
Top-level <code>this</code>	<code>console.log(this)</code>	<code>Window {...}</code> (strict too)	<code>{}</code>
Plain call, sloppy	<code>greet()</code>	<code>Window {...}</code>	the global object

Experiment	Code	Browser	Node.js
Plain call, strict	<code>greet()</code>	<code>undefined</code>	<code>undefined</code>
Explicit	<code>greet.call(user)</code>	<code>hi Rohit</code>	<code>hi Rohit</code>
call / apply / bind	<code>incrementAge.call(...)</code>	<code>20, Mohan</code>	<code>20, Mohan</code>
Constructor	<code>new Person(...)</code>	<code>Person {name: 'Rohit', age: 20}</code>	same object
Arrow at the top	<code>greet()</code> (arrow)	<code>Window {...}</code>	<code>{}</code>
Inner fn + <code>that</code>	<code>user.greet()</code>	the <code>user</code> object	the <code>user</code> object
Stopwatch, <code>that</code>	<code>stopWatch.start()</code>	<code>1, 2, 3...</code>	<code>1, 2, 3...</code>
Stopwatch, arrow	<code>stopWatch.start()</code>	<code>1, 2, 3...</code>	<code>1, 2, 3...</code>
Arrow as a method	<code>user.greet()</code>	<code>Window {...}</code>	<code>{}</code>
Arrow listener	click the button	<code>Window {...}</code>	(no buttons)

10.13 Things to Remember



Things to Remember

- For **regular functions**, `this` is decided by **how the function is called** (the call-site), not where it is written.
- Method call** `obj.fn()` → `obj`. **Simple call** `fn()` → `undefined` (strict) / global object (sloppy).
Explicit `fn.call(x)` / `fn.apply(x)` / `fn.bind(x)` → `x`. **Constructor** `new Fn()` → the new object.
- `call(obj, a, b)` runs now with commas; `apply(obj, [a, b])` runs now with an array; `bind(obj, a, b)` returns a **new function** for later.
- Arrow functions have no own `this`** — they use the `this` of the place where they are **written**. `call` / `apply` / `bind` / dot can't change it.
- Use **arrows for callbacks inside methods** (`setInterval` , `forEach`) to keep `this`. **Don't** use arrows for **object methods** or when you need `this` to be the **clicked element**.
- Top-level `this`: browser → `window`; Node.js → `{}` (`module.exports`). `globalThis` always means the global object.
- Old fixes for lost `this`: `const that = this;` or `.bind(this)`. Modern fix: an arrow function.

 Quick Quiz

1. What is printed (in a browser, sloppy mode)?

JAVASCRIPT

```
const obj = { name: "X", show() { console.log(this.name); } };
const f = obj.show;
obj.show();
f();
```

2. What does `fn.bind(obj)` return?

3. What is printed?

JAVASCRIPT

```
function add(a, b) { return this.base + a + b; }
const o = { base: 100 };
console.log(add.call(o, 1, 2), add.apply(o, [3, 4]));
```

4. Inside `setInterval(function () { ... }, 1000)` written in a method, what is `this` (browser, sloppy mode)? How do you fix it?

5. What is `this` at the top level of a Node.js file?

6. Why is `sayHi: () => console.log(this.name)` a bad way to write an object method?

 Answers

- `X`, then an empty line — `f()` is a plain call, so `this` is `window`, and `window.name` is `""`. (In strict mode `f()` would throw a `TypeError`, because `this` is `undefined`.)
- A new function whose `this` is permanently `obj` (it doesn't run anything by itself).
- `103 107`
- The global object `window` — the timer calls your callback without your object, and browsers pass `window` as `this`. Fix: use an arrow function `setInterval(() => { ... })`, or `const that = this`, or `.bind(this)`.
- `{}` — `module.exports`, not `global`.
- Arrow functions don't get `this` from the call-site, so `this` is the outer (global) `this`, not the object.

CHAPTER 11 All lectures · All Day code



Final Revision: Everything on a Few Pages

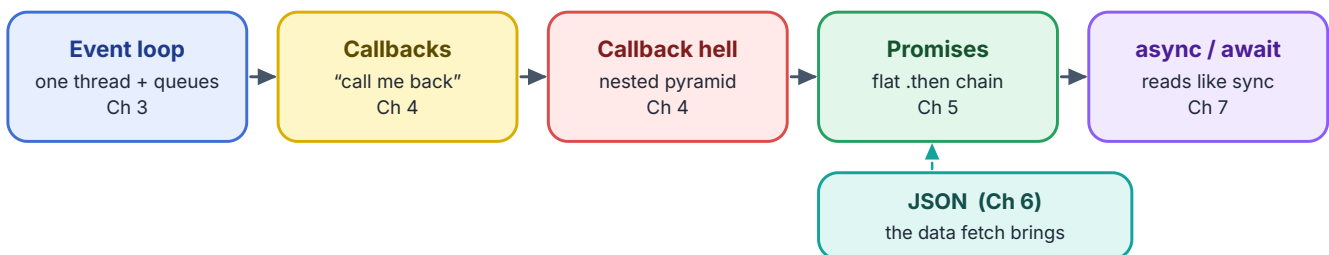
IN THIS CHAPTER YOU WILL LEARN

- How all the topics connect (one big picture)
- A cheat sheet for every chapter
- Browser vs Node.js differences in one table
- 30 rapid-fire questions with short answers
- 8 mixed “predict the output” puzzles

Use this chapter the night before an exam or an interview. If any line here feels unclear, go back to the chapter mentioned in brackets.

11.1 The big picture

Story 1 — how JavaScript handles waiting



Story 2 — how objects share behaviour, and how this finds the object

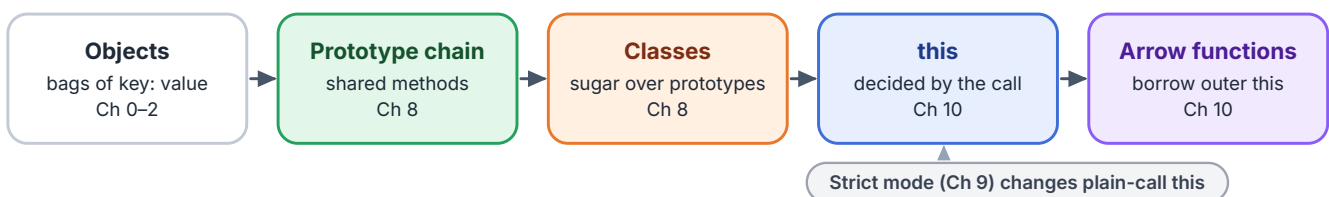


Figure 11.1 The two big stories of these notes. Top: how JavaScript handles waiting. Bottom: how objects share behaviour and how `this` finds the right object.

11.2 Cheat sheets

Arrays, Set and Map (Chapters 1–2)

You want to...	Use	Returns
do something with each item	<code>arr.forEach(fn)</code>	<code>undefined</code>
transform every item	<code>arr.map(fn)</code>	new array, same length
keep only some items	<code>arr.filter(fn)</code>	new array, same or shorter
combine into one value	<code>arr.reduce(fn, initial)</code>	anything
get the first match	<code>arr.find(fn)</code>	the element or <code>undefined</code>
check "at least one" / "all"	<code>arr.some(fn)</code> / <code>arr.every(fn)</code>	<code>true</code> / <code>false</code>
remove duplicates	<code>[...new Set(arr)]</code>	new array
fast "have I seen it?"	<code>set.has(x)</code>	<code>true</code> / <code>false</code>
keys of any type	<code>new Map()</code> + <code>set</code> / <code>get</code> / <code>has</code>	—

The event loop (Chapter 3)

The order of execution

1. All **synchronous** code (the call stack runs your file top to bottom).
2. All **microtasks**: `.then` / `.catch` / `.finally` callbacks, code after `await`.
3. One **macrotask**: a `setTimeout` / `setInterval` callback, a click handler...
4. Back to step 2. Forever.

Async patterns (Chapters 4, 5, 7)

	Callback	Promise	<code>async</code> / <code>await</code>
Success	call <code>cb(data)</code>	<code>resolve(data)</code> , read it in <code>.then</code>	<code>const data = await p;</code>
Failure	extra error parameter	<code>reject(err)</code> , read it in <code>.catch</code>	<code>try</code> / <code>catch</code>
Cleanup	manual	<code>.finally(...)</code>	<code>finally { ... }</code>
Many at once	messy flags	<code>Promise.all([...])</code>	<code>await Promise.all([...])</code>

JSON (Chapter 6)

`JSON.stringify(obj)` → text · `JSON.parse(text)` → object · keys and strings in **double quotes** · no functions, no `undefined`, no comments, no trailing commas · `await response.json()` parses a `fetch` body.

Prototypes and classes (Chapter 8)

- Lookup: own property → `[[Prototype]]` → ... → `Object.prototype` → `null`.
- `new F()` = create `{}` → link to `F.prototype` → run `F` with `this` = new object → return it.
- `class` = sugar over constructor functions; methods live on `Class.prototype`; `extends` + `super(...)` (before `this`!).

this (Chapters 9–10)

How the function is called	this is
arrow function	the <code>this</code> of the surrounding code (where it is written)
<code>new Fn()</code>	the new object
<code>fn.call(x)</code> / <code>fn.apply(x)</code> / <code>fn.bind(x)</code>	<code>x</code>
<code>obj.fn()</code>	<code>obj</code>
plain <code>fn()</code>	<code>undefined</code> in strict mode, the global object in sloppy mode
event listener with a regular function	the element (<code>button</code>)

11.3 Browser vs Node.js — the differences you met in these notes

Thing	Browser	Node.js
Global object	<code>window</code>	<code>global</code>
<code>globalThis</code>	<code>window</code>	<code>global</code>
Top-level <code>this</code>	<code>window</code>	<code>{}</code> (<code>module.exports</code>)
Top-level <code>var x</code>	becomes <code>window.x</code>	stays local to the file
<code>setTimeout(...)</code> returns	a number (timer ID)	a <code>Timeout</code> object
<code>console.log(promise)</code>	<code>Promise {<pending>}</code>	<code>Promise { <pending> }</code>
<code>document</code> , buttons, <code>localStorage</code>	✓	✗
Files (<code>fs.readFile</code>)	✗	✓
Arrow method <code>this.name</code>	<code>""</code> (because <code>window.name</code> exists)	<code>undefined</code>

11.4 Rapid-fire questions

1. What does single-threaded mean? One call stack; only one piece of your code runs at a time. (Ch 3)
2. Who counts the time for `setTimeout`? The browser (a Web API) — not the JavaScript thread. (Ch 3)

3. **Microtask vs macrotask?** Microtasks = Promise callbacks and code after `await` — all of them run first. Macrotasks = `setTimeout`, `setInterval`, events — one at a time, after the microtasks. (Ch 3)
4. **Why does `Promise.resolve().then(f)` run before `setTimeout(g, 0)`?** Microtasks run before macrotasks. (Ch 3)
5. **Is the `setTimeout` delay exact?** No — it is a *minimum*; the callback also waits for an empty stack. (Ch 3)
6. **What is a callback?** A function passed to another function to be called now or later. (Ch 0, 4)
7. **What is callback hell?** Deeply nested callbacks for sequential async steps: hard to read, change, debug; errors must be handled at every level. (Ch 4)
8. **What are the states of a Promise?** Pending → fulfilled or rejected (settled, final). (Ch 5)
9. **When does the executor of `new Promise` run?** Immediately, synchronously. (Ch 5)
10. **What does `.then()` return?** A new promise — that's why chaining works. (Ch 5)
11. **Where does an error in a promise chain go?** It skips the `.then`s and goes to the nearest `.catch`. (Ch 5)
12. **What does `.finally()` receive?** Nothing. It runs in both cases and passes the value through. (Ch 5)
13. **Does `fetch` reject on 404?** No — check `response.ok`. It rejects only on network failure. (Ch 5, 7)
14. **Why is `response.json()` a Promise?** Reading and parsing the body takes time. (Ch 5, 6)
15. **What does an `async` function return?** Always a Promise. (Ch 7)
16. **What does `await` do?** Pauses **this function** until the promise settles; gives the value or throws the error. (Ch 7)
17. **Where can you use `await`?** Inside `async` functions (or at the top level of modules). (Ch 7)
18. **How do you run independent requests in parallel?** `await Promise.all([p1, p2])`. (Ch 7)
19. **JSON vs a JS object?** JSON is text with strict rules; an object is data in memory. (Ch 6)
20. **What does `JSON.stringify` drop?** Functions and `undefined` properties. (Ch 6)
21. **What is `[[Prototype]]`?** The hidden link from an object to its parent object, used for property lookup. (Ch 8)
22. **`obj.__proto__` vs `Fn.prototype`?** The first is the link on every object; the second is an object on a function that becomes the `[[Prototype]]` of instances created with `new`. (Ch 8)
23. **Is `class` a new object model?** No — syntactic sugar over prototypes.
`typeof MyClass === "function"`. (Ch 8)
24. **Why must `super()` come before `this` in a child constructor?** The parent part of the object must be built first (like C++ base-class construction). (Ch 8)
25. **What does `'use strict'` change?** Undeclared assignment → error; `this` in plain calls → `undefined`; duplicate params, `delete x`, `010` → errors. (Ch 9)
26. **How is `this` decided for a regular function?** By **how it is called** (the call-site). (Ch 10)
27. **And for an arrow function?** From the surrounding code where it is written (lexical `this`). (Ch 10)
28. **`call` vs `apply` vs `bind`?** `call` = run now, args with commas; `apply` = run now, args in an array; `bind` = return a new function for later. (Ch 10)
29. **`map` vs `forEach`?** `map` returns a new array; `forEach` returns `undefined`. (Ch 1)

30. **Set** vs **Array**, **Map** vs **Object**? Set = unique values, fast **has**; Map = any type of key, **.size**, keeps order. (Ch 2)

11.5 Mixed practice — predict the output

Try each one on paper first. Answers are in the box at the end.

Puzzle 1 — event loop + async

JAVASCRIPT

```
console.log("A");
setTimeout(() => console.log("B"), 0);
(async () => {
  console.log("C");
  await null;
  console.log("D");
})();
Promise.resolve().then(() => console.log("E"));
console.log("F");
```

Puzzle 2 — **this** and chaining

JAVASCRIPT

```
const counter = {
  count: 0,
  inc() {
    this.count++;
    return this;    // return the object itself
  },
};
counter.inc().inc().inc();
console.log(counter.count);
```

Puzzle 3 — array methods

JAVASCRIPT

```
const nums = [1, 2, 3, 4, 5];
const result = nums
  .filter(n => n % 2)    // hint: 0 is falsy
  .map(n => n * n)
  .reduce((a, b) => a + b, 0);
console.log(result);
```

Puzzle 4 — classes and overriding

JAVASCRIPT

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal {
  speak() { return `${this.name} barks`; }
}
const d = new Dog("Tommy");
console.log(d.speak());
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype);
```

Puzzle 5 — regular vs arrow method (in Node.js)

JAVASCRIPT

```
const obj = {
  name: "JS",
  regular() { return this.name; },
  arrow: () => this.name,
};
console.log(obj.regular());
console.log(obj.arrow());
```

Puzzle 6 — promise chain with recovery

JAVASCRIPT

```
Promise.reject("oops")
  .then(() => console.log("then 1"))
  .catch(e => { console.log("caught", e); return "recovered"; })
  .then(v => console.log("then 2", v))
  .finally(() => console.log("finally"));
```

Puzzle 7 — Map and Set

JAVASCRIPT

```
const m = new Map();
m.set("a", 1).set("b", 2).set("a", 3);
console.log(m.size, m.get("a"), [...new Set([3, 1, 3, 2, 1])]);
```

Puzzle 8 — JSON

JAVASCRIPT

```
console.log(JSON.stringify({ a: [1, 2], b: null, c: undefined, d: "x" }));
```

✓ Answers (all verified by running the code)

1. **A**, **C**, **F**, **D**, **E**, **B** — sync first (the async function runs until **await**), then microtasks in the order they were queued (**D**, then **E**), then the timer.

2. `3` — each `inc()` returns the object, so the next `.inc()` is again a method call on `counter`.
3. `35` — odd numbers `[1, 3, 5]` → squares `[1, 9, 25]` → sum 35.
4. `Tommy barks` and `true` — `speak` is found first on `Dog.prototype` (it **overrides** the parent's, like a C++ virtual function); `Dog.prototype` is linked to `Animal.prototype`. `Dog` has no constructor, so JavaScript adds a default one that calls `super(...)` for you.
5. `JS`, then `undefined` — the arrow function takes `this` from the file's top level (`{}` in Node.js). In a browser the second line would be an empty string.
6. `caught oops`, `then 2 recovered`, `finally` — the first `.then` is skipped; `.catch` returns a value, so the chain continues.
7. `2 3 [ 3, 1, 2 ]` — setting `"a"` again **updates** it; the Set keeps the first occurrence of each value in order.
8. `{"a":[1,2],"b":null,"d":"x"}` — `null` is kept, the `undefined` property is dropped.

11.6 A last word from your teacher

You now understand the parts of JavaScript that confuse most beginners: **how asynchronous code really runs**, **how objects share methods**, and **how `this` is decided**. The best way to make it permanent is to **type the examples yourself** — change a line, predict the output, run it, and see if you were right. Every wrong prediction teaches you something.

All the best for the next part of the course (React!) — everything you learned here (`map`, `filter`, promises, `async` / `await`, classes, arrow functions and `this`) is used there every day. 🚀